

Fiches de Cours

Master IMA

Axel PIGEON

axel.pigeon@univ-tlse3.fr

Université Toulouse III – Paul Sabatier

Années universitaires 2025–2027

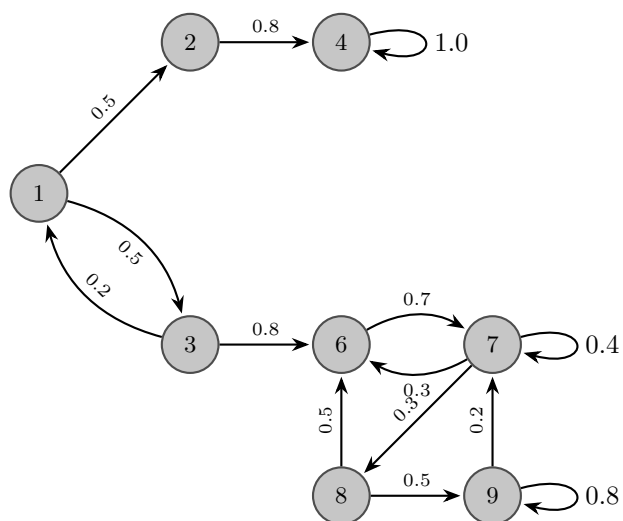
28 octobre 2025

Table des matières

I Probabilités	4
1 Chaînes de Markov	5
1.1 Définitions et Généralités	5
1.2 Propriétés de Markov	10
1.3 Mesures Invariantes	13
1.4 Classification des états	20
1.5 Mesures invariantes dans le cas dénombrable	23
II Simulation Aléatoire	27
1 Simulation de variables aléatoires	28
1.1 Simulation de variables aléatoires discrètes	29
1.2 Simulation de variables aléatoires à densité	32
1.3 Méthodes ad-hoc	36
2 Simulation de chaînes de Markov	43
2.1 Généralités	43
2.2 Comportement asymptotique d'une chaîne de Markov	46
2.3 Méthode MCMC	48
3 Processus de Markov en temps continu	52
3.1 Lois exponentielles et lemme des réveils	53
3.2 Processus de Markov en temps continu	56
III Optimisation	61
1 Optimisation sans contraintes	62
1.1 CNS d'optimalité sans contraintes	63
1.2 Algorithmes de Descente	64
1.3 Problèmes Quadratiques	70
1.4 Récapitulatif	73
IV Algorithmique	74
1 Flot maximal dans un graphe	75
1.1 Rappels sur les graphes	75
1.2 Cycle et flot	77
1.3 Flot et coupure	78

<i>TABLE DES MATIÈRES</i>	3
1.4 Flot maximal : principe de Ford-Fulkerson	80
2 Programmation Linéaire	82
2.1 Programmation Linéaire	82
2.2 Programmation Linéaire en Nombres Entiers	91
V Compilation	95
1 Analyse Syntaxique	96
1.1 Analyse Syntaxique et approche descendante (Grammaires $LL(k)$)	96
1.2 Analyse Syntaxique et approche ascendante (Grammaires $LR(k)$)	100

Probabilités



Chapitre 1

Chaînes de Markov

Contents

1.1	Définitions et Généralités	5
1.2	Propriétés de Markov	10
1.2.1	Propriété de Markov Faible	10
1.2.2	Temps d'arrêt et propriété de Markov Forte	11
1.3	Mesures Invariantes	13
1.3.1	Définition et propriétés	13
1.3.2	Existence des mesures invariantes	14
1.3.3	Unicité des mesures de probabilité invariantes et irréductibilité	15
1.3.4	Construction de mesures de probabilité invariantes	17
1.3.5	Réversibilité	19
1.4	Classification des états	20
1.4.1	Nombre de visites en un point	20
1.4.2	État transiant et état récurrent	21
1.5	Mesures invariantes dans le cas dénombrable	23
1.5.1	États récurrents positif ou nul	23
1.5.2	Propriétés des chaînes irréductibles	23
1.5.3	Critère de récurrence positive	25

Jusqu'à maintenant, nous avons étudié des suites de variables aléatoires dites iid (indépendantes et identiquement distribuées). Ces propriétés nous ont permis de développer beaucoup d'outils pour les étudier tels que la loi forte des grands nombres ou le théorème central limite.

Or nous n'utilisons pas beaucoup ce type de suite de variables aléatoires en simulation. En effet, ces propriétés plutôt fortes ne permettent pas de modéliser des phénomènes dont l'évolution est plus complexe. Nous allons donc continuer à étudier des suites de variables aléatoires mais en rajoutant des relations de dépendance entre celles-ci. Cela nous permettra de modéliser des systèmes dynamiques plus complexes.

1.1 Définitions et Généralités

On se place ici dans un espace probabilisé $(E, \mathcal{F}, \mathbb{P})$ et E un *espace d'état* fini ou dénombrable. On s'intéresse à une suite de variables aléatoires $(X_n)_{n \in \mathbb{N}}$ ou X_k est l'état du système au temps k .

Définition (Trajectoire) . On appelle *trajectoire du système* jusqu'au temps $n \in \mathbb{N}^*$ la donnée $(X_1, \dots, X_n)$.

Définition (Chaîne de Markov) . On dit qu'une suite $(X_n)_{n \geq 0}$ est une *chaîne de Markov* si elle vérifie la *propriété de Markov* :

$$\forall n \in \mathbb{N}, \forall i_0, i_1, \dots, i_{n-1} \in E,$$

$$\boxed{\mathbb{P}(X_{n+1} = j \mid X_n = i, X_{n-1} = i_{n-1}, \dots, X_0 = i_0) = \mathbb{P}(X_{n+1} = j \mid X_n = i)}$$

Remarque Conditionnellement à l'état présent $\{X_n = i\}$, la probabilité d'arriver à l'état $\{X_{n+1} = j\}$ à l'itération suivante ne dépend pas des événements du passé $\{X_0 = i_0, \dots, X_{n-1} = i_{n-1}\}$. On parle d'*absence de mémoire*.

Définition (Chaîne de Markov Homogène) . Soit $(X_n)_{n \geq 0}$ une chaîne de Markov. On dit qu'elle est *homogène* si :

$$\forall n \in \mathbb{N}, \forall i, j \in E, \quad \mathbb{P}(X_{n+1} = j \mid X_n = i) = \mathbb{P}(X_1 = j \mid X_0 = i)$$

On se placera toujours dans ce cadre.

Remarque Une chaîne de Markov homogène est une chaîne de Markov où la probabilité de passer d'un état à un autre ne dépend pas du temps.

Exemple (Marche Aléatoire) On modélise le mouvement d'une particule sur une grille par une chaîne de Markov où X_n représente la position de la particule au temps $n \in \mathbb{N}$. On considère que la particule se déplace sur une seule dimension. Elle peut donc avancer d'une case ou reculer d'une case à chaque itération. On notera $p \in [0, 1]$ la probabilité qu'elle avance et $1 - p$ la probabilité qu'elle recule indépendamment du passé.

Soit $(\varepsilon_i)_{i \geq 0}$ à valeurs dans $\{-1, 1\}$ telle que :

$$\mathbb{P}(\varepsilon_i = 1) = 1 - \mathbb{P}(\varepsilon_i = -1) = p \in [0, 1]$$

la variable aléatoire qui représente le mouvement de la particule. Déterminons la loi de X_n :

$$X_{n+1} = X_n + \varepsilon_{n+1} = X_{n-1} + \varepsilon_n + \varepsilon_{n+1} = \dots = \sum_{i=1}^n \varepsilon_i$$

Montrons que c'est bien une chaîne de Markov :

$$\begin{aligned} X_{n+1} &= X_n + \varepsilon_{n+1} \\ &= \left(\sum_{i=1}^n \varepsilon_i \right) + \varepsilon_{n+1} \\ &= \sum_{i=1}^{n+1} \varepsilon_i \end{aligned}$$

où tous les $\varepsilon_i, i \in \llbracket 1, n+1 \rrbracket$ sont indépendants. On a donc :

$$\begin{aligned} \mathbb{P}(X_{n+1} = j \mid X_n = i, \dots, X_0 = i_0) &= \mathbb{P}(X_n + \varepsilon_{n+1} = j \mid X_n = i, \dots, X_0 = i_0) \\ &= \mathbb{P}(i + \varepsilon_{n+1} = j \mid X_n = i, \dots, X_0 = i_0) \\ &= \mathbb{P}(\varepsilon_{n+1} = j - i) \end{aligned}$$

On pourra alors s'intéresser au nombre de fois où une trajectoire passe par un point $n \in \mathbb{N}$. La théorie des chaînes de Markov permet de répondre à ce genre de questions.

Plus généralement, on peut construire récursivement des chaînes de Markov en définissant un X_0 fixé ou tiré par une loi π_0 puis en posant :

$$X_{n+1} = f(X_n, \varepsilon_{n+1})$$

où (ε_i) est une suite de variables aléatoires iid et f une fonction mesurable.

Définition (Matrice de Transition) . Soit $(X_n)_{n \geq 0}$ une chaîne de Markov. On définit la *matrice de transition* P de la chaîne comme la matrice, potentiellement infinie, telle que :

$$P(i, j) = \mathbb{P}(X_1 = j | X_0 = i)$$

Ainsi le coefficient de la i -ème ligne et de la j -ème colonne correspond à la probabilité de passer de l'état i à l'état j en une étape.

Exemple Soit $P \in \mathcal{M}_3(\mathbb{R})$ la matrice de transition suivante :

$$P = \begin{pmatrix} 1/4 & 3/4 & 0 \\ 0 & 1/2 & 1/2 \\ 1/2 & 0 & 1/2 \end{pmatrix}$$

On peut alors la représenter par un graphe pondéré orienté dont les noeuds représentent les états au temps i de la chaîne de Markov et les poids les probabilités de passer d'un état à l'autre en une étape.

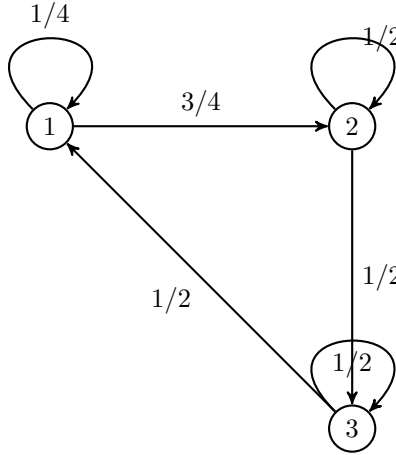


FIGURE 1.1 – Graphe Orienté Pondéré de la matrice de transition P

Définition (Matrice Stochastique) . La matrice de transition P est dite *stochastique* si

$$\forall i \in E, \quad \sum_{j \in E} P(i, j) = 1$$

(i.e si la somme des poids des arrêtes de tout noeud du graphe associé est égale à 1)

Proposition La loi d'une chaîne de Markov homogène $(X_n)_{n \geq 0}$ est complètement déterminée par la loi de π_0 de la condition initiale X_0 et par la matrice de transition P . On a :

$$\forall i_0, i_1, \dots, i_n \in E, \quad \mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n) = \pi_0(i_0)P(i_0, i_1)P(i_1, i_2) \dots P(i_{n-1}, i_n)$$

Démonstration Démontrons ce résultat par récurrence sur $\mathbb{N}$.

— Initialisation : Pour $n = 0$, par définition, on a :

$$\mathbb{P}(X_0 = i_0) = \pi_0(i_0)$$

— Hérédité : Soient $i_0, \dots, i_n \in E$, supposons que :

$$\mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n) = \pi_0(i_0)P(i_0, i_1)P(i_1, i_2) \dots P(i_{n-1}, i_n)$$

Soit $i_{n+1} \in E$ alors :

$$\begin{aligned} & \mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n, X_{n+1} = i_{n+1}) \\ &= \mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n) \times \mathbb{P}(X_{n+1} = i_{n+1} | X_0 = i_0, \dots, X_n = i_n) \\ &= \mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n) \times \mathbb{P}(X_{n+1} = i_{n+1} | X_n = i_n) \\ &= \pi_0(i_0)P(i_0, i_1)P(i_1, i_2) \dots P(i_{n-1}, i_n)P(i_n, i_{n+1}) \end{aligned}$$

On s'intéresse maintenant à la loi de X_n au temps $n \in \mathbb{N}$.

Propriété (Relation de Chapman-Kolmogorov) . Soit $(X_n)_{n \geq 0}$ une chaîne de markov homogène de matrice P et de loi initiale π_0 . Pour tout $i, j \in E$, on a :

$$\mathbb{P}(X_{n+m} = j | X_0 = i_0) = \sum_{k \in E} \mathbb{P}(X_{n+m} = j | X_n = k) \times \mathbb{P}(X_n = k | X_0 = i_0)$$

En particulier :

$$\mathbb{P}(X_n = j | X_0 = i_0) = P^n(i_0, j)$$

Démonstration Prouvons les résultats précédents.

Rappelons tout d'abord la formule des probabilités totales. Soit $(B_k)_{k \in \mathbb{N}}$ une partition de E , alors pour tout $A \in E$, on a :

$$\mathbb{P}(A) = \sum_{k \in \mathbb{N}} \mathbb{P}(A \cap B_k)$$

ou en conditionnel sur C :

$$\mathbb{P}(A | C) = \sum_{k \in \mathbb{N}} \mathbb{P}(A \cap B_k | C)$$

Or ici, $\{X_k | k \geq 0\}$ forme bien une partition de E puisque à chaque instant k , le processus est dans un unique état X_k . D'après la formule des probabilités totales, on a donc :

$$\begin{aligned} \mathbb{P}(X_{n+m} = j | X_0 = i_0) &= \sum_{k \in E} \mathbb{P}(X_{n+m} = j, X_n = k | X_0 = i_0) \\ &= \sum_{k \in E} \mathbb{P}(X_{n+m} = j | X_n = k, X_0 = i_0) \times \mathbb{P}(X_n = k | X_0 = i_0) \\ &= \sum_{k \in E} \mathbb{P}(X_{n+m} = j | X_n = k) \mathbb{P}(X_n = k | X_0 = i_0) \end{aligned}$$

On remarque de $\mathbb{P}(X_{n+m} = j | X_n = k)$ est la probabilité de passer de l'état k à j en m pas d'où :

$$\mathbb{P}(X_{n+m} = j | X_n = k) = \mathbb{P}(X_m = j | X_0 = k)$$

Déduisons maintenant la seconde formule de la première :

$$\begin{aligned}\mathbb{P}(X_2 = j | X_0 = i_0) &= \sum_{k \in E} \mathbb{P}(X_2 = j | X_1 = k) \mathbb{P}(X_1 = k | X_0 = i_0) \\ &= \sum_{k \in E} P(i, k) P(k, i) \\ &= P^2(i, j) \quad \text{etc} \dots\end{aligned}$$

Remarque (Notation) Si X_0 est distribué selon la loi π_0 , on notera :

$$\begin{aligned}\mathbb{P}_{\pi_0}(X_n = j) &= \sum_{i_0 \in E} \mathbb{P}(X_n = j | X_0 = i_0) \pi_0(i_0) \\ &= \sum_{i_0 \in E} \mathbb{P}(X_n = j | X_0 = i_0) \mathbb{P}(X_0 = i_0) \\ &= \sum_{i_0 \in E} P(i_0, j) \pi_0(i_0) \\ &= \pi_0 P^n(j)\end{aligned}$$

où $P^n(j)$ est la i ème colonne de P^n et π_0 un vecteur ligne.

Exemple (Simple) Soit $(X_n)_{n \geq 0}$ une suite de variables aléatoires réelles iid de loi π .

$$\text{i.e } \forall i \in E, \mathbb{P}(X_n = i) = \pi(i)$$

On a donc :

$$P(i, j) = \mathbb{P}(X_1 = j | X_0 = i) = \mathbb{P}(X_1 = j) = \pi(j)$$

C'est un cas très simple et peu intéressant.

Exemple (Ruine du joueur) Soit $n \geq 1$ et $p \in [0, 1]$ et une suite de variables aléatoires $(X_n)_{n \in \mathbb{N}}$ à valeurs dans $\llbracket 1, n \rrbracket$. On définit X_m comme le montant gagné par le joueur après m parties. On considère que le joueur ne peut pas jouer avec un montant supérieur à n . On peut donc représenter son évolution dans le jeu par le graphe ci-dessous :

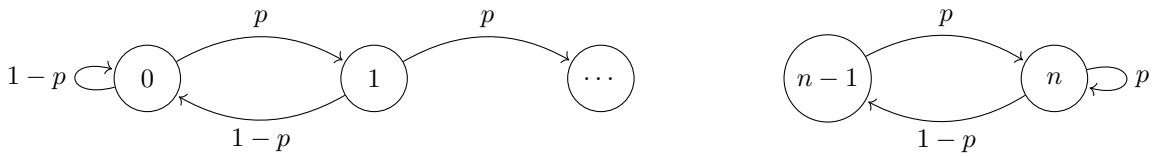


FIGURE 1.2 – Ruine du Joueur

où :

$$\begin{cases} P(k, k+1) = p & \text{si } k \in \{0, \dots, n-1\} \\ P(k, k+1) = 1-p & \text{si } k \in \{1, \dots, n\} \end{cases}$$

En pratique, on s'intéressera souvent à :

$$\mathbb{P}(X_n \text{ atteint } 0 \text{ avant } m | X_0 = i)$$

Exemple (File d'attente) On s'intéresse au nombre de personnes dans une file d'attente où, à chaque itération, une personne rentre ou sort de la file. Soit $(X_n)_{n \in \mathbb{N}}$ une suite à valeur dans $\mathbb{N}$. Ici, un pas de temps n représente un événement qui se passe. On va s'intéresser au fait que la chaîne tende vers 0 ou $+\infty$ en fonction de $i_0 \in \llbracket 1, n \rrbracket$.

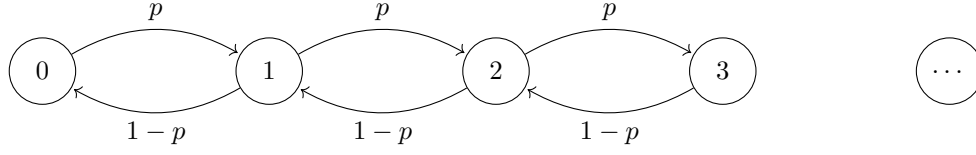


FIGURE 1.3 – File d'attente

Exemple (Processus de Galton-Watson) Ici $(X_n)_{n \geq 1}$ compte le nombre d'individus d'une population dans la génération n . On suppose que chaque individu vivant a des descendants indépendamment des autres. On appelle $(\zeta_{n,k})_{n \leq 1, k \leq 1}$ la suite donnant le nombre de descendants de l'individu k vivant à la génération n .

On peut alors en déduire que $(\zeta_{n,k})_{n \leq 1, k \leq 1}$ est une suite de var iid et que :

$$X_{n+1} = \sum_{k=1}^{X_n} \zeta_{n,k}$$

1.2 Propriétés de Markov

1.2.1 Propriété de Markov Faible

Théorème (Propriété de Markov Faible) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène de matrice de transition P et de loi initiale π_0 . Soit $m \in \mathbb{N}$ fixé. Alors, conditionnellement à l'événement $\{X_m = i\}$, la chaîne décalée $(X_{m+k})_{k \in \mathbb{N}}$ est une chaîne de Markov homogène de même matrice de transition P et de loi initiale δ_i .

En particulier, le *futur* $(X_{m+1}, X_{m+2}, \dots)$ est indépendant du *passé* $(X_0, X_1, \dots, X_{m-1})$ conditionnellement à X_m . Autrement dit, pour tous événements A mesurables par rapport à $\sigma(X_{m+1}, X_{m+2}, \dots)$ et B mesurables par rapport à $\sigma(X_0, X_1, \dots, X_{m-1})$, on a :

$$\mathbb{P}(A \cap B \mid X_m = i) = \mathbb{P}(A \mid X_m = i) \mathbb{P}(B \mid X_m = i).$$

Démonstration Démontrons ce théorème en deux parties :

1. Égalité des lois :

Montrons que, pour tout $n \in \mathbb{N}$ et toute suite d'états $(y_1, \dots, y_n)$,

$$\begin{aligned} \mathbb{P}(X_{m+1} = y_1, \dots, X_{m+n} = y_n \mid X_m = i, X_{m-1} = i_{m-1}, \dots, X_0 = i_0) \\ = \mathbb{P}(X_1 = y_1, \dots, X_n = y_n \mid X_0 = i) \end{aligned}$$

On a :

$$\begin{aligned} & \mathbb{P}(X_{m+1} = y_1, \dots, X_{m+n} = y_n \mid X_m = i, X_{m-1} = i_{m-1}, \dots, X_0 = i_0) \\ &= \frac{\mathbb{P}(X_{m+1} = y_1, \dots, X_{m+n} = y_n, X_m = i, X_{m-1} = i_{m-1}, \dots, X_0 = i_0)}{\mathbb{P}(X_m = i, X_{m-1} = i_{m-1}, \dots, X_0 = i_0)} \\ &= \frac{\pi_0(i_0)P(i_0, i_1) \cdots P(i_{m-1}, i)P(i, y_1) \cdots P(y_{n-1}, y_n)}{\pi_0(i_0)P(i_0, i_1) \cdots P(i_{m-1}, i)} \\ &= P(i, y_1)P(y_1, y_2) \cdots P(y_{n-1}, y_n) \\ &= \mathbb{P}(X_1 = y_1, \dots, X_n = y_n \mid X_0 = i). \end{aligned}$$

Ainsi, la chaîne décalée $(X_{m+k})_{k \in \mathbb{N}}$, conditionnellement à $X_m = i$, suit bien la loi d'une chaîne de Markov homogène de matrice P et de loi initiale δ_i .

2. Indépendance conditionnelle :

Soient A un événement mesurable par rapport au futur $\sigma(X_{m+1}, X_{m+2}, \dots)$ et B un événement mesurable par rapport au passé $\sigma(X_0, X_1, \dots, X_{m-1})$.

Montrons que :

$$\mathbb{P}(A \cap B \mid X_m = i) = \mathbb{P}(A \mid X_m = i) \mathbb{P}(B \mid X_m = i).$$

Par définition de la probabilité conditionnelle :

$$\mathbb{P}(A \cap B \mid X_m = i) = \frac{\mathbb{P}(A \cap B, X_m = i)}{\mathbb{P}(X_m = i)}.$$

D'après la propriété de Markov, conditionnellement à $X_m = i$, le futur $(X_{m+1}, X_{m+2}, \dots)$ ne dépend pas du passé $(X_0, \dots, X_{m-1})$, donc :

$$\mathbb{P}(A \cap B, X_m = i) = \mathbb{P}(A, X_m = i) \mathbb{P}(B, X_m = i) / \mathbb{P}(X_m = i).$$

Ainsi :

$$\begin{aligned} \mathbb{P}(A \cap B \mid X_m = i) &= \frac{\mathbb{P}(A, X_m = i) \mathbb{P}(B, X_m = i)}{\mathbb{P}(X_m = i)^2} \\ &= \mathbb{P}(A \mid X_m = i) \mathbb{P}(B \mid X_m = i). \end{aligned}$$

Ce qui montre bien l'indépendance conditionnelle du passé et du futur, sachant X_m .

Remarque Cette propriété de Markov est aussi vraie de façon plus générale quand on conditionne par l'état de la chaîne en un temps aléatoire appelé temps d'arrêt. Ce sera exactement le propos de la propriété de Markov forte que nous verrons plus tard.

1.2.2 Temps d'arrêt et propriété de Markov Forte

Définition (Tribu $\mathcal{F}_n$) . On définit $\mathcal{F}_n$ la tribu engendrée par les variables aléatoires $(X_1, \dots, X_n)$ la tribu :

$$\begin{aligned} \mathcal{F}_n &= \sigma(X_0, \dots, X_n) \\ &= \sigma(\{X_0 = x_0, \dots, X_n = x_n\}, x_0, \dots, x_n \in E) \end{aligned}$$

Ainsi, la tribu engendrée par l'ensemble $\{X_0 = x_0, \dots, X_n = x_n\}$ est la plus petite tribu contenant cette ensemble.

Définition (Temps d'arrêt) . Soit $(X_n)_{n \geq 0}$ une chaîne de Markov à valeurs dans un espace d'états E , définie sur un espace probabilisé $(\Omega, \mathcal{F}, \mathbb{P})$, avec

$$\forall n \in \mathbb{N}, \quad X_n : \Omega \longrightarrow E$$

On note $\mathcal{F}_n = \sigma(X_0, X_1, \dots, X_n)$ la tribu engendrée par les n premières variables de la chaîne. Une application

$$\tau : \Omega \longrightarrow \mathbb{N} \cup \{\infty\}$$

est appelée *temps d'arrêt* de la chaîne de Markov (ou pour la filtration $(\mathcal{F}_n)_{n \geq 0}$) si elle

vérifie :

$$\boxed{\forall n \in \mathbb{N}, \quad \{\tau \leq n\} \in \mathcal{F}_n}$$

Donc τ est mesurable pour la tribu $\mathcal{F}_n$. Autrement dit, le fait que l'évènement $\{\tau \leq n\}$ ne dépend que de l'évolution de la chaîne jusqu'au temps n .

Exemple Soit (X_n) une chaîne de Markov sur $\mathbb{Z}$. On définit le temps d'attente du dépassement de $j \in \mathbb{Z}$ comme la variable aléatoire :

$$\tau_j = \min\{i \geq 0 \mid X_i > j\}$$

τ_j est bien un temps d'arrêt pour la CMH car :

$$\{\tau_j \leq n\} = \{\exists k \in \mathbb{N}, X_k \geq j\}$$

donc τ_j ne dépend que de $X_0, \dots, X_n$. De façon générale, soit $A \subset E$, le temps d'attente de A défini par :

$$\tau_A = \min\{i \geq 0, X_i \in A\}$$

est un temps d'arrêt. Si cet ensemble est vide (i.e la chaîne de Markov n'atteint jamais A), alors $\tau_A = +\infty$. Par contre, $S_j = \max\{i \geq 0, X_i > j\}$ n'est pas un temps d'arrêt car $\{S_j \leq n\} = X_{n+1} < j, X_{n+2} < j, \dots$ dépend de ce qui s'est passé "après".

Théorème (Propriété de Markov forte) . Soit $(X_n)_{n \geq 0}$ une chaîne de Markov homogène et τ un temps d'arrêt. Alors conditionnellement à $\{\tau < \infty\}$ et $\{X_\tau = x\}$ la chaîne décalée $(X_{\tau+n})_{n \geq 0}$ est encore une chaîne de Markov de condition initiale x .
De plus, conditionnellement à $\{\tau < \infty\} \cap \{X_\tau = x\}$ on a :

$$(X_{\tau+n})_{n \geq 0} \text{ est indépendante de } (X_0, \dots, X_{\tau-1})$$

Remarque Ici, la chaîne de Markov est décalée conditionnellement à une variable aléatoire, c'est beaucoup plus fort que la propriété précédente.

Exemple (Application - Ruine du Joueur) Soit l'espace d'état $E = \{0, \dots, m\}$ et $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov sur E telle que $X_0 = k \in \{1, \dots, m\}$. On s'intéresse à deux temps d'arrêts :

- $\tau_0 = \inf\{n \geq 0 \mid X_n = 0\}$
- $\tau_m = \inf\{n \geq 0 \mid X_n = m\}$

On souhaite estimer le temps de jeu moyen d'un joueur. Pour cela, on considère le temps d'arrêt :

$$\tau = \min(\tau_0, \tau_m)$$

On veut savoir quand $t < t_0$ (i.e quand le joueur gagne avant de perdre). Soit $m > 1$, alors τ_0 est distinct de τ_m si l'un des deux est fini. En conséquence si $\tau < \infty$, on a :

- soit $\tau = \tau_0 < \tau_m$
- soit $\tau = \tau_m < \tau_0$.

On cherche la "probabilité de ruine" :

$$f(k) = \mathbb{P}(\tau < \infty, \tau = \tau_0 \mid X_0 = k)$$

Pour cela, nous devons donc déterminer toutes les valeurs de f . On utilise donc les propriétés de Markov pour en déduire des relations de récurrence entre $f(k)$, $f(k+1)$ et $f(k-1)$. On remarque ainsi que :

$$\text{si } X_0 = 0 \text{ alors } \tau = \tau_0 = 0 \text{ donc } f(0) = 1$$

si $X_0 = m$ alors $\tau = \tau_m = 0$ donc $f(m) = 0$

Si $X_0 \notin \{0, m\}$, on doit au moins faire un pas pour atteindre τ_0 ou τ_m . On en déduit donc que :

$$\tau_0((X_n)_{n \in \mathbb{N}}) = 1 + \tau_0((X_{n+1})_{n \in \mathbb{N}})$$

$$\tau_m((X_n)_{n \in \mathbb{N}}) = 1 + \tau_m((X_{n+1})_{n \in \mathbb{N}})$$

On a donc, via l'analyse à un pas :

$$\begin{aligned} f(k) &= \mathbb{P}(\tau((X_n)_{n \in \mathbb{N}}) = \tau_0((X_n)_{n \in \mathbb{N}}) < \infty \mid X_0 = k) \\ &= \mathbb{P}(\tau((X_{n+1})_{n \in \mathbb{N}}) = \tau_0((X_{n+1})_{n \in \mathbb{N}}) < \infty \mid X_0 = k) \end{aligned}$$

En appliquant la formule des probabilités totales à X_1 , on a donc :

$$\begin{aligned} f(k) &= \mathbb{P}(X_1 = k + 1 \mid X_0 = k) \times \mathbb{P}(\tau((X_{n+1})_{n \in \mathbb{N}}) = \tau_0((X_{n+1})_{n \in \mathbb{N}}) < \infty \mid X_1 = k + 1, X_0 = k) \\ &\quad + \mathbb{P}(X_1 = k - 1 \mid X_0 = k) \times \mathbb{P}(\tau((X_{n+1})_{n \in \mathbb{N}}) = \tau_0((X_{n+1})_{n \in \mathbb{N}}) < \infty \mid X_1 = k - 1, X_0 = k) \end{aligned}$$

Or, conditionnellement à $\{X_1 = k + 1\}$, la chaîne $(X_{n+1})_{n \in \mathbb{N}}$ est une chaîne de Markov partant de $k + 1$. On a donc :

$$\begin{aligned} f(k) &= p\mathbb{P}(\tau_0((X_n)_{n \in \mathbb{N}}) = \tau((X_n)_{n \in \mathbb{N}}) \mid X_0 = k + 1) \\ &\quad + (1 - p)\mathbb{P}(\tau_0((X_n)_{n \in \mathbb{N}}) = \tau((X_n)_{n \in \mathbb{N}}) \mid X_0 = k - 1) \\ &= pf(k + 1) + (1 - p)f(k - 1) \end{aligned}$$

Résolvons cette relation de récurrence pour $p = 1/2$. On a donc $\forall k \in \llbracket 1, m \rrbracket, k \notin \{0, m\}$:

$$f(k) = \frac{f(k + 1) + f(k - 1)}{2}$$

$$\iff f(k + 1) - f(k) = f(k) - f(k - 1) = \Delta$$

On peut donc poser :

$$\begin{aligned} f(m) - f(0) &= \sum_{k=0}^{m-1} f(k + 1) - f(k) = (m - 1)\Delta \\ \iff \Delta &= -\frac{1}{m} \end{aligned}$$

D'où :

$$f(k) = 1 + k\Delta = 1 - \frac{k}{m}$$

1.3 Mesures Invariantes

Dans cette section, nous supposons tout d'abord que l'espace d'états E est fini. Au début du cours on a vu que, pour une chaîne de Markov $(X_n)_{n \in \mathbb{N}}$ de matrice P et de loi initiale π_0 , la relation de Chapman-Kolmogorov nous donne :

$$\mathbb{P}(X_n = j \mid X_0 = i_0) = P^n(i_0, j)$$

Nous allons ici chercher des mesures de probabilités telles que :

$$\pi P = \pi$$

Nous appellerons de telles mesures des *mesures de probabilités invariantes*.

1.3.1 Définition et propriétés

Définition (Mesures Invariantes) . Soit une mesure positive π sur E . On dit que π est *invariante* pour une matrice de transition P si :

$$\forall x \in E, \quad \pi(x) = \pi P(x) = \sum_{y \in E} \pi(y) P(y, x)$$

On dit alors que π est un *vecteur propre à gauche* de P pour la valeur propre 1. Par convention, les mesures sont écrites comme des vecteurs ligne.

Remarque Attention, une mesure invariante n'est pas forcément une mesure de probabilité. Pour que ce soit le cas, il faut que sa masse totale soit égale à 1.

$$\text{i.e.} \quad \sum_{x \in E} \pi(x) = 1$$

Or ici, puisque E est fini, cette somme l'est aussi donc toute mesure invariante peut être normalisée en mesure de probabilités. Nous verrons plus tard que ce n'est pas toujours le cas.

Lemme Soit π une mesure invariante pour une matrice de transition P . Alors :

$$\forall n \in \mathbb{N}^*, \quad \pi = \pi P^n$$

De plus si X_0 est distribuée selon la loi π alors la loi de X_n l'est également pour tout $n > 0$. En conséquence :

$$\forall n, m \in \mathbb{N}, \quad (X_0, X_1, \dots, X_m) \stackrel{\text{loi}}{=} (X_n, X_{n+1}, \dots, X_{n+m})$$

Propriété (Mesure Invariante et Loi) . Soit π une mesure de probabilité invariante. Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène de matrice P et de loi initiale π_0 alors, pour tout $n \in \mathbb{N}$, $X_n \sim \pi$.

Ce lemme justifie toute l'utilisation que nous ferons des mesures invariantes. En effet, une mesure invariante permet de déterminer le comportement asymptotique de la chaîne de Markov sans avoir à la simuler en entier un grand nombre de fois.

1.3.2 Existence des mesures invariantes

Théorème (Mesure invariance, existence) . Toute chaîne de Markov homogène sur un espace d'état fini admet une mesure de probabilité invariante.

Démonstration Soit ψ une mesure de probabilité quelconque sur E où $|E| = d$. Posons :

$$\forall n \in \mathbb{N}, \quad \psi_n = \frac{1}{n} \sum_{k=0}^{n-1} \psi P^k$$

On sait que pour tout $k \in \mathbb{N}$, ψP^k est une mesure de probabilité sur E . Donc ψ_n l'est aussi une par construction.

De plus, pour tout $x \in E$, $(\psi_n(x))_{n \in \mathbb{N}}$ est une suite à valeurs dans $[0, 1]^d$. Par compacité, il existe une sous-suite convergente $(\psi_{\varphi(n)}(x))_{n \in \mathbb{N}}$. Comme E est fini, il existe une extraction telle que :

$$\forall x \in E, \quad \psi_{\varphi(n)}(x) \xrightarrow{n \rightarrow \infty} \pi(x)$$

Montrons maintenant que π est une mesure de probabilités invariante pour P . On a :

$$\forall n \in \mathbb{N}, \quad \sum_{x \in E} \psi_n(x) = 1 \quad \text{et} \quad \forall x \in E, \quad \psi_n(x) \geq 0$$

Donc lorsque $n \rightarrow \infty$, on a :

$$\sum_{x \in E} \pi(x) = 1 \quad \text{et} \quad \forall x \in E, \pi(x) \geq 0$$

Donc π est une mesure de probabilités sur E . De plus, pour tout $n \in \mathbb{N}$, on a :

$$\begin{aligned} \psi_n P &= \frac{1}{n} \sum_{k=1}^n \psi P^k P = \frac{1}{n} \sum_{k=1}^n \psi P^{k+1} = \frac{1}{n} \sum_{k=2}^{n+1} \psi P^k \\ &= \frac{1}{n} \frac{n+1}{n+1} \left(\sum_{k=1}^{n+1} \psi P^k - \psi P \right) \\ &= \frac{n+1}{n} \psi_{n+1} - \frac{1}{n} \psi P \end{aligned}$$

Donc lorsque $n \rightarrow \infty$, on obtient :

$$\pi P = \pi$$

Remarque Il existe pleins de méthodes numériques pour approcher une mesure invariante d'une chaîne de Markov homogène. Ce n'est pas un problème facile car il faut en général résoudre un système de d équations à d inconnues. Nous nous intéresserons donc aux *mesures réversibles*, plus faciles à calculer et qui nous donnent une mesure invariante quand elle existe.

Exemple Soit le graphe suivant et $p, q \in]0, 1[$. Soit $\pi = (\pi_1, \pi_2)$ une mesure invariante. On a

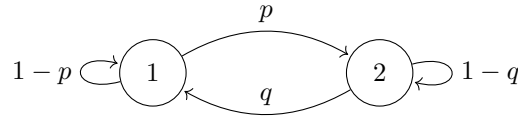


FIGURE 1.4 – Mesure Invariante

alors $\pi P = \pi$.

$$\begin{aligned} &\iff (\pi_1, \pi_2) \begin{pmatrix} 1-p & p \\ q & 1-q \end{pmatrix} = (\pi_1, \pi_2) \\ &\iff \begin{cases} \pi_1(1-p) + q\pi_2 = \pi_1 \\ \pi_1 p + \pi_2(1-q) = \pi_2 \end{cases} \\ &\iff \pi_1 = \frac{q}{p} \pi_2 \end{aligned}$$

Or π est une mesure de probabilité ssi :

$$p\pi_1 + \pi_2 = 1 \iff \pi_1 = \frac{q}{p+q} \text{ et } \pi_2 = \frac{p}{p+q}$$

1.3.3 Unicité des mesures de probabilité invariantes et irréductibilité

Définition (Communication) . Soient $x, y \in E$. On dit que x *communique avec* y s'il existe un chemin de probabilité positive de x à y dans E . On le note alors $x \rightsquigarrow y$. Plus formellement :

$$\begin{aligned} x \rightsquigarrow y &\iff \exists x, x_1, \dots, x_{n-1}, y \in E, P(x, x_1), P(x_1, x_2), \dots, P(x_{n-1}, y) > 0 \\ &\iff \forall n \geq 1, \quad P^n(x, y) > 0 \end{aligned}$$

Propriété (Transitivité) . La relation *communique* $\rightsquigarrow$ est transitive.

Définition (Classe fermée) . Soit $E_0 \subset E$ un sous-ensemble de E que l'on appellera *classe de* E . On dit que la classe E_0 est *fermée* si :

$$\forall x \in E_0, \quad x \rightsquigarrow y \implies y \in E_0$$

Définition (Classe irréductible) . Une classe E_0 de E est dite *irréductible* si elle est fermée et si :

$$\forall x, y \in E_0, \quad x \rightsquigarrow y \text{ et } y \rightsquigarrow x$$

Par extension, on dira qu'une chaîne de Markov homogène est irréductible si son espace d'état l'est aussi.

Théorème (Unicité des mesures invariantes) . Toute chaîne de Markov homogène irréductible sur un espace d'état fini admet une *unique mesure de probabilité invariante* π et :

$$\forall x \in E, \quad \pi(x) > 0$$

Démonstration Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène d'espace d'état E . Supposons que E est irréductible et soit π une mesure invariante de $(X_n)_{n \in \mathbb{N}}$.

1. Positivité : Montrons que π est positive. On sait que π est une mesure de probabilités donc :

$$\sum_{x \in E} \pi(x) = 1$$

donc il existe nécessairement $x_0 \in E$ tel que $\pi(x_0) > 0$. Comme E est irréductible, alors pour tout $x \in E, x_0 \rightsquigarrow x$. Donc il existe $n \in \mathbb{N}$ tel que :

$$P^n(x_0, x) > 0$$

De plus, on sait que π vérifie :

$$\begin{aligned} \pi(x) &= \sum_{x \in E} \pi(x) P(x, x) = \pi P \\ \implies \pi(y) &= \sum_{x \in E} \pi(x) P^n(x, y) \\ &= \underbrace{\pi(x_0) P^n(x_0, y)}_{>0} + \underbrace{\sum_{x \in E, x \neq x_0} \pi(x) P^n(x, y)}_{\geq 0} \end{aligned}$$

Donc $\forall y \in E, \pi(y) > 0$.

2. Unicité : voir TD

1.3.4 Construction de mesures de probabilité invariantes

Nous avons vu que les mesures de probabilité invariantes jouent un rôle fondamental dans l'étude des chaînes de Markov. Elles permettent notamment de décrire le comportement stationnaire du système. Nous allons à présent présenter quelques outils permettant de construire de telles mesures.

Soit E un espace d'états fini, et $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène et irréductible sur E . L'idée est de chercher à faire converger la loi de X_n lorsque n tend vers l'infini, de façon à obtenir une mesure de probabilité invariante pour la matrice de transition associée.

Définition (Temps de retour) . Soit $x \in E$, on définit le temps de retour en x comme :

$$\tau_x^+ := \inf\{n \geq 1 \mid X_n = x\} \in \overline{\mathbb{N}}$$

remarquons que c'est un *temps d'arrêt*.

Remarque Nous pouvons facilement montrer que tout temps de retour est un temps d'arrêt. En effet, pour rappel nous avons défini les temps d'arrêt comme :

Une application

$$\tau : \Omega \longrightarrow \mathbb{N} \cup \{\infty\}$$

est appelée temps d'arrêt de la chaîne de Markov (ou pour la filtration $(\mathcal{F}_n)_{n \geq 0}$) si elle vérifie :

$$\forall n \in \mathbb{N}, \quad \{\tau \leq n\} \in \mathcal{F}_n$$

Soit $n \in \mathbb{N}$ on a alors :

$$\begin{aligned} \{\tau_x^+ \leq n\} &= \{\exists k \in \llbracket 1, n \rrbracket \mid X_k = x\} \\ &= \bigcup_{k=1}^n \{X_k = x\} \end{aligned}$$

or, pour tout $k \in \llbracket 1, n \rrbracket$, l'évènement $\{X_k = x\} \in \mathcal{F}_k \subseteq \mathcal{F}_n$. Donc par union dénombrable, on a :

$$\{\tau_x^+ \leq n\} \in \mathcal{F}_n$$

Donc tout temps d'attente est un temps d'arrêt. De plus, on peut remarquer que la différence avec un temps de retour est que si $\{X_0 = x\}$ alors :

$$\tau_x = 0 \quad \text{et} \quad \tau_x^+ \geq 1$$

Lemme Pour tout $x, y \in E$, on a :

$$\mathbb{E}(\tau_y^+ \mid X_0 = x) < \infty$$

Théorème (Mesure Invariante) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène sur un espace d'états E fini. Alors son unique mesure de probabilités invariante est :

$$\pi(x) = \frac{1}{\mathbb{E}(\tau_x^+ \mid X_0 = x)} \quad \forall x \in E$$

Remarque Cette mesure charge plus les points $x \in E$ tels que $\mathbb{E}(\tau_x^+ \mid X_0 = x)$ est petit. C'est à dire les points que la chaîne de Markov visite souvent.

Démonstration Soit $x \in E$ une condition initiale. Soit $\bar{\pi} : E \rightarrow \mathbb{N}$ une application qui compte le nombre de fois que la chaîne visite un point $y \in E$.

$$\begin{aligned} \forall y \in E, \quad \bar{\pi}(y) &:= \mathbb{E} \left(\sum_{n=0}^{\tau_x^+ - 1} \mathbb{1}_{X_n=y} \mid X_0 = x \right) \\ &= \mathbb{E} \left(\sum_{n \geq 0} \mathbb{1}_{X_n=y} \times \mathbb{1}_{\tau_x^+ > n} \mid X_0 = x \right) \\ &= \sum_{n \geq 0} \mathbb{P}(X_n = y, \tau_x^+ > n \mid X_0 = x) \end{aligned}$$

On veut montrer que $\bar{\pi}$ est invariante. On veut donc :

$$\forall y \in E, \quad \bar{\pi}(y) = \sum_{z \in E} \bar{\pi}(z) P(z, y)$$

(On va également montrer que $\bar{\pi}$ ne dépend pas de la condition initiale x .) Pour tout $z \in E$, calculons donc :

$$\sum_{z \in E} \bar{\pi}(z) P(y, z) = \sum_{z \in E} \sum_{n \geq 0} \mathbb{P}(X_n = z, \tau_x^+ > n \mid X_0 = x) \times P(z, y)$$

Or on a :

$$\begin{aligned} &\mathbb{P}(X_n = z, \tau_x^+ > n, X_{n+1} = y \mid X_0 = x) \\ &= \mathbb{P}(X_{n+1} = y \mid X_n = z, \tau_x^+ > n, X_0 = x) \times \mathbb{P}(X_n = z, \tau_x^+ > n, X_0 = x) \\ &= \mathbb{P}(X_{n+1} = y \mid X_n = z) \times \mathbb{P}(X_n = z, \tau_x^+ > n, X_0 = x) \quad (\text{Propriété de Markov}) \\ &= P(z, y) \times \mathbb{P}(X_n = z, \tau_x^+ > n, X_0 = x) \end{aligned}$$

On en déduit alors que :

$$\begin{aligned} \sum_{z \in E} \bar{\pi}(z) P(y, z) &= \sum_{z \in E} \sum_{n \geq 0} \mathbb{P}(X_n = z, \tau_x^+ > n, X_{n+1} = y \mid X_0 = x) \\ &= \sum_{n \geq 0} \mathbb{P}(\tau_x^+ > n, X_{n+1} = y \mid X_0 = x) \quad (\text{Probabilités Totales}) \\ &= \sum_{n \geq 0} \mathbb{P}(\tau_x^+ \geq n+1, X_{n+1} = y \mid X_0 = x) \\ &= \sum_{n \geq 1} \mathbb{P}(\tau_x^+ \geq n, X_n = y \mid X_0 = x) \quad (\text{Changement d'indices}) \\ &= \sum_{n \geq 1} \mathbb{P}(\tau_x^+ > n, X_n = y \mid X_0 = x) + \sum_{n \geq 1} \mathbb{P}(\tau_x^+ = n, X_n = y \mid X_0 = x) \\ &= \bar{\pi}(y) - \mathbb{P}(X_0 = y, \tau_x^+ > 0 \mid X_0 = x) + \sum_{n \geq 1} \mathbb{P}(\tau_x^+ = n, X_n = y \mid X_0 = x) \end{aligned}$$

Or :

$$\begin{aligned}
& \sum_{n \geq 1} \mathbb{P}(\tau_x^+ = n, X_n = y \mid X_0 = x) \\
&= \sum_{n \geq 1} \mathbb{P}(\tau_x^+ = n, X_{\tau_x^+} = y \mid X_0 = x) \\
&= \sum_{n \geq 1} \mathbb{P}(\tau_x^+ < \infty, X_{\tau_x^+} = y \mid X_0 = x) \\
&= \mathbb{P}(X_{\tau_x^+} = y \mid X_0 = x)
\end{aligned}$$

D'où :

$$\sum_{z \in E} \bar{\pi}(z) P(z, y) = \bar{\pi}(y) - \underbrace{\mathbb{P}(X_0 = y, \tau_x^+ > 0 \mid X_0 = x)}_{1 \text{ si } y=x \text{ et } 0 \text{ sinon}} + \underbrace{\mathbb{P}(X_{\tau_x^+} = y \mid X_0 = x)}_{-1 \text{ si } y=x \text{ et } 0 \text{ sinon}} = \bar{\pi}(y)$$

Donc $\bar{\pi}$ est bien une mesure invariante de masse totale :

$$\sum_{y \in E} \bar{\pi}(y) = \mathbb{E}(\tau_x^+ \mid X_0 = x)$$

Or, d'après le lemme précédent, $\mathbb{E}(\tau_x^+ \mid X_0 = x) < \infty$. Donc l'application :

$$\forall y \in E, \quad \pi(y) = \frac{\bar{\pi}(y)}{\mathbb{E}(\tau_x^+ \mid X_0 = x)}$$

est bien une mesure de probabilités invariante. D'après le théorème d'unicité, c'est donc la seule. De plus, on a $\pi(x) = 1$ donc :

$$\forall y \in E, \quad \pi(y) = \frac{\bar{\pi}(y)}{\mathbb{E}(\tau_x^+ \mid X_0 = x)} = \frac{1}{\mathbb{E}(\tau_x^+ \mid X_0 = x)}$$

1.3.5 Réversibilité

Trouver la mesure de probabilité invariante en résolvant le système est compliqué en général. La *réversibilité* est une propriété suffisante pour qu'une mesure soit invariante.

Définition (Mesure réversible) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène de matrice P . Une mesure μ est dite réversible pour la matrice P si :

$$\boxed{\forall x, y \in E, \quad \mu(x)P(x, y) = \mu(y)P(y, x)}$$

Remarque La réversibilité peut être vue comme le fait d'être capable de parcourir les trajectoires d'une chaîne de Markov dans le sens inverse.

$$\text{i.e } \mathbb{P}(X_0 = x_0, X_1 = x_1, \dots, X_n = x_n) = \mathbb{P}(X_0 = x_n, X_1 = x_{n-1}, \dots, X_n = x_0)$$

Ce résultat provient de la relation de Chapman-Kolmogorov. En effet, si X_0 est choisie selon μ réversible, alors :

$$\begin{aligned}
\mathbb{P}(X_0 = x_0, X_1 = x_1, \dots, X_n = x_n) &= \mu(x_0)P(x_0, x_1) \dots P(x_{n-1}, x_n) \\
&= P(x_1, x_0)\mu(x_1)P(x_1, x_2) \dots P(x_{n-1}, x_n) \\
&= P(x_1, x_0)P(x_2, x_1) \dots P(x_n, x_{n-1})\mu(x_n) \\
&= \mu(x_n)P(x_n, x_{n-1}) \dots P(x_2, x_1)P(x_1, x_0) \\
&= \mathbb{P}(X_0 = x_n, X_1 = x_{n-1}, \dots, X_n = x_0)
\end{aligned}$$

Proposition Une mesure réversible pour une matrice de transition est invariante.

L'étude des mesures réversibles, plus simples que les mesures invariantes, sera donc privilégiée pour déterminer des mesures invariantes.

1.4 Classification des états

Ici, on suppose maintenant que E est un espace d'états fini ou dénombrable et $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène sur E de matrice de transition P .

1.4.1 Nombre de visites en un point

Définition (Nombre de visites) . Soit $x \in E$, on note V_x le nombre de visites de la chaîne de Markov $(X_n)_{n \in \mathbb{N}}$ dans x . Plus formellement :

$$V_x := |\{n \geq 1, X_n = x\}|$$

V_x est donc une variable aléatoire à valeurs dans $\overline{\mathbb{N}}$.

Théorème (Nombre de visites et temps de retour) . Soit τ_x^+ le temps de retour de la chaîne de Markov en $x \in E$. On a alors :

- si $\mathbb{P}(\tau_x^+ < \infty \mid X_0 = x) = 1$ alors $V_x = \infty$ presque sûrement.
- si $\mathbb{P}(\tau_x^+ < \infty \mid X_0 = x) = 1 - p < 1$ alors le nombre de visites est fini presque sûrement et il suit une loi géométrique.

$$\text{i.e. } \mathbb{P}(V_x = k) = p(1 - p)^k \quad \forall k \geq 0$$

Démonstration Pour démontrer ce théorème, il serait suffisant de montrer que $\forall k \in \mathbb{N}$, on ait :

$$\begin{aligned} \mathbb{P}(V_x \geq k \mid X_0 = x) &= \mathbb{P}(\tau_x^+ < \infty \mid X_0 = x)^k \\ \iff \mathbb{P}_x(V_x \geq k + 1) &= \mathbb{P}_x(\tau_x^+) \mathbb{P}_x(V_x \geq k) \end{aligned}$$

Montrons donc que :

$$\forall k \geq 0, \quad \mathbb{P}_x(V_x \geq k + 1) = \mathbb{P}_x(\tau_x^+ < \infty) \mathbb{P}_x(V_x \geq k)$$

Pour l'évènement $\{V_x \geq k + 1\}$ on a $V_x \geq 1$ donc $\tau_x^+ < \infty$ d'où :

$$\begin{aligned} \mathbb{P}_x(V_x \geq k + 1) &= \mathbb{P}_x(V_x \geq k + 1, \tau_x^+ < \infty) \\ &= \sum_{m \geq 1} \mathbb{P}_x(V_x \geq k + 1, \tau_x^+ = m) \\ &= \sum_{m \geq 1} \mathbb{P}_x(V_x \geq k + 1 \mid \tau_x^+ = m) \mathbb{P}_x(\tau_x^+ = m) \end{aligned}$$

Or on a :

$$\begin{aligned} &\mathbb{P}_x(V_x(X_n) \geq k + 1 \mid \tau_x^+ = m) \\ &= \mathbb{P}_x(V_x(X_{n+m}) + 1 \geq k + 1 \mid \tau_x^+ = m) \\ &= \mathbb{P}_x(V_x(X_n) \geq k) \end{aligned}$$

Donc :

$$\begin{aligned}\mathbb{P}_x(V_x \geq k+1) &= \sum_{m \geq 1} \mathbb{P}_x(V_x \geq k) \mathbb{P}_x(\tau_x^+ = m) \\ &= \mathbb{P}_x(V_x \geq k) \sum_{m \geq 1} \mathbb{P}_x(\tau_x^+ = m) \\ &= \mathbb{P}_x(V_x \geq k) \times \mathbb{P}_x(\tau_x^+ < \infty)\end{aligned}$$

D'où :

$$\mathbb{P}_x(V_x \geq k) = \mathbb{P}_x(\tau_x^+ < \infty)^k$$

Donc si $\mathbb{P}(\tau_x^+ < \infty) = 1$ alors $\mathbb{P}(V_x \geq k) = 1$ pour tout $k \geq 0$ donc $V_x = \infty$. D'autre part, si $\mathbb{P}(\tau_x^+ < \infty) = 1 - p < 1$ alors $\mathbb{P}(V_x \geq k) = (1 - p)^k$.

Remarque Précédemment, nous avons vu que dans le cas où E est fini et $(X_n)_{n \in \mathbb{N}}$ irréductible, alors :

$$\begin{aligned}\mathbb{E}(\tau_x^+ \mid X_0 = x) &< \infty \\ \implies \mathbb{P}(\tau_x^+ < \infty \mid X_0 = x) &= 1\end{aligned}$$

Donc on visitera le point x infiniment de fois.

Nous allons maintenant chercher à découper une trajectoire en "excursions" entre deux passages successifs par le point x . D'après la propriété de Markov forte, ces excursions auront la même loi et seront indépendantes conditionnellement.

Proposition Conditionnellement à l'évènement $\{\tau_x^+ = m\}$, $(X_{n+m})_{n \in \mathbb{N}}$ est une chaîne de Markov de condition initiale x et indépendante de $X_0, \dots, X_m$.

1.4.2 État transiant et état récurrent

Définition (État récurrent/transiant) . Si $x \in E$ tel que $\mathbb{P}_x(\tau_x^+ < \infty) = 1$, on dit alors que x est *récurrent* et $V_x = \infty$. Sinon, on dit que x est *transiant* et $V_x < \infty$ presque sûrement.

Proposition On dit qu'un état $x \in E$ est *récurrent* si :

$$\sum_{n \geq 0} P^n(x, x) = \infty$$

Or $P^n(x, x) = \mathbb{P}(X_n = x \mid X_0 = x)$ et $V_x = \sum_{n \geq 1} \mathbb{1}_{X_n = x}$ donc :

$$\mathbb{E}(V_x) = \sum_{n \geq 1} \mathbb{E}(\mathbb{1}_{X_n = x} \mid X_0 = x) = \sum_{n \geq 1} P^n(x, x)$$

Donc si x est *récurrent* alors $V_x = \infty$ presque sûrement. D'où :

$$\mathbb{E}(V_x) = \infty \implies \sum_{n \geq 1} P^n(x, x) = \infty$$

D'autre part, si x est *transiant*, alors $V_x \sim \mathcal{G}(p)$ où :

$$\mathbb{E}(V_x) < \infty \implies \sum_{n \geq 1} P^n(x, x) < \infty$$

Théorème (Récurrence et Transitivité) . Soient $x, y \in E$. Alors, si x est récurrent et que $x \rightsquigarrow y$ alors y est récurrent.

Proposition Donc $\rightsquigarrow$ est une relation d'équivalence pour les points récurrents. On peut donc partitionner l'ensemble des points récurrents de E en classes d'équivalences pour $\rightsquigarrow$ (i.e en parties connexes dans le graphe de la chaîne).

$$E = \mathcal{R} \sqcup \mathcal{C}$$

où $\mathcal{R}$ est l'ensemble des états récurrents de E et $\mathcal{C}$ l'ensemble des états transients. On a donc :

$$\mathcal{R} = \bigsqcup_{x \in E} \mathcal{R}_x$$

avec $\mathcal{R}_x$ la classe de x pour la relation $\rightsquigarrow$

$$\text{i.e } \forall x \in E, \quad \mathcal{R}_x = \{y \in E, x \rightsquigarrow y\}$$

Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov sur E .

- Si $X_0 = x \in \mathcal{R}$ alors pour tout $n \in \mathbb{N}$, $X_n \in \mathcal{R}_x$ presque sûrement. Donc la chaîne visitera tout les états de cette classe un nombre infini de fois.
- En revanche si $x \in \mathcal{C}$ on peut distinguer deux cas :
 - Si $\mathcal{C}$ est infini, alors pour tout $n \in \mathbb{N}$, $X_n \in \mathcal{C}$ et chaque état de $\mathcal{C}$ est visité un nombre fini de fois.
 - Si $\mathcal{C}$ est fini, alors il va exister $n \in \mathbb{N}$ tel que $X_n \in \mathcal{R}$ et on se retrouve dans le premier cas.

Exemple Soit E un espace d'état fini et $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov sur E définie par le graphe suivant :

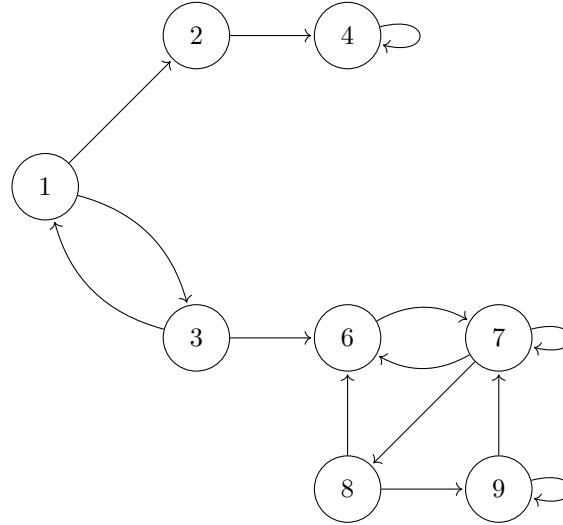


FIGURE 1.5 – Espace d'état fini

On a alors une partition de E :

$$E = \underbrace{\{1, 2, 3\}}_{\text{états transients}} \cup \underbrace{\underbrace{\{4\}}_{\mathcal{R}_4} \cup \underbrace{\{6, 7, 8, 9\}}_{\mathcal{R}_2}}_{\text{états récurrents}}$$

1.5 Mesures invariantes dans le cas dénombrable

1.5.1 États récurrents positif ou nul

De puis le début de ce chapitre, nous nous plaçons dans un ensemble d'états E fini. Or ce n'est pas toujours le cas. En général, E est infini et dénombrable. Nous pouvons donc définir une mesure de probabilité invariante π telle que :

$$\forall x \in E, \quad \pi(x) = \frac{1}{\mathbb{E}(\tau_x^+ \mid X_0 = x)}$$

Du fait de la dénombrabilité de E , nous aurons plusieurs classes d'états qui communiquent entre eux. On peut donc redéfinir la notion d'*irréductibilité* pour ces chaînes.

Définition (Chaîne irréductible (cas dénombrable)) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène sur un espace d'état E dénombrable. On dira que $(X_n)_{n \in \mathbb{N}}$ est irréductible :

- si tous les états de E sont transients
- si il existe une unique classe de récurrence dans E

$$\text{i.e. } \forall x, y \in E, \quad x \text{ et } y \text{ récurrent et } x \rightsquigarrow y$$

Remarque Si on considère une chaîne de Markov irréductible et récurrente alors :

$$\forall x \in E, \mathbb{P}_x(\tau_x^+ < \infty) = 1$$

Or quand E est fini, on n'a pas forcément :

$$\mathbb{E}(\tau_x^+ \mid X_0 = x) < \infty$$

Cela va donc nous mener à introduire une classification plus fine des états en distinguant les états récurrents selon que $\mathbb{E}(\tau_x^+ \mid X_0 = x) < \infty$ ou pas.

Définition (État récurrent positif) . On dit qu'un état $x \in E$ est *récurrent positif* s'il est récurrent et que $\mathbb{E}(\tau_x^+ \mid X_0 = x) < \infty$. On dira en revanche que x est *récurrent nul* s'il est récurrent et que :

$$\mathbb{P}_x(\tau_x^+ < \infty) = 1 \text{ et } \mathbb{E}_x(\tau_x^+) = \infty$$

La notion de récurrence positive ou nulle permet de quantifier la vitesse de retour de la chaîne dans un état x . Un état récurrent positif est donc un état où la chaîne revient souvent alors qu'un état récurrent nul est un état où la chaîne revient en temps fini mais potentiellement très long. C'est ce que traduit le fait que $\mathbb{E}_x(\tau_x^+)$ soit fini ou pas.

Remarque Dans le cas des espaces d'états finis, lors de la construction de la mesure de probabilités invariante, nous avons posé :

$$\bar{\pi}(y) = \mathbb{E} \left(\sum_{n=0}^{\tau_x^+ - 1} \mathbb{1}_{X_n = y} \right)$$

en tant que mesure invariante de masse totale $\mathbb{E}_x(\tau_x^+)$. Or dans le cas actuel si $(X_n)_{n \in \mathbb{N}}$ est récurrente positive, on aura $\mathbb{E}_x(\tau_x^+) < \infty$ donc on pourra construire notre mesure de probabilités invariante. Dans le cas contraire, notre mesure ne sera pas de probabilité.

1.5.2 Propriétés des chaînes irréductibles

Propriété (Positivité) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov irréductible récurrente. Si π est une mesure de probabilités invariante non nulle, alors :

$$\forall x \in E, \quad \pi(x) > 0.$$

Démonstration Comme π est non nulle, il existe $x_0 \in E$ tel que $\pi(x_0) > 0$. Comme la chaîne est irréductible, pour tout $y \in E$, il existe $n_0 \geq 0$ tel que :

$$P^{n_0}(x_0, y) > 0$$

En utilisant l'invariance de π et en isolant le terme $z = x_0$, on obtient :

$$\pi(y) = \sum_{z \in E} \pi(z) P^{n_0}(z, y) \geq \pi(x_0) P^{n_0}(x_0, y) > 0$$

Ainsi, chaque état $y \in E$ a une mesure strictement positive sous π .

Théorème (Unicité) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov irréductible et récurrente. Il existe alors une unique mesure invariante à constante multiplicative près.

- De plus, si tous les états sont *récurrents nuls* alors toute mesure invariante non nulle est de masse infinie.
- En revanche, si $(X_n)_{n \in \mathbb{N}}$ est *récurrente positive*, alors toutes les mesures invariantes sont de masse finie donc il existe une unique mesure de probabilités invariante telle que :

$$\forall x \in E, \quad \pi(x) = \frac{1}{\mathbb{E}_x(\tau_x^+)}$$

Exemple (Modèle de file d'attente) Soit $(X_n)_{(n \in \mathbb{N})}$ une chaîne de Markov définie par le graphe suivant :

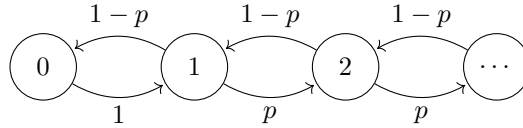


FIGURE 1.6 – Graphe d'un modèle de file d'attente

Donc les transitions sont décrites par la matrice de transition P :

$$\begin{cases} P(k, k+1) = p & \text{si } k \geq 1 \\ P(k, k-1) = 1-p & \text{si } k \geq 1 \\ P(0, 1) = 1 \end{cases}$$

Cette chaîne de Markov est irréductible car :

$$\forall x, y \in \mathbb{N}, \exists n \in \mathbb{N}, \quad \mathbb{P}_x(X_n = y) > 0$$

$$\iff \forall x, y \in \mathbb{N}, \exists n \in \mathbb{N}, \quad P^n(x, y) > 0$$

Déterminons une mesure invariante pour cette chaîne de Markov. Soit π une mesure invariante pour P . Alors elle vérifie $\pi P = \pi$ d'où :

$$\begin{cases} \pi(0) = \sum_{k \in \mathbb{N}} \pi(k) P(k, 0) = (1-p)\pi(1) \\ \pi(1) = \pi(0) + (1-p)\pi(2) \end{cases}$$

et $\forall k \geq 1$, on a $\pi(k) = p\pi(k-1) + (1-p)\pi(k+1)$. Nous avons donc une relation de récurrence d'ordre 2 :

$$Q = (1-p)X^2 - X + p$$

de racines évidentes 1 et $p/(1-p)$. Les solutions de la relation de récurrence sont donc de la forme :

$$\forall k \geq 1, \quad \pi(k) = a1^k + b\left(\frac{p}{1-p}\right)^k$$

Déterminons a et b . On a :

$$\begin{cases} \pi(1) = \frac{\pi(0)}{1-p} = a + \frac{bp}{1-p} \\ \pi(2) = \frac{\pi(0)}{1-p} \left(\frac{1}{1-p} - 1 \right) = \frac{\pi(0)p}{(1-p)^2} = a + b\frac{p^2}{(1-p)^2} \end{cases}$$

Par identification, on obtient donc :

$$a = 0 \quad \text{et} \quad b = \frac{\pi(0)}{p}$$

Donc la solution de l'équation de récurrence s'écrit :

$$\forall k \geq 1, \quad \pi(k) = \frac{\pi(0)}{p} \left(\frac{p}{1-p} \right)^k$$

Pour aller plus loin, en considérant la série $\sum_{k \geq 0} \pi(k)$ on peut affirmer que si $p < 1/2$ alors la mesure peut être de probabilité. Dans le cas contraire, toutes les mesures invariantes seront de masse infinie.

1.5.3 Critère de récurrence positive

Il peut être très difficile de calculer $\mathbb{E}_x(\tau_x^+)$ car il faut très bien connaître les propriétés des excursions de la chaîne. En pratique, on utilisera le critère suivant :

Théorème (Réciproque) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov irréductible. Si il existe une mesure invariante π non nulle et de masse finie alors la chaîne est récurrente positive.

Ce théorème traduit le fait qu'il y a équivalence entre le fait qu'une chaîne de Markov soit récurrente positive et qu'elle admette une mesure de probabilités invariante.

Démonstration Revenons à notre théorème d'unicité des mesures invariantes. Le principe de la démonstration est le même que dans le cas discret. Soit π une mesure invariante non nulle de masse finie. On veut comparer π avec $\bar{\pi}$ qui compte le nombre moyen de passage en $y \in \mathbb{E}$.

$$\forall x \in E, \quad \bar{\pi}(x) = \mathbb{E}_x \left(\sum_{n=0}^{\tau_x^+-1} \mathbb{1}_{X_n=y} \right) = \sum_{n \geq 0} \mathbb{P}_x(X_n = y, \tau_x^+ > n)$$

or π est invariante donc $\forall y \in E$, on a :

$$\begin{aligned}
\pi(y) &= \sum_{z \in E} \pi(z)P(z, y) \\
&= \pi(x)P(x, y) + \sum_{z_1 \neq x} \pi(z_1)P(z_1, y) \\
&= \pi(x)P(x, y) + \sum_{z_1 \neq x} \left(\sum_{z_2 \in E} \pi(z_2)P(z_2, z_1) \right) P(z_1, y) \\
&= \pi(x)P(x, y) + \underbrace{\pi(x) \sum_{z_1 \neq x} P(x, z_1)P(z_1, y)}_{\text{terme } z_2=x} + \sum_{z_1 \neq x} \left(\sum_{z_2 \neq x} \pi(z_2)P(z_2, z_1)P(z_1, y) \right)
\end{aligned}$$

où $\pi(x) \sum_{z_1 \neq x} P(x, z_1)P(z_1, x) = \mathbb{P}(X_2 = y, X_1 \neq x, X_0 = x)$. Si on itère $l \in \mathbb{N}$ fois ce processus, on aura donc :

$$\begin{aligned}
\pi(y) &= \pi(x) \left(P(x, y) + \sum_{k=1}^{l-1} \sum_{\substack{z_1 \neq x \\ \vdots \\ z_k \neq x}} P(x, z_k)P(z_k, z_{k-1}) \dots P(z_1, y) \right) \\
&\quad + \sum_{\substack{z_1 \neq x \\ \vdots \\ z_l \neq x}} \pi(z_l)P(z_l, z_{l-1}) \dots P(z_1, y)
\end{aligned}$$

Donc

$$\pi(y) \geq \pi(x) \left[\sum_{k=1}^{l-1} \mathbb{P}_x(X - k = y, \tau_x^+ > k) \right]$$

or ceci est valable pour tout $l \in \mathbb{N}$ donc on peut faire tendre l vers ∞ d'où :

$$\pi(y) \geq \pi(x)\bar{\pi}(y)$$

Si on choisit $x \in E$ tel que $\pi(x) > 0$ alors :

$$\infty > \frac{\pi(y)}{\pi(x)} \geq \bar{\pi}(y)$$

donc $\bar{\pi}$ est bien définie. Or $\bar{\pi}$ est définie de telle sorte que :

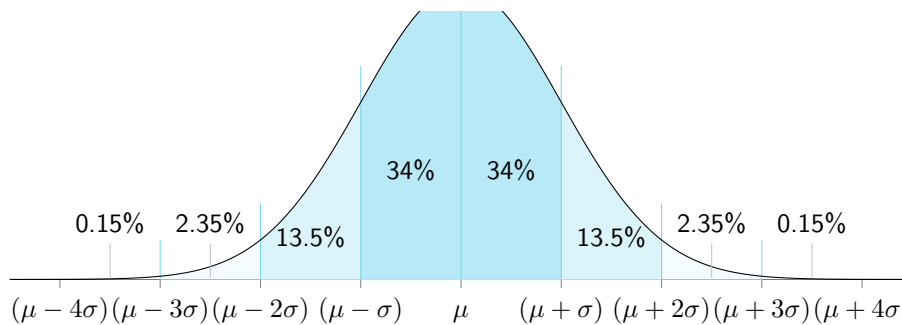
$$\sum_{y \in E} \bar{\pi}(y) = \mathbb{E}_x(\tau_x^+) \leq \sum_{y \in E} \frac{\pi(y)}{\pi(x)}$$

or $\sum_{y \in E} \frac{\pi(y)}{\pi(x)} < \infty$ car π est finie donc :

$$\forall x \in E, \quad \mathbb{E}_x(\tau_x^+) < \infty$$

i.e x est récurrent positif donc la chaîne des récurrente positive irréductible. On peut donc construire l'unique mesure de probabilité invariante.

Simulation Aléatoire



Chapitre 1

Simulation de variables aléatoires

Contents

1.0.1	Introduction	28
1.1	Simulation de variables aléatoires discrètes	29
1.1.1	Exemples Introductifs	29
1.1.2	Méthode Générale - Inverse de la fonction de masse	30
1.1.3	Méthode du rejet (Cas discret)	31
1.2	Simulation de variables aléatoires à densité	32
1.2.1	Méthode de l'inverse généralisée	33
1.2.2	Méthode du rejet (Cas continu)	34
1.3	Méthodes ad-hoc	36
1.3.1	Loi Normale - Méthode de Box-Muller	37
1.3.2	Lois de Poisson	38
1.3.3	Mélange de lois	40
1.3.4	Méthode de Monte-Carlo	40

1.0.1 Introduction

La simulation peut se définir comme la prédiction de l'évolution d'un système en résolvant des équations venues de domaines appliqués. La notion d'aléatoire vient du fait que l'on simule des phénomènes dont nous n'aurons que des informations imparfaites tels que des phénomènes de désintégration nucléaire ou de prévention des risques. Les utilisations possibles de la simulation sont :

- L'étude d'un phénomène (ex : la trajectoire d'un avion dans des turbulences)
- L'estimation de caractéristiques d'un système (ex : déterminer les caractéristiques d'une simulation météo)
- Tester des hypothèses (ex : en médecine, on peut utiliser la simulation aléatoire pour déterminer à l'avance les effets d'un médicament sur un patient.)
- La prise de décision dans un environnement incertain.
- Réaliser des expériences coûteuses voire impossibles (ex : effectuer des essais nucléaires informatiquement en fonction des essais passés ou simuler une propagation virale dans une population).

La simulation aléatoire est donc un outil rapide économique et contrôlé.

Définition (Simuler une variable aléatoire) . Simuler une variable aléatoire de loi μ consiste à produire un algorithme dont les réalisations donnent des copies indépendantes d'une variable de loi μ .

La simulation aléatoire est impossible sans l'accès à des *données aléatoires*. En pratique, on se contentera de données *pseudo-aléatoires* tel que le nombre de secondes écoulées depuis le 01/01/1070, la température d'un processeur ou les dernières touches entrées avant l'exécution du programme. Ceci est obtenu en **Python** avec la fonction `rand()`. Dans ce cours, on considère que chaque appel à cette fonction nous renvoie une variable iid de loi Uniforme sur $[0, 1]$. Simuler une variable aléatoire consiste donc à construire un algorithme qui transforme un nombre fini de variables aléatoires iid de loi $\mathcal{U}([0, 1])$ en une variable aléatoire de loi μ :

$$\mathcal{U}([0, 1]) \xrightarrow{\text{simulation}} \mu$$

1.1 Simulation de variables aléatoires discrètes

1.1.1 Exemples Introductifs

Exemple (Simuler une variable aléatoire de loi $\mathcal{B}(p)$) Soit $X \sim \mathcal{U}([0, 1])$ avec $p \in [0, 1]$. Alors la variable $Y = \mathbb{1}_{X \leq p}$ suit une loi de Bernoulli de paramètre p .

$$\mathbb{P}(Y = 1) = \mathbb{P}(\mathbb{1}_{X \leq p} = 1) = \mathbb{P}(X \leq p) = p$$

$$\mathbb{P}(Y = 0) = 1 - p$$

On peut donc développer l'algorithme suivant :

```

1 def bernouilli(p):
2     x = rand()
3     if x <= p:
4         return 1
5     return 0
6 
```

Exemple (Simuler une loi binomiale $\mathcal{B}(n, p)$) Soient $(X_1, \dots, X_n)$ une suite de variables aléatoires iid de loi $\mathcal{B}(n, p)$. La variable $Y := \sum_{i=1}^n X_i \sim \mathcal{B}(n, p)$ telle que :

$$\mathbb{P}(Y = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

Il nous faut donc seulement générer la somme de n variables aléatoires suivant une loi de Bernoulli. On a donc :

```

1 def binom(n, p):
2     s = 0
3     for i in range(n):
4         s += 1 * (rand() <= p)
5     return s
6 
```

On cherchera à développer des programmes qui terminent en temps fini. On préférera ceux qui terminent en temps borné (comme `binom(n, p)`).

1.1.2 Méthode Générale - Inverse de la fonction de masse

Proposition Soit $(p_n)_{n \geq 1}$ une suite réelle positive telle que :

$$\sum_{k=1}^{\infty} p_k = 1$$

et (x_n) une suite d'éléments d'un ensemble E . On définit la loi μ par :

$$\mu = \sum_{k=1}^{\infty} p_k \delta_{x_k}$$

si une variable aléatoire X est de loi μ on aura alors $\mathbb{P}(X = x_k) = p_k$. On pose $f_0 = 0$ et $s_k = p_1 + \dots + p_k$. Soit U une variable aléatoire de loi $\mathcal{U}([0, 1])$. Alors la variable :

$$Y = \sum_{k=1}^{\infty} x_k \times \mathbb{1}_{s_{k-1} \leq U \leq s_k}$$

suit la loi μ . L'algorithme suivant simule une variable aléatoire de loi μ .

```

1 def simulation_mu([p1, ..., pk], [x1, ..., xk]):
2     u = rand()
3     s = p1
4     while u > s :
5         i++
6         s += pi
7     return xi
8

```

Démonstration Pour tout $n \in \mathbb{N}$, on a :

$$\mathbb{P}(s_{n-1} \leq U \leq s_n) = s_n - s_{n-1} = p_n$$

donc $\mathbb{P}(Y = x_n) = \mathbb{P}(s_{n-1} \leq U \leq s_n) = p_n$.

Exemple (Simuler une loi de Poisson $\mathcal{P}(\lambda)$) Pour rappel, si $X \sim \mathcal{P}(\lambda)$ alors :

$$\forall k \in \mathbb{N}, \quad \mathbb{P}(X = k) = e^{-\lambda} \frac{\lambda^k}{k!}$$

Posons $p_k := e^{-\lambda} \frac{\lambda^k}{k!}$. On remarque que :

$$p_{k+1} = e^{-\lambda} \frac{\lambda^{k+1}}{(k+1)!} = p_k \frac{\lambda}{k+1}$$

On en déduit donc l'algorithme suivant :

```

1 def poisson(lambda):
2     x = 0
3     p = exp(-lambda)
4     s = p
5     u = rand()
6     while u > s :
7         x += 1
8         p = p * lambda / x
9         s += p
10    return x
11

```

Exemple (Loi uniforme) Simulons une loi uniforme sur $\{1, \dots, n\}$. On a donc l'algorithme suivant :

```

1 def uniforme(n):
2     x = 1
3     s = x/n
4     u = rand()
5     while u > s :
6         x += 1
7         s += 1/n
8     return x
9
10 def uniforme(n):
11     return ceil(n * rand(u))
12

```

1.1.3 Méthode du rejet (Cas discret)

Proposition (Algorithme du Rejet) Soit $(E, \mathcal{E}, \mu)$ un espace probabilisé et $A \subset E$ un événement tel que $\mu(A) > 0$. La loi μ conditionnellement à A , notée μ_A , est définie par :

$$\mu_A(B) = \frac{\mu(A \cap B)}{\mu(A)}, \quad B \in \mathcal{E}.$$

Soit $(X_n)_{n \geq 1}$ une suite de variables aléatoires i.i.d. de loi μ . On pose :

$$T := \inf\{n \geq 1 \mid X_n \in A\},$$

c'est-à-dire le rang du premier tirage appartenant à A . Alors :

1. $T \sim \mathcal{G}(\mu(A))$, c'est-à-dire

$$\mathbb{P}(T = k) = (1 - \mu(A))^{k-1} \mu(A), \quad k = 1, 2, \dots$$

2. $X_T \sim \mu_A$ (loi conditionnelle sur A)
3. X_T et T sont indépendantes.

Démonstration Soit $B \in \mathcal{E}$ et $k \geq 1$. On calcule :

$$\begin{aligned}
 \mathbb{P}(X_T \in B, T = k) &= \mathbb{P}(X_k \in B, T = k) \\
 &= \mathbb{P}(X_1 \notin A, \dots, X_{k-1} \notin A, X_k \in A, X_k \in B) \\
 &= \mathbb{P}(X_1 \notin A)^{k-1} \times \mathbb{P}(X_1 \in A \cap B) \\
 &= (1 - \mu(A))^{k-1} \mu(A) \frac{\mu(A \cap B)}{\mu(A)} \\
 &= \underbrace{(1 - \mu(A))^{k-1} \mu(A)}_{\mathbb{P}(T=k)} \underbrace{\frac{\mu(A \cap B)}{\mu(A)}}_{\mu_A(B)}.
 \end{aligned}$$

On en déduit :

- $\mathbb{P}(T = k) = (1 - \mu(A))^{k-1} \mu(A)$ (loi géométrique)
- $\mathbb{P}(X_T \in B) = \mu_A(B)$
- $\mathbb{P}(X_T \in B, T = k) = \mathbb{P}(T = k) \mathbb{P}(X_T \in B)$, donc T et X_T sont indépendantes.

Interprétation. L'algorithme du rejet décrit la procédure suivante :

1. Tirer $X_1, X_2, \dots$ de manière indépendante selon la loi μ .
2. Rejeter tout tirage $X_n \notin A$ et recommencer.
3. Accepter le premier $X_T \in A$: sa loi est exactement μ_A .

Ainsi :

- T représente le nombre de tirages nécessaires avant d'obtenir un succès.
- X_T est la valeur de ce premier succès, distribuée comme μ conditionnellement à A .
- T et X_T sont indépendants : le temps d'attente n'influence pas la valeur obtenue.

Exemple Si E est l'ensemble des notes d'étudiants, μ la distribution des notes et $A = \{\text{note} > 15\}$, alors :

- T = nombre d'étudiants observés avant d'en trouver un avec une note > 15 ,
- X_T = note de ce premier étudiant (loi conditionnelle sur A).

La loi de T est géométrique et la loi de X_T est la loi des notes sachant qu'elles sont > 15 .

Exemple On veut simuler une variable aléatoire U de loi uniforme sur les nombres premiers inférieurs à n . D'après la proposition précédente, on peut donc utiliser l'algorithme suivant :

```

1 def unif_prem(n):
2     x = uniforme(n)
3     while not premier(x):
4         x = uniforme(n)
5     return x
6

```

Ainsi le premier nombre premier sur lequel on tombe est uniformément distribué sur l'ensemble des nombres premiers inférieurs ou égaux à n .

1.2 Simulation de variables aléatoires à densité

Dans cette section, nous allons voir comment simuler des variables aléatoires continues. Nous découvrirons la méthode d'inversion de la fonction de répartition, une autre version de la méthode du rejet ainsi que d'autres exemples.

Définition ((Rappel) Fonction de répartition) . Soit μ une mesure de probabilité sur $\mathbb{R}$. La fonction de répartition de μ est la fonction :

$$F : \begin{cases} \mathbb{R} \longrightarrow \mathbb{R} \\ x \longmapsto \mu(]-\infty, x]) \end{cases}$$

Proposition La fonction de répartition F d'une loi admet plusieurs propriétés :

- F est croissante
- F est continue à droite, elle tend vers 1 en $+\infty$ et vers 0 en $-\infty$.

De plus, si μ admet une densité f par rapport à la mesure de Lebesgue, alors F est dérivable presque partout et $\forall x \in \mathbb{R}, F'(x) = f(x)$.

1.2.1 Méthode de l'inverse généralisée

La fonction de répartition F d'une variable aléatoire X nous donne la probabilité que X soit inférieure à $x \in \mathbb{R}$ notée $\mathbb{P}(X < x) \in [0, 1]$. Puisque l'on sait facilement générer un nombre aléatoire entre 0 et 1 avec la fonction `rand()`, il serait intéressant de voir si on ne peut pas, à partir de celui-ci, déterminer une valeur aléatoire de la variable X . C'est ce que fait la méthode de l'inverse généralisée.

Définition (Inverse Généralisée) . Soit $F : U \longrightarrow V$ une fonction croissante définie à droite. L'inverse généralisée de F est la fonction :

$$F^{-1} : \begin{cases} V \longrightarrow U \\ y \longmapsto \inf \{x \in U | F(x) > y\} \end{cases}$$

Proposition Si $F : U \longrightarrow V$ est inversible sur $I \subset U$ alors :

$$\forall y \in F(I), \quad F^{-1}(y) = (F|_I)^{-1}(y)$$

Exemple Soit F la fonction de répartition de la loi uniforme $\mathcal{U}([a, b])$. On sait que F est une bijection de $[a, b]$ sur $[0, 1]$ comme fonction continue strictement croissante.

$$F|_{[a, b]} = \varphi : x \longmapsto \frac{x - a}{b - a}$$

d'inverse φ^{-1} telle que $\forall y \in [0, 1]$, on ait :

$$\begin{aligned} \varphi^{-1}(y) = x &\iff \varphi(x) = y \\ &\iff y = \frac{x - a}{b - a} \\ &\iff x = (b - a)y + a \end{aligned}$$

D'où $\forall y \in [0, 1]$, $\varphi^{-1}(y) = (b - a)y + a$.

Théorème (Inverse de la fonction de répartition) . Soit μ une mesure de probabilité sur $\mathbb{R}$. On note F sa fonction de répartition et U une variable aléatoire de loi Uniforme sur $[0, 1]$. On a alors :

$$F^{-1}(U) \sim \mu$$

Démonstration Soit F la fonction de répartition d'un loi μ et $U \sim \mathcal{U}([0, 1])$. Montrons que $F^{-1}(U) \sim \mu$. Soit $x \in \mathbb{R}$, on a :

$$\mathbb{P}(F^{-1}(U) < x) = \mathbb{P}(U < F(x)) = F(x)$$

Remarque Ce théorème nous assure que la méthode de l'inverse de la fonction de répartition nous permet de générer des valeurs de la même loi. Pour les variables aléatoires discrètes, cette méthode revient au même que le premier algorithme vu dans ce cours.

Proposition Soit X une variable aléatoire de densité f .

1. Calculer $F(x) = \int_{-\infty}^x f(t) dt$ la fonction de répartition de X .
2. Calculer F^{-1} l'inverse généralisée de F sur un intervalle bien choisi.
3. Écrire l'algorithme :

```
1 def simulerX():
2     return invF(rand())
3
```

Exemple Simulons une variable aléatoire X dont la densité est :

$$f_X(x) = \frac{1}{\pi} \frac{1}{\sqrt{1-x^2}}, \quad \forall x \in]-1, 1[$$

Calculons sa fonction de répartition :

$$\begin{aligned} F_X(x) &= \int_{-\infty}^x \frac{1}{\pi} \frac{1}{\sqrt{1-t^2}} dt \\ &= \int_{-1}^x \frac{1}{\pi} \frac{1}{\sqrt{1-t^2}} dt \\ &= \frac{1}{\pi} [\arcsin t]_{-1}^x \\ &= \frac{1}{\pi} \arcsin x - \frac{1}{2} \end{aligned}$$

or F est strictement croissante et continue sur $] -1, 1[$ donc elle réalise une bijection de $] -1, 1[$ dans $[0, 1]$. Soient $x \in] -1, 1[$ et $y \in [0, 1]$ tels que :

$$\begin{aligned} F(x) = y &\iff \frac{1}{\pi} \arcsin x + \frac{1}{2} = y \\ &\iff x = \sin \left(\pi \left(y + \frac{1}{2} \right) \right) \end{aligned}$$

Donc F est bijective d'inverse :

$$F^{-1} = x \mapsto \sin \left(\pi \left(y + \frac{1}{2} \right) \right)$$

1.2.2 Méthode du rejet (Cas continu)

On souhaite simuler une variable aléatoire X de fonction de densité f que l'on ne sait pas bien simuler. On va donc utiliser une autre fonction auxiliaire g que l'on sait bien simuler.

Théorème (Méthode du rejet) . Soient f, g deux fonctions de densité telles qu'il existe $c > 0$ vérifiant :

$$f \leq cg$$

Soient $(Y_i)_{i \geq 1}$ des variables iid de densité g et $(U_i)_{i \geq 1}$ des variables iid de loi $\mathcal{U}([0, 1])$.

Posons $T = \inf \left\{ n \geq 1 \mid U_n \leq \frac{f(Y_n)}{cg(Y_n)} \right\}$. Alors :

1. Y_T a pour densité f
2. T est de loi géométrique $\mathcal{G}(1/c)$
3. T et Y_T sont indépendantes.

Proposition On peut donc ensuite simuler la variable aléatoire de densité f grâce à la densité de g avec l'algorithme suivant :

```

1 def simul_f():
2     y = simul_g()
3     u = rand()
4     while u > f(y) / (c * g(y)):
5         y = simul_g()
6         u = rand()
7     return y
8
```

Démonstration Soient φ, f deux fonctions continues et bornées sur $\mathbb{R}$. Montrons que Y_T est une variable aléatoire de densité f :

$$\begin{aligned}
\mathbb{E}(\varphi(Y_T)) &= \mathbb{E}\left(\sum_{n=1}^{\infty} \mathbb{1}_{T=n} \varphi(Y_T)\right) \\
&= \sum_{n=1}^{\infty} \mathbb{E}(\varphi(Y_T) \times \mathbb{1}_{T=n}) \\
&= \sum_{n=1}^{\infty} \mathbb{E}(\varphi(Y_n) \times \mathbb{1}_{T=n}) \\
&= \sum_{n=1}^{\infty} \mathbb{E}\left(\varphi(Y_n) \times \mathbb{1}_{U_n \leq \frac{f(Y_n)}{cg(Y_n)}} \times \mathbb{1}_{U_{n-1} > \frac{f(Y_{n-1})}{cg(Y_{n-1})}} \times \cdots \times \mathbb{1}_{U_1 > \frac{f(Y_1)}{cg(Y_1)}}\right) \\
&= \sum_{n=1}^{\infty} \left(\mathbb{E}\left(\varphi(Y_n) \mathbb{1}_{U_n \leq \frac{f(Y_n)}{cg(Y_n)}}\right) \mathbb{P}\left(U_{n-1} > \frac{f(Y_{n-1})}{cg(Y_{n-1})}\right) \cdots \mathbb{P}\left(U_1 > \frac{f(Y_1)}{cg(Y_1)}\right)\right)
\end{aligned}$$

or on a :

$$\begin{aligned}
\mathbb{P}(U_1 > \frac{f(Y_1)}{cg(Y_1)}) &= \mathbb{E}\left(\mathbb{P}\left(U_1 > \frac{f(Y_1)}{cg(Y_1)} \mid Y_1\right)\right) \\
&= \mathbb{E}\left(1 - \frac{f(Y_1)}{cg(Y_1)}\right) = 1 - \mathbb{E}\left(\frac{f(Y_1)}{cg(Y_1)}\right) \\
&= 1 - \int_{\mathbb{R}} \frac{f(y)}{cg(y)} \times g(y) dy = 1 - \frac{1}{c}
\end{aligned}$$

de plus :

$$\begin{aligned}
&\mathbb{E}\left(\varphi(Y_n) \times \mathbb{1}_{U_n \leq \frac{f(Y_n)}{cg(Y_n)}}\right) \\
&= \mathbb{E}\left(\varphi(Y_n) \frac{f(Y_n)}{cg(Y_n)}\right) = \int_{\mathbb{R}} g(y) \varphi(y) \frac{f(y)}{cg(y)} dy \\
&= \frac{1}{c} \int_{\mathbb{R}} \varphi(y) f(y) dy
\end{aligned}$$

enfin :

$$\begin{aligned}
\mathbb{E}(\varphi(y)) &= \sum_{n=1}^{\infty} \mathbb{E}\left(\varphi(Y_n) \times \mathbb{1}_{U_n \leq \frac{f(Y_n)}{cg(Y_n)}}\right) \times \left(1 - \frac{1}{c}\right)^{n-1} \\
&= \left(\int_{\mathbb{R}} \varphi(y) f(y) dy\right) \times \frac{1}{c} \times \left(\sum_{n=1}^{\infty} \left(1 - \frac{1}{c}\right)^{n-1}\right) = \int_{\mathbb{R}} \varphi(y) f(y) dy
\end{aligned}$$

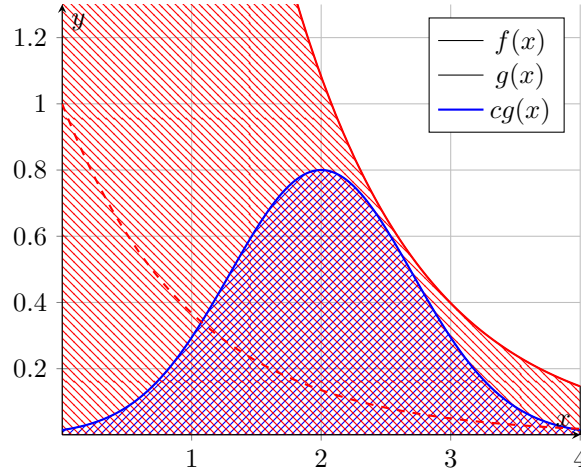
Donc Y_n est une variable aléatoire de densité f . Calculons la loi de T :

$$\begin{aligned}
\mathbb{P}(T > n) &= \mathbb{P}\left(U_1 > \frac{f(Y_1)}{cg(Y_1)}, \dots, U_n > \frac{f(Y_n)}{cg(Y_n)}\right) \\
&= \mathbb{P}\left(U_1 > \frac{f(Y_1)}{cg(Y_1)}\right)^n = \left(1 - \frac{1}{c}\right)^n
\end{aligned}$$

d'où $T \sim \mathcal{G}(1/c)$. Pour finir, montrons que T et Y_T sont indépendantes :

$$\begin{aligned}
\mathbb{E}(\varphi(Y_T) \times \mathbb{1}_{T=n}) &= \mathbb{E}(\varphi(Y_T)) \times \frac{1}{c} \times \left(1 - \frac{1}{c}\right)^{n-1} \\
&= \mathbb{E}(\varphi(Y_T)) \times \mathbb{P}(T = n)
\end{aligned}$$

Remarque (Méthode du rejet : Principe) Comme dit précédemment, la méthode du rejet nous permet de simuler une densité f à partir d'une autre densité g que l'on sait simuler. Pour cela, on supposera qu'il existe $c > 0$ tel que $f \leq cg$.



Soit $X \sim g$ et $U \sim \mathcal{U}([0, 1])$. L'idée est donc de tirer un point aléatoirement selon la densité g , $(X, U \times cg(X))$ dans la zone rouge. Si la valeur $U \times cg(X)$ est sous la courbe de f (i.e dans la zone bleu), on accepte le point, sinon on recommence. La probabilité d'acceptation d'un point est donc de $1/c$.

Exemple Soit f la fonction suivante :

$$f : \begin{cases} \mathbb{R}_+^* \rightarrow \mathbb{R} \\ x \mapsto \frac{\sqrt{2}}{\pi} e^{x^2/2} \end{cases}$$

et g la fonction $g : x \mapsto -e^x$ définie sur $\mathbb{R}_+^*$. Pour tout $x \in \mathbb{R}_+^*$, on a :

$$f(x) \leq cg(x) \quad \text{où} \quad c = \frac{\sqrt{2}}{\pi} e$$

On peut donc simuler f via l'algorithme suivant :

```

1 def simul_f():
2     u = rand()
3     y = -log(1 - rand())
4     while u > exp(-y**2/2) / exp(1 - y):
5         u = rand()
6         y = -log(1 - rand())
7     return y
8 
```

1.3 Méthodes ad-hoc

Dans cette section, nous allons détailler un ensemble de méthodes destinées à des usages précis. Nous verrons comment simuler efficacement une loi normale une loi de poisson ainsi qu'un mélange de lois. Nous finirons par aborder la méthode de Monte-Carlo pour exprimer des quantités qui s'expriment à partir d'intégrales.

1.3.1 Loi Normale - Méthode de Box-Muller

Rappel (Loi Normale) Une variable aléatoire X suit une *loi normale* de moyenne μ et de variance σ^2 si la densité de la loi sur $\mathbb{R}$ est :

$$x \mapsto \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

On note alors $X \sim \mathcal{N}(\mu, \sigma^2)$ et on a :

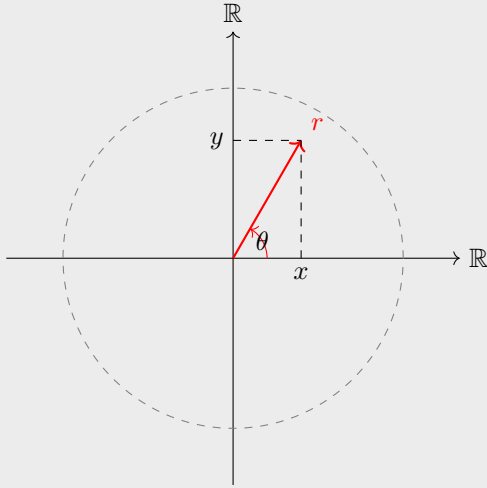
$$\mathbb{E}(X) = \mu \quad \text{et} \quad \mathbb{V}(X) = \sigma^2$$

Propriété (Loi centrée réduite.) . Si $Y \sim \mathcal{N}(0, 1)$, alors :

$$X = \mu + \sigma Y \sim \mathcal{N}(\mu, \sigma^2)$$

On dit alors que Y suit une *loi normale centrée réduite*.

Théorème (Méthode de Box-Muller) . Soit (X, Y) un couple de variables aléatoires iid de loi $\mathcal{N}(0, 1)$. Alors $X^2 + Y^2$ est une variable aléatoire de loi $\mathcal{E}(1/2)$ indépendante de $\arctan(Y/X) \sim \mathcal{U}([-\frac{\pi}{2}; \frac{\pi}{2}])$.



Le théorème nous dit que θ est indépendant de r .

Démonstration Soient (X, Y) deux variables aléatoires indépendantes de loi $\mathcal{N}(0, 1)$. Calculons l'espérance du couple :

$$\mathbb{E}(g(X, Y)) = \frac{1}{2\pi} \int_{\mathbb{R}^2} g(x, y) e^{-\frac{x^2}{2}} e^{-\frac{y^2}{2}} dx dy$$

Pour cela, utilisons la formule de changement de variable. Posons :

$$\Phi : \begin{cases} U = R_+ \times [0, 2\pi[\longrightarrow V = \mathbb{R}^2 \\ (r, \theta) \mapsto (x, y) = (r \cos \theta, r \sin \theta) \end{cases}$$

Φ est bijective de $R_+^* \times [0, 2\pi[$ dans $\mathbb{R}^2 / \{(0, 0)\}$ donc c'est un $\mathcal{C}^1$ difféomorphisme. On peut donc appliquer la formule de changement de variable :

$$\int_V f(x, y) dx dy = \int_U f(\Phi(r, \theta)) \times |\det J_\Phi(r, \theta)| dr d\theta$$

On a donc :

$$\Phi(r, \theta) = \begin{pmatrix} r \cos \theta \\ r \sin \theta \end{pmatrix} \quad \text{d'où} \quad J_\Phi(r, \theta) = \begin{pmatrix} \cos \theta & -r \sin \theta \\ \sin \theta & r \cos \theta \end{pmatrix}$$

Avec $D_\Phi(r, \theta) = \det J_\Phi(r, \theta) = r$. D'où :

$$\begin{aligned} \mathbb{E}(g(X, Y)) &= \frac{1}{2\pi} \int_{\mathbb{R}^2} g(x, y) e^{-\frac{x^2}{2}} e^{-\frac{y^2}{2}} dx dy \\ &= \frac{1}{2\pi} \int_0^{2\pi} \int_0^\infty g(r \cos \theta, r \sin \theta) \times e^{-\frac{r^2}{2}} \times r dr d\theta \end{aligned}$$

Posons donc $z = r^2$ on a donc $dz = 2r dr$ d'où :

$$\begin{aligned} \mathbb{E}(g(X, Y)) &= \frac{1}{4\pi} \int_0^{2\pi} \int_0^\infty g(\sqrt{z} \cos \theta, \sqrt{z} \sin \theta) e^{-z/2} dz d\theta \\ &= \int_0^{2\pi} \int_0^\infty g(\sqrt{z} \cos \theta, \sqrt{z} \sin \theta) f_z(z) f_\theta(\theta) dz d\theta \end{aligned}$$

où :

$$\begin{cases} f_z(z) = \frac{1}{2} e^{-z/2} \\ f_\theta(\theta) = \frac{1}{2\pi} \end{cases} \implies \begin{cases} Z \sim \mathcal{E}(1/2) \\ \Theta \sim \mathcal{U}([0, 2\pi]) \end{cases}$$

On en déduit donc que $\Theta \perp Z$.

Proposition On peut donc simuler deux lois normales à partir d'une loi exponentielle et d'une loi uniforme via l'algorithme suivant :

```
1 def simuler_N():
2     u = 2 * pi * rand()
3     e = -2 * log(rand())
4     return sqrt(e) * cos(u)
5
```

Ou plus généralement :

```
1 def simuler_N(mu, sigma):
2     u = 2 * pi * rand()
3     e = -2 * log(rand())
4     return sqrt(sigma**2 * e) * cos(u + mu)
5
```

1.3.2 Lois de Poisson

Rappel (Loi de Poisson) Une variable aléatoire X suit une loi de Poisson de paramètre λ si $\forall k \in \mathbb{N}$:

$$\mathbb{P}(X = k) = e^{-\lambda} \frac{\lambda^k}{k!}$$

Propriété (Loi de Poisson et loi Exponentielle) . Soit $(E_n)_{n \geq 1}$ une suite de variables aléatoires iid de loi $\mathcal{E}(1)$. On note $S_n = \sum_{k=1}^n E_k$. Posons :

$$N = \max \{k \in \mathbb{N} \mid S_k < \lambda\}$$

Alors N suit une loi de Poisson de paramètre λ .

Démonstration Démontrons ce résultat par récurrence sur $\mathbb{N}$:

— Initialisation : On observe que :

$$\mathbb{P}(N = 0) = \mathbb{P}(E_1 > \lambda) = e^{-\lambda}$$

— Hérédité : Soit $k \geq 1$, on suppose que :

$$\mathbb{P}(N = k - 1) = \frac{\lambda^{k-1}}{(k-1)!} e^{-\lambda} \quad \forall \lambda \in \mathbb{R}_+$$

En utilisant le fait que pour tout évènement A et pour toute partie mesurable $\mathcal{F}$, on a :

$$\mathbb{P}(A) = \mathbb{E}(\mathbb{1}_A) = \mathbb{E}(\mathbb{E}(\mathbb{1}_A | \mathcal{F})) = \mathbb{E}(\mathbb{P}(A | \mathcal{F}))$$

alors :

$$\mathbb{P}(N = k) = \mathbb{P}(S_k < \lambda < S_{k+1}) = \mathbb{E}(\mathbb{P}(S_k < \lambda < S_{k+1} | E_1))$$

or :

$$\begin{aligned} \mathbb{P}(S_k < \lambda < S_{k+1}) &= \mathbb{P}(E_1 + \dots + E_k < \lambda < E_1 + \dots + E_{k+1} | E_1) \\ &= \mathbb{P}(E_2 + \dots + E_k < \lambda - E_1 < E_2 + \dots + E_{k+1} | E_1) \\ &= \mathbb{P}(S'_{k-1} < \lambda - E_1 < S_{k+1} | E_1) \\ &= \mathbb{P}(N' = k - 1) \end{aligned}$$

où S' est une copie indépendante de S et indépendante de E_1 , d'où :

$$\mathbb{P}(N' = k - 1) = \frac{(\lambda - E_1)^{k-1}}{(k-1)!} e^{-(\lambda - E_1)}$$

On a donc :

$$\begin{aligned} \mathbb{P}(N = k) &= \mathbb{E} \left(\frac{(\lambda - E_1)^{k-1}}{(k-1)!} e^{-(\lambda - E_1)} \times \mathbb{1}_{E_1 \leq \lambda} \right) \\ &= \int_0^\lambda \frac{(\lambda - e)^{k-1}}{(k-1)!} \underbrace{e^{-(\lambda - e)} e^{-e}}_{e^{-\lambda}} de \\ &= \frac{e^{-\lambda}}{(k-1)!} \int_0^\lambda (\lambda - e)^{k-1} de \\ &= \frac{e^{-\lambda}}{(k-1)!} \times \frac{\lambda^k}{k} = \frac{\lambda^k}{k!} e^{-\lambda} \end{aligned}$$

D'où $N \sim \mathcal{P}(\lambda)$.

Proposition Pour simuler une loi de poisson de paramètre λ , on a donc l'algorithme suivant :

```

1 def simuler_poisson(lambda):
2     s = 1 * log(rand())
3     k = 0
4     while s <= lambda :
5         k += 1
6         s = -log(rand())
7     return k
8 
```

Algorithme donc la complexité dépend linéairement de λ .

Remarque Quand λ revient très grand, on peut approcher une loi de Poisson de paramètre λ par une loi normale $\mathcal{N}(\lambda, \lambda)$.

1.3.3 Mélange de lois

On cherche ici à déterminer la loi d'une combinaison linéaire de loi μ_1 et μ_2 de la forme :

$$\mu = t\mu_1 + (1 - t)\mu_2$$

Pour simuler ce genre de variables aléatoires, on utilise le résultat suivant.

Lemme Soit $X \sim \mu_1$ et $Y \sim \mu_2$ et $B \sim \mathcal{B}(t)$ avec $B \perp (X, Y)$. On a alors :

$$BX + (1 - B)Y \sim t\mu_1 + (1 - t)\mu_2$$

Attention, ici X et Y ne sont pas forcément indépendantes.

Démonstration Calculons la loi de $BX + (1 - B)Y$. Soit $f \in \mathcal{M}_+(\mathbb{R}, \mathcal{B}_{\mathbb{R}})$ on a :

$$\begin{aligned} \mathbb{E}(g(BX + (1 - B)Y)) &= \mathbb{E}[\mathbb{E}(g(BX + (1 - B)Y)) \mid B] \\ &= \mathbb{E}[g(X) \times \mathbb{1}_{B=1} + g(Y) \times \mathbb{1}_{B=0}] \\ &= t\mathbb{E}(g(X)) + (1 - t)\mathbb{E}(g(Y)) \\ &= t \int g \, d\mu_1 + (1 - t) \int g \, d\mu_2 \\ &= \int g \, d_{t\mu_1 + (1-t)\mu_2} \end{aligned}$$

d'où $BX + (1 - B)Y \sim t\mu_1 + (1 - t)\mu_2$.

Proposition On a donc l'algorithme suivant :

```

1 def simuler_mu(t):
2     if rand() < t :
3         return simuler_mu1()
4     else :
5         return simuler_mu2()
6 
```

1.3.4 Méthode de Monte-Carlo

Pour la fin de ce chapitre, on souhaite simuler des variables aléatoires pour estimer des quantités déterministes pouvant s'écrire sous forme d'intégrale.

Principe Soit $I = \int_a^b f(x) \, dx$ une intégrale que l'on se sait pas calculer algébriquement mais que l'on souhaite estimer. On peut alors réécrire :

$$I = \int_a^b \frac{f(x)}{g(x)} g(x) \, dx$$

où g est une densité de probabilité d'une variable que l'on sait simuler. Or, d'après la loi des grands nombres, on a :

$$\hat{I}_n = \frac{1}{n} \sum_{i=1}^n \frac{f(X_i)}{g(X_i)} \xrightarrow[n \rightarrow \infty]{\text{p.s.}} \mathbb{E} \left(\frac{f(X)}{g(X)} \right)$$

où $(X_1, \dots, X_n)$ sont des variables aléatoires iid de densité g . Ainsi, on va pouvoir estimer toute intégrale de la forme :

$$I \simeq \mathbb{E} \left(\frac{f(X_1)}{g(X_1)} \right)$$

par un estimateur convergent $\hat{I}_n$.

Théorème (Rappel - Loi des grands nombres) . Soit (X_n) une suite de variables aléatoires iid et intégrables. On a alors :

$$\frac{1}{n} \sum_{i=1}^n X_i \xrightarrow[n \rightarrow \infty]{} \mathbb{E}(X_1)$$

Remarque On peut aussi approcher I par une méthode de Riemann :

$$I = \frac{1}{n} \sum_{i=1}^n f\left(a + \frac{\tau}{n}\right)$$

or elle ne nous permet d'estimer l'intégrale que pour des fonctions Riemann-intégrables. Alors que la Loi des grands nombres (LGN) nous permet d'estimer des intégrales de fonctions mesurables. De plus, les méthodes déterministes sont très lentes en plusieurs dimension comparé aux méthodes stochastiques.

Théorème (Rappel - Théorème Central Limite) . Soit (X_n) une suite de variables iid de carré intégrable. On a alors :

$$\sqrt{n} \left(\frac{1}{n} \sum_{i=1}^n X_i - \mathbb{E}(X_1) \right) \xrightarrow[n \rightarrow \infty]{\text{Loi}} \mathcal{N}(0, \mathbb{V}(X_1))$$

Autrement dit :

$$\lim_{n \rightarrow \infty} \mathbb{P} \left(\left| \frac{1}{n} \sum_{i=1}^n X_i - \mathbb{E}(X_1) \right| > \frac{t}{\sqrt{n}} \right) = \int_t^\infty \frac{e^{-\frac{y^2}{2\mathbb{V}(X_1)}}}{\sqrt{2\pi\mathbb{V}(X_1)}} dy$$

Rappel Soit (X_n) une suite de variables iid de carré intégrable. On note $\overline{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ et on pose :

$$S_n^2 := \frac{1}{n} \sum_{i=1}^n (X_i - \overline{X}_n)^2$$

D'après la loi des grands nombres, on a :

$$\overline{X}_n \xrightarrow[n \rightarrow \infty]{} \mathbb{E}(X_1)$$

D'après le théorème central limite, on a :

$$\sqrt{n} \frac{\overline{X}_n - \mathbb{E}(X_1)}{\sqrt{S_n^2}} \xrightarrow[n \rightarrow \infty]{} \mathcal{N}(0, 1)$$

avec : $\mathbb{P}(I \in IC_n) \xrightarrow[n \rightarrow \infty]{} \alpha$.

Proposition On peut donc utiliser le TCL pour proposer des intervalles de confiance asymptotiques pour la valeur de I . On a donc , pour $I = \mathbb{E}(X_1)$, l'intervalle asymptotique de niveau α :

$$IC_n = \left[\overline{X}_n \pm \frac{q_{1-\alpha/2}}{\sqrt{n}} \sqrt{S_n^2} \right]$$

Exemple On souhaite calculer :

$$I = \int_{[0,1]^3} \mathbb{1}_{\substack{x_1 \leq \cos x_2 \sin x_3 \\ x_2 \leq x_3^2}} dx_1 dx_2 dx_3$$

D'après la méthode de Monte-Carlo, soit $f \in \mathcal{M}_+(\mathbb{R}, \mathcal{B}_{\mathbb{R}})$ telle que :

$$I = \mathbb{E}(f(Z))$$

où $f(Z) = \mathbb{1}_{\substack{z_1 \leq \cos z_2 \sin z_3 \\ z_2 \leq z_3^2}}$ et $Z = (z_1, z_2, z_3) \sim \mathcal{U}([0, 1]^3)$. Autrement dit :

$$I = \mathbb{P}(z_1 \leq \cos z_2 \sin z_3, z_2 \leq z_3^2)$$

On peut donc approcher I par :

$$\frac{1}{n} \sum_{i=1}^n \mathbb{1}_{\substack{z_1 \leq \cos z_2 \sin z_3 \\ z_2 \leq z_3^2}} = \hat{I}_n$$

D'après le TCL, on a :

$$\begin{aligned} \lim_{n \rightarrow \infty} \sqrt{n} \frac{\hat{I}_n - I}{\sqrt{I(1-I)}} &= \mathcal{N}(0, 1) \\ \text{i.e. } \lim_{n \rightarrow \infty} \sqrt{n} \frac{\hat{I}_n - I}{\sqrt{\hat{I}(1-\hat{I})}} &= \mathcal{N}(0, 1) \end{aligned}$$

On peut donc construire l'intervalle de confiance de niveau α :

$$\left[\hat{I}_n \pm \frac{\sqrt{\hat{I}_n(1-\hat{I}_n)}}{\sqrt{n}} q_\alpha \right] \subseteq \left[\hat{I}_n \pm \frac{q_\alpha}{2\sqrt{n}} \right]$$

Chapitre 2

Simulation de chaînes de Markov

Contents

2.1	Généralités	43
2.1.1	Définition et exemples	43
2.1.2	La simulation en pratique	44
2.2	Comportement asymptotique d'une chaîne de Markov	46
2.3	Méthode MCMC	48
2.3.1	Principe général	48
2.3.2	Méthode par batches (lots)	50
2.3.3	Méthode de Newley-West	51

Dans ce chapitre, nous nous intéresserons à la simulation de processus de Markov grâce à l'étude de celles-ci. Pour rappel, une chaîne de Markov décrit l'évolution d'une quantité soumise à des fluctuations aléatoires. C'est une notion plus générale que les variables i.i.d.

2.1 Généralités

2.1.1 Définition et exemples

Définition (Chaîne de Markov) . Une chaîne de Markov est un processus aléatoire $(X_n)_{n \geq 1}$ défini sur un espace E tel qu'il existe $f \in \mathcal{M}(E \times [0, 1], E)$ et $(u_n)_{n \geq 0}$ une suite iid de loi $\mathcal{U}([0, 1])$ telle que :

$$\forall n \geq 1, \quad X_{n+1} = f(X_n, u_{n+1})$$

avec :

$$X_0 \perp (u_n)_{n \geq 0}$$

Exemple Soit $P \in \mathcal{M}_n(\mathbb{R})$ une matrice stochastique. Soit une chaîne de Markov sur un espace $E = \llbracket 1, n \rrbracket$ de matrice de transition P . Alors c'est bien une chaîne de Markov au sens de la définition précédente. En effet, on pose :

$$\forall i, j \in E, \quad f(i, x) = j \quad \text{avec} \quad x \in \left[\sum_{k=1}^{j-1} P(i, k); \sum_{k=1}^j P(i, k) \right]$$

On aura donc :

$$\begin{aligned}\mathbb{P}(X_0 = x_0, \dots, X_n = x_n) \\ &= \mathbb{P}(X_0 = x_0, f(x_0, u_1) = x_1, \dots, f(x_{n-1}, u_n) = x_n) \\ &= \mathbb{P}(X_0 = x_0) \mathbb{P}(f(x_0, u_1) = x_1) \dots \mathbb{P}(f(x_{n-1}, u_n) = x_n) \\ &= \mathbb{P}(X_0 = x_0) P(x_0, x_1) \times \dots \times P(x_{n-1}, x_n)\end{aligned}$$

Exemple Soit le processus $(X_n)_{n \in \mathbb{N}}$ décrivant l'évolution de la température locale au cours du temps défini par :

$$X_0 = 25 \quad X_{n+1} = \frac{X_n + 25}{2} + Z_{n+1}$$

où $(Z_n)_{n \in \mathbb{N}}$ est une suite de variables aléatoires i.i.d de loi $\mathcal{N}(0, 10)$. Alors ce processus est une chaîne de Markov avec :

$$f(x, u) = \frac{x + 25}{2} + \Phi^{-1}(u)$$

avec $N \sim \mathcal{N}(0, 10)$ et :

$$\Phi(u) = \mathbb{P}(N \leq u) = \left(\int_{-\infty}^u e^{-x^2/20} dx \right) \times \frac{1}{\sqrt{20\pi}}$$

Remarque Cet exemple nous pose une problématique. En effet, Nous avons défini notre chaîne de Markov avec seulement une suite de variables i.i.d de loi uniforme et chaque terme de la chaîne ne dépend que d'une seule réalisation de cette loi uniforme. Pour nous permettre de faire dépendre un X_n d'une loi normale, nous devrions pouvoir utiliser deux variables uniforme pour la simuler (par la méthode de Box-Muller).

Le lemme suivant nous l'assure donc en affirmant que l'on peut produire autant de loi uniformes à partir d'une seule.

2.1.2 La simulation en pratique

Lemme (Doublement de la loi uniforme) Il existe $g : [0, 1] \rightarrow [0, 1]^2$ mesurable telle que si $U \sim \mathcal{U}([0, 1])$ et en posant $(V, W) = g(U)$ alors :

$$V \perp W \quad \text{et} \quad W, V \sim \mathcal{U}([0, 1])$$

Corollaire (Génération de lois uniformes) . Il existe une fonction mesurable $h : [0, 1] \rightarrow [0, 1]^{\mathbb{N}}$ telle que $h(U)$ est une suite de variables iid de loi $\mathcal{U}([0, 1])$.

Remarque Ces deux derniers résultats nous permettent d'écrire :

$$X_{n+1} = f(X_n, Z_{n+1,1}, \dots, Z_{n+1,k})$$

avec $(Z_{n,k})_{\substack{k \geq 1 \\ n \geq 1}}$ une matrice de suites i.i.d. On a alors que $(X_n)_{n \in \mathbb{N}}$ est une chaîne de Markov.

Exemple (Processus de Galton-Watson) Soit $(X_{n,k})_{n \geq 1, k \geq 1}$ un tableau de variables aléatoires i.i.d à valeurs dans $\mathbb{N}$. Le processus défini par :

$$Z_0 = 1 \quad \text{et} \quad Z_{n+1} = \sum_{j=1}^{Z_n} X_{n+1,j}$$

est une chaîne de Markov. C'est un exemple de processus représentant l'évolution d'une population au cours du temps. Ici, Z_n représente le nombre d'individus à la génération n et $X_{n+1,j}$ le nombre de descendants de j -ème individu de cette génération. Z_{n+1} est donc le nombre d'individus créés à la génération $n + 1$.

Notation. Soit $(X_n)_{n \geq 1}$ une chaîne de Markov. Pour tout $x \in E$, on pose $\mathbb{P}_x$ la loi de $(X_n)_{n \geq 1}$ conditionnellement à $X_0 = x$. On a donc :

$$\begin{cases} X_0 = x \\ X_{n+1} = f(X_n, u_{n+1}) \end{cases}$$

Propriété (Markov faible) . Soit $(X_n)_{n \geq 1}$ une chaîne de Markov et $N \in \mathbb{N}$, alors :

$$\mathbb{E}(h(X_{N+k}, \dots, X_{N+1}) \mid X_N, \dots, X_0) = \mathbb{E}_{X_N}(h(X_k, \dots, X_2, X_1))$$

Autrement dit, à partir du moment où l'on connaît X_N , les événements $X_{N+1}, \dots, X_{N+k}$ ne dépendent plus du passé.

Démonstration Cette propriété est juste une reformulation de la propriété de Markov vue en probabilités. En effet, d'après la propriété de Markov faible, on sait que, étudier la chaîne $X_{N+k+1}, \dots, X_{N+1}$ sachant que l'on est passé par $X_N, \dots, X_0$ revient à étudier la chaîne décalée $X_k, \dots, X_1$ sachant que l'on est parti de X_N . Plus formellement :

$$\begin{aligned} \mathbb{P}(X_{N+k} = i_{N+k}, \dots, X_{N+1} = i_{N+1} \mid X_N = i_N, \dots, X_0 = i_0) \\ = \mathbb{P}(X_k = i_k, \dots, X_1 = i_1 \mid X_N = i_N) \end{aligned}$$

Donc en terme d'espérance :

$$\begin{aligned} \mathbb{E}(h(X_{N+k}, \dots, X_{N+1}) \mid X_N, \dots, X_0) \\ = \sum_{i_1, \dots, i_k \in E} h(i_k, \dots, i_1) \times \mathbb{P}(X_{N+k} = i_k, \dots, X_{N+1} = i_1 \mid X_N, \dots, X_0) \\ = \sum_{i_1, \dots, i_k \in E} h(i_k, \dots, i_1) \times \mathbb{P}(X_k = i_k, \dots, X_1 = i_1 \mid X_N) \\ = \mathbb{E}(h(X_k, \dots, X_1) \mid X_N) \\ = \mathbb{E}_{X_N}(h(X_k, \dots, X_1)) \end{aligned}$$

Exemple On définit par récurrence :

$$F_0 = 1, F_1 = 1 \quad \text{et} \quad F_{n+1} = F_n + F_{n-1} + \varepsilon_{n+1}$$

avec $(\varepsilon_n)_{n \in \mathbb{N}}$ une suite de variables aléatoires iid de loi $\mathcal{U}(\{-1, 0, 1\})$. La suite $(F_n)_{n \in \mathbb{N}}$ représente la suite de Fibonacci pour laquelle on ferait des erreurs de calculs. Cette suite n'est pas une chaîne de Markov car :

$$\mathbb{P}(F_4 = 2 \mid F_3 = 1) \neq \mathbb{P}(F_4 = 2 \mid F_3 = 1, F_2 = 1)$$

elle ne satisfait pas la propriété de Markov. Par contre, le couple $(F_n, F_{n-1})_{n \geq 1}$ est une chaîne de Markov avec :

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = f \left(\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}, \varepsilon_{n+1} \right)$$

avec :

$$f : \begin{cases} \mathbb{R}^2 \times \mathbb{R} \longrightarrow \mathbb{R}^2 \\ (x, y, \varepsilon) \longmapsto (x + y + \varepsilon, x) \end{cases}$$

Remarque Soit $(Z_n)_{n \geq 1}$ une suite de variables aléatoires iid et :

$$S_n = Z_1 + \dots + Z_n$$

alors :

- S_n est une chaîne de Markov
- $\max\{S_k, k \leq n\}$ n'est pas une chaîne de Markov.
- $(S_n, \max\{S_k, k \leq n\})$ est une chaîne de Markov.

Proposition (Algorithme de simulation) Comme dit précédemment, une chaîne de Markov est définie telle que :

$$\begin{cases} X_0 = x_0 \\ X_{n+1} = f(X_n, u_{n+1}) \end{cases}$$

On peut donc utiliser l'algorithme suivant pour les simuler :

```

1 def simul_chaine_Markov(n, x0, f):
2     traj = [x0]
3     for i in range(n):
4         traj.append(f(traj[-1], rand()))
5     return traj

```

Remarque La principale difficulté est donc de définir la fonction f qui caractérise notre chaîne de Markov.

2.2 Comportement asymptotique d'une chaîne de Markov

L'utilisation des chaînes de Markov en simulation est liée au théorème suivant.

Théorème (Ergodique de Birkhoff) . Soit $(X_n)_{n \geq 1}$ une chaîne de Markov. Si cette chaîne admet une mesure de probabilités π invariante et ergodique alors $\forall f \in L^1(\pi)$, on a :

$$\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n f(X_i) = \int f d\pi \quad \text{p.s}$$

Par conséquent, pour calculer $I = \int f d\pi$ avec π une mesure coûteuse à simuler mais une mesure invariante et ergodique d'une chaîne de Markov, on peut approcher I par $\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n f(X_i)$. Évidemment, puisque les variables X_i ne sont pas i.i.d, la convergence a lieu à un rythme plus lent. Sous certaines conditions, on peut avoir :

$$\sqrt{n} \left(\frac{1}{n} \sum_{i=1}^n f(X_i) - \int f d\pi \right) \xrightarrow[n \rightarrow \infty]{} \mathcal{N}(0, 1)$$

Ce théorème peut être vu comme une généralisation de la loi des grands nombres aux chaînes de Markov.

Définition (Mesure Invariante) . Soient un espace mesurable $(E, \mathcal{F})$ et $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov et $u \sim \mathcal{U}([0, 1])$. Soit π une mesure sur $(E, \mathcal{F})$. On dit que π est *invariante* si pour tout $A \in \mathcal{F}$ on a :

$$\begin{aligned} \int_E \mathbb{P}_x(X_1 \in A) d\pi(x) &= \pi(A) \\ \iff \int_E \mathbb{P}(f(x, u_1) \in A) d\pi(x) &= \pi(A) \end{aligned}$$

Si π est une *mesure de probabilité*, cette définition est équivalente à :

$$X_0 \sim \pi \quad \text{et} \quad U_1 \sim \mathcal{U}([0, 1]) \implies X_1 = f(X_0, U_1) \sim \pi$$

Exemple Reprenons notre exemple de simulation de l'évolution de la température :

$$X_{n+1} = \frac{X_n + 25}{2} + Z_n \quad \text{avec} \quad Z_n \sim \mathcal{N}(0, 1)$$

Si $X_0 \sim \mathcal{N}(\mu, \sigma^2)$ alors :

$$X_1 = \frac{X_0 + 25}{2} + Z_1 \sim \mathcal{N}$$

comme combinaison de gaussiennes indépendantes. De plus on a :

$$\mathbb{E}(X_1) = \frac{25 + \mu}{2} \quad \text{et} \quad \mathbb{V}(X_1) = \frac{\sigma^2}{4} + 10$$

On a donc $X_1 \sim \mathcal{N}\left(\frac{25+\mu}{2}, \frac{\sigma^2}{4} + 10\right)$. La loi $\mathcal{N}(\mu, \sigma^2)$ est donc invariante si :

$$\mu = \frac{25 + \mu}{2} \quad \text{et} \quad \sigma^2 = \frac{\sigma^2}{4} + 10$$

$$\iff \mu = 25, \sigma^2 = \frac{40}{3}$$

Remarque Attention, si $X_0 \sim \pi$ avec π une mesure invariante, alors $\forall n \in \mathbb{N}^*, X_n \sim \pi$ mais $X_n \not\sim X_{n-1}$ en général.

Définition (Mesure Ergodique) . Une mesure π sur un espace mesurable $(E, \mathcal{F})$ est dite *ergodique* pour une chaîne de Markov $(X_n)_{n \in \mathbb{N}}$ si pour tout $A \in \mathcal{F}$ on a :

$$\boxed{\forall x \in E, \quad \mathbb{P}_x(X_1 \in A) = \mathbb{1}_{x \in A} \implies \pi(A) = 0 \text{ ou } \pi(\bar{A}) = 0}$$

Une mesure ergodique est une mesure sur E qui ne peut pas "partager" sa masse entre différents sous-ensembles invariants pour la chaîne. Autrement dit, pour tout $A \subset E$ tel que, partant de A , la chaîne reste dans A et partant de $\bar{A}$, la chaîne reste dans $\bar{A}$, la mesure ergodique charge soit A , soit $\bar{A}$, mais pas les deux.

Pour une chaîne de Markov dans E , cela implique que la chaîne ne peut pas rester bloquée entre plusieurs sous-ensembles invariants : elle explore tous les états accessibles dans l'ensemble invariant où elle a démarré, et finit par "oublier" son point de départ.

Remarque Si une chaîne de Markov n'est pas ergodique, cela signifie qu'il existe au moins un sous-ensemble invariant non trivial $A \subset E$ tel que :

- partant de A , la chaîne reste dans A ;
- partant de A^c , la chaîne reste dans A^c .

Une mesure non invariante peut alors mettre du poids à la fois sur A et sur A^c . En pratique, cela signifie que la chaîne peut rester confinée dans différents "blocs" selon l'état initial, et donc qu'elle ne se mélange pas complètement.

Propriété (Ergodicité et Irréductibilité) . Pour les chaînes de Markov sur des espaces d'états discrets, être *ergodique* est équivalent à être *irréductible*.

Corollaire (Sur les espaces d'états discrets) . Si E est un espace d'états discret, alors toute chaîne de Markov irréductible est μ -ergodique pour toute mesure de probabilité μ .

Théorème (Birkhoff - Variante) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov de mesure invariante et ergodique π . Soient $f, g \in L^1(\pi)$ on a alors :

$$\lim_{n \rightarrow \infty} \frac{\sum_{i=1}^n f(X_i)}{\sum_{i=1}^n g(X_i)} = \frac{\int f d\pi}{\int g d\pi} \quad \text{p.s.}$$

Intuitivement, puisque qu'une chaîne ergodique se "mélange bien" dans son espace d'état, alors les fréquences empiriques (la moyenne des trajectoires) reflètent bien les fréquences théoriques (la mesure π) de la chaîne de Markov.

Exemple Soit $(S_n)_{n \geq 1}$ la marche aléatoire simple sur $\mathbb{Z}$. Cette chaîne de Markov n'admet pas de mesure de probabilité invariante. En revanche, la mesure de comptage sur $\mathbb{Z}$ est invariante et ergodique. Par conséquent :

$$\forall A, B \subset \mathbb{Z}, \quad \lim_{n \rightarrow \infty} \frac{\sum_{k=1}^n \mathbb{1}_{S_k \in A}}{\sum_{k=1}^n \mathbb{1}_{S_k \in B}} = \frac{|A|}{|B|}$$

alors que :

$$\frac{1}{n} \sum \mathbb{1}_{S_k \in A} = 0$$

2.3 Méthode MCMC

2.3.1 Principe général

La méthode *Monte-Carlo Markov Chain* est une méthode qui généralise la méthode de Monte-Carlo classique pour calculer des intégrales. Si on souhaite calculer $I = \int f d\pi$ avec π une mesure difficile à simuler mais qui est mesure invariante et ergodique d'une chaîne de Markov, on peut alors estimer :

$$\hat{I}_n = \frac{1}{n} \sum_{i=1}^n f(X_i)$$

Exemple (Sac à dos) On souhaite charger un sac à dos avec des objets de poids w_i . On définit :

$$C = \{u \in \{0, 1\}^n \mid \langle u, w \rangle \leq 25\}$$

l'ensemble des remplissages possibles du sac, et π la loi uniforme sur C :

$$\forall x \in C, \quad \pi(x) = \frac{1}{|C|}$$

Si $|C|$ est très grand, le dénombrement direct devient impossible, et la méthode du rejet est inefficace si $|C| \ll 2^n$. On peut alors définir une chaîne de Markov $(X_n)_{n \in \mathbb{N}}$ sur C telle que :

- Pour tout $x \in C$, on choisit uniformément un indice $i \in \{1, \dots, n\}$ et on propose de "flipper" x_i pour obtenir y :

$$y_j = \begin{cases} x_j & j \neq i \\ 1 - x_i & j = i \end{cases}$$

- Si $y \in C$, alors on accepte la transition $X_{n+1} = y$, sinon on reste en X_n .

Cette chaîne de Markov est *ergodique* (on peut atteindre tout état depuis n'importe quel autre) et π est sa *mesure invariante* :

$$\pi(x) = \sum_{y \in C} \pi(y) P(y \rightarrow x), \quad \forall x \in C$$

où $\mathbb{P}(y \rightarrow x)$ est la probabilité de transition de y vers x . Ainsi, pour n grand, la loi de X_n est approximativement π , et on peut estimer des quantités d'intérêt :

$$\mathbb{E}_\pi[f(X)] \approx \frac{1}{N} \sum_{n=1}^N f(X_n).$$

Ce procédé permet également d'estimer $|C|$ de façon approchée par des techniques de comptage MCMC.

Théorème (Vitesse de convergence du théorème ergodique) . Sous certaines hypothèses de régularité (que l'on ne détaillera pas ici), si $(X_n)_{n \geq 1}$ est une chaîne de Markov de mesure de probabilité π invariante et ergodique, on a :

$$\lim_{n \rightarrow \infty} \sqrt{n} \left(\frac{1}{n} \sum_{i=1}^n f(X_i) - \int f d\pi \right) = \mathcal{N}(0, \sigma_f^2) \quad \text{en loi}$$

où :

- $\sigma_f^2 = \mathbb{V}_\pi(f(X_0)) + 2 \sum_{k \geq 1} \text{cov}_\pi(f(X_0), f(X_k))$
- $f \in L^2(\pi)$.

Remarque Supposons que $X_0 \sim \pi$, posons $S_n = \frac{1}{n} \sum_{i=1}^n f(X_i)$, alors :

$$\begin{aligned} \mathbb{E}(S_n) &= \mathbb{E} \left(\frac{1}{n} \sum_{i=1}^n f(X_i) \right) = \frac{1}{n} \sum_{i=1}^n \mathbb{E}(f(X_i)) \\ &= \frac{1}{n} \sum_{i=1}^n \left(\sum_{x \in E} f(x) d\pi \right) = \int f d\pi \end{aligned}$$

$$\begin{aligned}
\mathbb{V}(S_n) &= \mathbb{E}(S_n^2) - \mathbb{E}(S_n)^2 \\
&= \mathbb{E} \left[\left(\frac{1}{n} \sum_{i=0}^n f(X_i) \right)^2 \right] - \left(\mathbb{E} \left[\frac{1}{n} \sum_{i=0}^n f(X_i) \right] \right)^2 \\
&= \frac{1}{n^2} \left(\sum_{i=0}^n \sum_{j=0}^n \mathbb{E}(f(X_i)f(X_j)) - \sum_{i=0}^n \sum_{j=0}^n \mathbb{E}(f(X_i))\mathbb{E}(f(X_j)) \right) \\
&= \frac{1}{n^2} \sum_{i=0}^n \sum_{j=0}^n \mathbb{E}(f(X_i)f(X_j)) - \mathbb{E}(f(X_i))\mathbb{E}(f(X_j)) \\
&= \frac{1}{n^2} \sum_{i=0}^n \sum_{j=0}^n \text{Cov}(f(X_i), f(X_j)) \\
&= \frac{1}{n^2} \left(2 \sum_{\substack{i,j=0 \\ i \neq j}}^n \text{Cov}(f(X_i), f(X_j)) + \sum_{i=0}^n \mathbb{V}(f(X_i)) \right) \\
&= \frac{1}{n^2} \sum_{i=0}^n \mathbb{V}(f(X_i)) + \frac{2}{n^2} \sum_{i=0}^{n-1} \text{Cov}(f(X_i), f(X_{i+1})) \\
&\quad \frac{2}{n^2} \sum_{i=0}^{n-2} \text{Cov}(f(X_i), f(X_{i+2})) + \cdots + \frac{2}{n^2} \text{Cov}(f(X_1), f(X_n)) \\
&= \frac{1}{n^2} (n\mathbb{V}(X_0) + 2(n-1)\text{Cov}(f(X_0), f(X_1)) + \cdots + \text{Cov}(f(X_0), f(X_n)))
\end{aligned}$$

Un obstacle à l'utilisation de méthodes MCMC est l'estimation de σ_f^2 par construire des intervalles de confiance. Plusieurs méthodes existent pour s'en affranchir.

2.3.2 Méthode par batchs (lots)

On souhaite toujours approcher l'intégrale $I = \int f d\pi$ par une chaîne de Markov $(X_n)_{n \in \mathbb{N}}$ de mesure invariante et ergodique π . Pour cela, on calculera :

$$\hat{I}_n = \frac{1}{n} \sum_{i=0}^n f(X_i)$$

Pour construire un intervalle de confiance pour I , nous devons calculer $\sigma_f^2 \simeq n\mathbb{V}(\hat{I}_n)$. Or on ne connaît pas $\mathbb{V}(\hat{I}_n)$ et les valeurs de X_i sont corrélées. L'idée est donc de "découper" notre chaîne de Markov en m batchs de longueur b tels que $n = b \times m$. On calculera alors la variance de chaque bloc en considérant qu'ils sont indépendants. On aura donc la moyenne de chaque lot :

$$\forall i \in \llbracket 1, m \rrbracket \quad \hat{I}_{b,i} = \frac{1}{b} \sum_{j=0}^b f(X_{bj+1})$$

puis on pose :

$$\hat{I}_n = \frac{1}{n} \sum_{i=1}^n \hat{I}_{b,i} = \frac{1}{n} \sum_{i=1}^n f(X_i)$$

on propose ensuite :

$$\hat{\sigma}_{f,n} = \frac{b}{m-1} \sum_{j=1}^m (\hat{I}_{b,j} - \hat{I}_n)^2$$

On peut donc construire l'intervalle de confiance de I suivant :

$$\left[\hat{I}_n \pm \frac{q_{1-\alpha/2}}{\sqrt{n}} \sqrt{\hat{\sigma}_{f,n}^2} \right]$$

2.3.3 Méthode de Newley-West

Une autre façon d'estimer σ_f^2 est d'estimer directement $\gamma_k = \text{Cov}(f(X_0), f(X_k))$ par l'estimateur empirique :

$$\hat{\gamma}_k = \frac{1}{n-k} \sum_{i=1}^{n-k} (f(X_i) - \hat{I}_n)(f(X_{i+k}) - \hat{I}_n)$$

on estime alors σ_f^2 par :

$$\hat{\sigma}_f^2 = \hat{\gamma}_0 + 2 \sum_{k=1}^L \gamma_k \left(1 - \frac{k}{L} \right)$$

où $L = n^{1/3}$ en général. On a donc l'intervalle de confiance suivant pour I :

$$\left[\hat{I}_n \pm \frac{q_{1-\alpha/2}}{\sqrt{n}} \sqrt{\hat{\sigma}_f^2} \right]$$

Chapitre 3

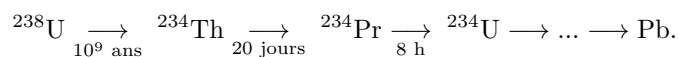
Processus de Markov en temps continu

Contents

3.1	Lois exponentielles et lemme des réveils	53
3.1.1	Rappels et Absence de mémoire	53
3.1.2	Lemme des réveils et exemples	54
3.2	Processus de Markov en temps continu	56
3.2.1	Définition, semi-groupe et générateur	56
3.2.2	Construction	57
3.2.3	Simulation	60

Pour certains modèles, la description d'un processus étape par étape à l'aide d'une chaîne de Markov peut masquer des informations importantes, comme le temps passé dans chaque état. Illustrons cela par un exemple.

Exemple (Désintégration radioactive) Considérons la désintégration radioactive de l'uranium 238, représentée par le processus suivant :



Si l'on modélisait ce processus par une chaîne de Markov classique, on ne connaîtrait que la succession des atomes traversés avant la formation du plomb, mais pas le temps passé dans chaque état, pourtant souvent l'information la plus pertinente.

Nous allons donc construire un processus de Markov en temps continu $(X_t)_{t \in \mathbb{R}_+}$ qui nous donnera cette information.

Dans la suite de ce cours, on cherchera donc à construire des processus de Markov $(X_t)_{(t \in \mathbb{R}_+)}$ sur des espaces d'états finis ou dénombrables qui vérifient une propriété de Markov. Autrement dit, tels que la variable :

$$T = \inf\{t > 0 \mid X_t \neq X_0\}$$

satisfasse une propriété d'absence de mémoire. On veut donc un équivalent à la propriété de Markov mais pour les processus à temps continu.

3.1 Lois exponentielles et lemme des réveils

3.1.1 Rappels et Abscence de mémoire

Rappel (Loi exponentielle) Une variable aléatoire E est de loi exponentielle de paramètre $\lambda > 0$, noté $X \sim \mathcal{E}(\lambda)$ si pour tout $x \geq 0$, on ait :

$$\mathbb{P}(X \geq x) = e^{-\lambda x} \iff \mathbb{P}(X \leq x) = 1 - e^{-\lambda x}$$

Cette variable aléatoire a donc pour densité la fonction :

$$x \mapsto \lambda e^{-\lambda x} \mathbb{1}_{x \geq 0}$$

par rapport à la mesure de Lebesgue.

Propriété (Loi exponentielle) . Soit $\lambda > 0$ et $X \sim \mathcal{E}(1)$ alors $X/\lambda \sim \mathcal{E}(\lambda)$. De plus, si $U \sim \mathcal{U}[0, 1]$ alors $-\log U \sim \mathcal{E}(1)$. On a aussi, pour $X \sim \mathcal{E}(\lambda)$:

$$\mathbb{E}(X) = \frac{1}{\lambda} \quad \text{et} \quad \mathbb{V}(X) = \frac{1}{\lambda^2}$$

Propriété (Abscence de Mémoire) . On dit qu'une variable aléatoire T satisfait la propriété d'absence de mémoire si :

$$\boxed{\forall t, s > 0, \quad \mathbb{P}(T > t + s \mid T > t) = \mathbb{P}(T > s)}$$

Les seules variables satisfaisant cette propriété sont les variables aléatoires de loi exponentielle.

Démonstration Soit T une variable aléatoire satisfaisant la propriété d'absence de mémoire. Montrons que T est de loi exponentielle. Posons :

$$f : \begin{cases} \mathbb{R}_+ \longrightarrow [0, 1] \\ t \longmapsto \mathbb{P}(T > t) \end{cases}$$

On a donc $\forall t, s \geq 0$:

$$\begin{aligned} & \mathbb{P}(T > t + s \mid T > t) = \mathbb{P}(T > s) \\ \iff & \frac{\mathbb{P}(T > t + s, T > t)}{\mathbb{P}(T > t)} = \mathbb{P}(T > s) \\ \iff & \frac{\mathbb{P}(T > t + s)}{\mathbb{P}(T > t)} = \mathbb{P}(T > s) \\ \iff & \frac{f(t + s)}{f(t)} = f(s) \\ \iff & f(t + s) = f(t)f(s) \end{aligned}$$

En particulier, pour tout $n \in \mathbb{N}$, on a :

$$f(n) = f(n - 1 + 1) = f(1)f(n - 1) = f(1)^n$$

Soient $p, q \in \mathbb{N}$, on a alors :

$$f(p) = f\left(\underbrace{\frac{p}{q} + \dots + \frac{p}{q}}_{q \text{ fois}}\right) \leq f\left(\frac{p}{q}\right)^q$$

or $f(p) = f(1)^p$ donc pour tout $q \in \mathbb{Q}_+$, on a :

$$f(q) = f(1)^q = e^{q \log(f(1))}$$

Soit $x \in \mathbb{R}_+$, par décroissance de f , pour tout $n \in \mathbb{N}$, on a :

$$\begin{aligned} f\left(\frac{\lceil nx \rceil + 1}{n}\right) &\leq f(x) \leq f\left(\frac{\lceil nx \rceil}{n}\right) \\ \iff f(x) &\in \left[\exp\left(\frac{\lceil nx \rceil + 1}{n} \log f(1)\right); \exp\left(\frac{\lceil nx \rceil}{n} \log f(1)\right) \right] \end{aligned}$$

d'où $f(x) = \exp(x \log f(1))$. Par conséquent $T \sim \mathcal{E}(-\log f(1))$.

3.1.2 Lemme des réveils et exemples

Lemme (Réveils) Soient $T_1, \dots, T_n$ des variables aléatoires indépendantes de lois respectives $\mathcal{E}(\lambda_1), \dots, \mathcal{E}(\lambda_n)$. On pose :

$$T = \min_{1 \leq i \leq n} T_i \quad \text{et} \quad I \in \llbracket 1, n \rrbracket \text{ tel que } T_I = T$$

On a donc :

$$\{I = k\} = \{T_k < T_1, T_k < T_2, \dots, T_k < T_n\}$$

Alors :

1. $T \sim \mathcal{E}(\lambda_1 + \dots, \lambda_n)$
2. $\mathbb{P}(I = k) = \frac{\lambda_k}{\lambda_1 + \dots + \lambda_k}$
3. $T \perp I$.

Démonstration Soient $T_1, \dots, T_n$ des variables aléatoires indépendantes de loi $\mathcal{E}(\lambda_1), \dots, \mathcal{E}_{\lambda_n}$. Posons :

$$T = \min_{1 \leq i \leq n} T_i \quad I \in \llbracket 1, n \rrbracket, T_I = T$$

Soient $x \geq 0$ et $k \in \llbracket 1, n \rrbracket$, on a :

$$\mathbb{P}(T > x, I = k) = \mathbb{P}(x \leq T_k \leq \underbrace{\min\{T_j \mid j \leq n, j \neq k\}}_{z_k})$$

on a donc $z_k = \min\{T_1, T_2, \dots, T_{k-1}, T_{k+1}, \dots, T_n\}$ de plus :

$$\begin{aligned} \mathbb{P}(z_k \geq y) &= \prod_{\substack{k=1 \\ j \neq k}}^n \mathbb{P}(T_j \geq y) = \prod_{\substack{k=1 \\ j \neq k}}^n e^{\lambda_j y} \\ &= e^{-(\lambda_1 + \dots + \lambda_n - \lambda_k)y} \end{aligned}$$

Par suite :

$$\begin{aligned}
\mathbb{P}(T > x, I = k) &= \mathbb{E}(\mathbb{P}(x \leq T_k \leq z_k \mid T_k)) \\
&= \mathbb{E}\left(\mathbb{1}_{T_k \geq x} \times e^{-((\lambda_1 + \dots + \lambda_n) - \lambda_k)T_k}\right) \\
&= \int_x^\infty e^{-((\lambda_1 + \dots + \lambda_n) - \lambda_k)t} \times f_{T_k}(t) dt \\
&= \lambda_k \int_x^\infty e^{-\lambda_k t} e^{-((\lambda_1 + \dots + \lambda_n) - \lambda_k)t} dt \\
&= \lambda_k \int_x^\infty e^{-(\lambda_1 + \dots + \lambda_n)t} dt \\
&= \lambda_k \left[-\frac{e^{-(\lambda_1 + \dots + \lambda_n)t}}{\lambda_1 + \dots + \lambda_n} \right]_x^\infty \\
&= \lambda_k \frac{e^{-(\lambda_1 + \dots + \lambda_n)x}}{\lambda_1 + \dots + \lambda_n}
\end{aligned}$$

En particulier, on a :

$$\mathbb{P}(I = k) = \mathbb{P}(I = k, T > 0) = \frac{\lambda_k}{\lambda_1 + \dots + \lambda_n}$$

Par indépendance, on a donc :

$$\begin{aligned}
\mathbb{P}(T \geq x) &= \prod_{i=1}^n \mathbb{P}(T_i \geq x) \\
&= \prod_{i=1}^n e^{-\lambda_i x} = e^{-(\lambda_1 + \dots + \lambda_n)x}
\end{aligned}$$

Enfin, on a donc :

$$\mathbb{P}(T \geq x, I = k) = \mathbb{P}(T \geq x) \mathbb{P}(I = k)$$

Remarque Si on souhaite tirer un entier x proportionnellement à un poids w_x sans calculer $\sum_{i=1}^n w_i$, le lemme des réveils nous donne une solution. On peut donc utiliser l'algorithme suivant :

```

1 def tirer(poids):
2     x = -1
3     val = inf
4     for k in range(len(poids)):
5         e = -log(rand())/poids[k]
6         if e < val:
7             val = e
8             x = k
9     return x
10

```

Exemple (Sélection sans remplacement) Soient $w_1, \dots, w_n$ des poids positifs associés aux entiers $\llbracket 1, n \rrbracket$. On souhaite tirer sans remplacement k entiers avec des probabilités proportionnelles à w_k . On procédera donc comme suit :

1. On sélectionne I_1 avec :

$$\mathbb{P}(I_1 = j) = \frac{w_j}{w_1 + \dots + w_n}$$

2. Conditionnellement à I_1 , on sélectionne I_2 avec :

$$\mathbb{P}(I_2 = j \mid I_1) = \frac{w_j \mathbb{1}_{I_1 \neq j}}{\sum_{i=1}^n w_i - w_{I_1}}$$

Par généralisation, du lemme des réveils, les variables $I_1, \dots, I_n$ sont de même loi que les variables construites comme suit. Soient $T_1, \dots, T_n$ des variables de loi $\mathcal{E}(w_k)$ on pose I_j l'indice correspondant à la j ème plus petite valeur de $\{T_1, \dots, T_n\}$.

$$\text{i.e } |\{k \in \llbracket 1, n \rrbracket \mid T_k \leq T_{I_j}\}|$$

Exemple Soient $w_1 = w_2 = \dots = w_{n-1} = 1$ et $w_n = a < 1$. Alors :

$$\begin{aligned} \mathbb{P}(\text{dernier choisi soit } n) &= \mathbb{P}(I_n = n) \\ &= \mathbb{P}(T_n \geq \max(T_1, \dots, T_{n-1})) \\ &= \int_0^\infty a e^{-at} \mathbb{P}(\max(T_1, \dots, T_{n-1}) \leq t) dt \\ &= a \int_0^\infty e^{-at} (1 - e^{-t})^{n-1} dt \end{aligned}$$

3.2 Processus de Markov en temps continu

3.2.1 Définition, semi-groupe et générateur

Nous considérons ici que E est un espace d'états fini ou dénombrable.

Définition (Processus de Markov en temps continu) . Un processus $(X_t)_{t \geq 0}$ sur E est un processus de Markov en temps continu si :

$$\forall j \in E, \forall t, s \geq 0, \quad \mathbb{P}(X_{t+s} = j \mid \mathcal{F}_t) = \mathbb{P}_s(X_t, j)$$

où $\mathcal{F}_t$ est la tribu engendrée par les événements $\{X_r = x_r\}, 0 \leq r \leq t$. En particulier, pour tout $i, j \in E$ et $\forall t \geq 0$, on a :

$$P_t(i, j) = \mathbb{P}(X_t = j \mid X_0 = i)$$

où (P_t) est le *semi-groupe* du processus de Markov.

Propriété (Semi-groupe) . $(P_t)_{t \geq 0}$ est un *semi-groupe* si :

$$\forall t, s \in E, \quad P_{t+s} = P_t P_s$$

si E est fini, alors $(P_t)_{t \geq 0}$ est une matrice et $P_t P_s$ est un produit matriciel. Dans un cas plus général, on a :

$$\begin{aligned} P_t P_s(i, j) &= \sum_{k \in E} P_t(i, k) P_s(k, j) = P_{t+s}(i, j) \\ &= \sum_{k \in E} \mathbb{P}(X_t = k \mid X_0 = i) \mathbb{P}(X_{t+s} = j \mid X_t = k) \\ &= \sum_{k \in E} \mathbb{P}(X_t = k, X_{t+s} = j \mid X_0 = i) \\ &= \mathbb{P}(X_{t+s} = j \mid X_0 = i) \end{aligned}$$

Donc le semi-groupe caractérise la loi du processus de Markov. De même, grâce à la cette propriété, il n'est pas nécessaire de connaître tout le semi-groupe.

Propriété (Génération du semi-groupe) . En particulier, $(P_t)_{t \geq 0}$ est uniquement déterminé par $(P_s)_{0 \leq s \leq a}$ avec a arbitrairement petit.

Définition (Générateur Infinitésimal) . Le *générateur infinitésimal* d'un processus de Markov $(X_t)_{t \geq 0}$ est défini par :

$$\begin{aligned} Q_{i,j} &= \lim_{t \rightarrow 0} \frac{P_t(i,j) - \mathbb{1}_{i=j}}{t} \\ &= \lim_{t \rightarrow 0} \frac{\mathbb{P}_i(X_t = j) - \mathbb{1}_{i=j}}{t} \end{aligned}$$

Il correspond à la dérivée de $P_t(i,j)$ par rapport à t en 0.

Lemme Soit $(X_t)_{t \geq 0}$ un processus de Markov sur E tel que $X_0 = i$ p.s. On pose :

$$T = \inf\{t > 0 \mid X_t \neq i\} \quad \text{et} \quad Y = X_T$$

La variable T suit une loi exponentielle et est indépendante de Y .

Intuitivement, cela signifie que le temps que prend le processus pour sortir d'un état suit une loi exponentielle et ne dépend pas de l'état suivant.

Remarque Le générateur infinitésimal caractérise la loi du processus de Markov. Donc $-Q_{i,i}$ donne le paramètre de la loi exponentielle correspondant au temps de sortie de l'état i par X . De même, $\frac{Q_{i,j}}{Q_{i,i}}$ donne la probabilité de finir en j après un saut hors de i .

3.2.2 Construction

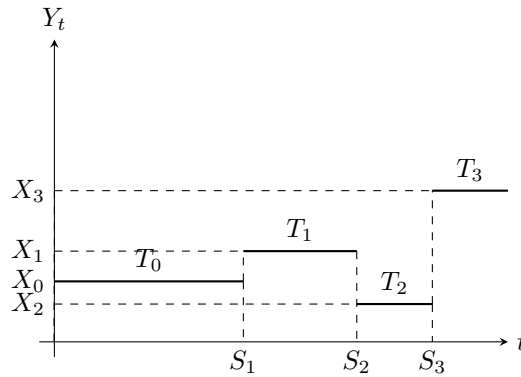
Proposition Un processus de Markov de générateur Q peut donc être construit comme suit :

- On fixe X_0 .
- On génère T_0 une variable aléatoire de loi $\mathcal{E}(-Q_{X_0, X_0})$ ainsi que X_1 indépendant de X_0 tel que :

$$\mathbb{P}(X_1 = j) = -\frac{Q_{X_0, X_1}}{Q_{X_0, X_0}}$$

- Par récurrence, X_n étant donné, on génère T_{n+1} de loi $\mathcal{E}(-Q_{X_n, X_n})$ et X_{n+1} de loi :

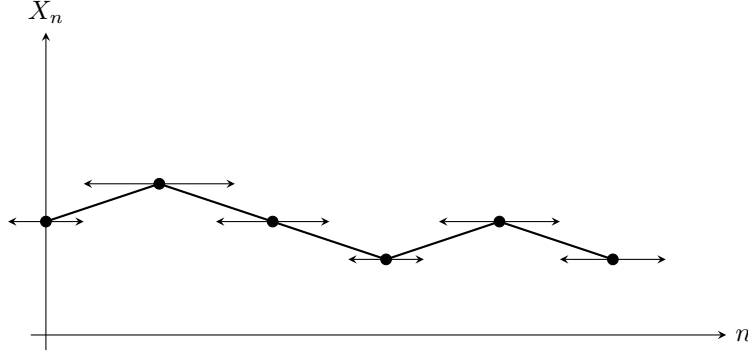
$$\mathbb{P}(X_{n+1} = j \mid X_n) = -\frac{Q_{X_n, j}}{Q_{X_n, X_n}}$$



Un processus de Markov en temps continu est donc un processus qui reste un temps exponentiel dans un état avant de sauter dans un autre. On peut alors poser :

$$S_k = T_0 + T_1 + \cdots + T_{k-1} \quad \text{et} \quad \forall t \in [S_k, S_{k+1}], Y_t = X_k$$

Les variables $S_1, S_2, \dots$ définissent les temps de saut du processus Y et $(X_n)_{n \geq 0}$ est une chaîne de Markov (à temps discret) appelée le *squelette* du processus de Markov.



Un processus de Markov en temps continu est donc une chaîne de Markov en temps discret auquel on a rajouté des durées aléatoires (exponentielles) entre le passage d'un état à l'autre. On peut observer une limite au processus. En effet, le processus de Markov est défini jusqu'au temps $\bar{S} = \lim_{k \rightarrow \infty} S_k$. Ainsi un processus tel que $\mathbb{P}(\bar{S} < \infty) > 0$ est dit *explosif*. Sinon il est dit *non explosif*.

Formalisons maintenant toutes ces nouvelles notions.

Définition (Squelette d'un processus de Markov) . Soit $(Y_t)_{t \geq 0}$ un processus de Markov à temps continu, et $(S_k)_{k \geq 0}$ la suite croissante de ses temps de saut, définie par :

$$S_0 = 0, \quad S_{k+1} = S_k + T_k$$

où les T_k sont les durées passées dans chaque état avant le saut suivant. On appelle *squelette* du processus (Y_t) la suite $(X_n)_{n \geq 0}$ définie par :

$$X_n = Y_{S_n}, \quad \text{pour tout } n \geq 0$$

Le processus $(X_n)_{n \geq 0}$ est alors une *chaîne de Markov à temps discret*, dont la matrice de transition P est donnée par :

$$P_{ij} = \mathbb{P}(X_{n+1} = j \mid X_n = i) = -\frac{Q_{ij}}{Q_{ii}}, \quad i \neq j.$$

Définition (Processus explosif) . Soit $(Y_t)_{t \geq 0}$ un processus de Markov à temps continu et $(S_k)_{k \geq 0}$ la suite de ses temps de saut. On définit :

$$\bar{S} = \lim_{k \rightarrow \infty} S_k$$

— Le processus (Y_t) est dit *explosif* si

$$\mathbb{P}(\bar{S} < \infty) > 0$$

c'est-à-dire qu'il effectue une infinité de sauts en un temps fini.

— Il est dit *non explosif* si

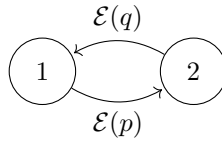
$$\mathbb{P}(\bar{S} = \infty) = 1$$

Vocabulaire. On appelle $Q_{i,j}$ le temps de saut de l'état i vers j . On a aussi $Q_{i,j} = \lambda_i \mathbb{P}_i(Y = j)$.

Exemple (Espace d'états fini) Soit $E = \{1, 2\}$ et $(X_t)_{t \geq 0}$ un processus de Markov sur E donné par le générateur infinitésimal :

$$Q = \begin{pmatrix} -p & p \\ q & -q \end{pmatrix}$$

On a donc le graphe suivant pour cette chaîne de Markov :



La durée de séjour du processus dans l'état 1 suit donc une loi $\mathcal{E}(p)$ et celle dans l'état 2 suit $\mathcal{E}(q)$.

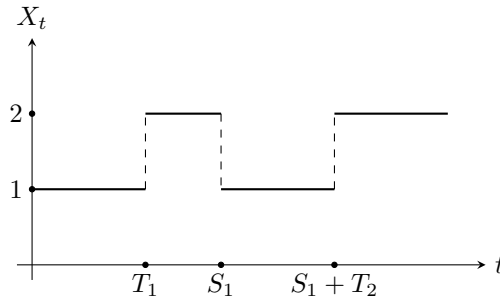
La probabilité de passer d'un état à l'autre suit une loi exponentielle. Soient $(T_k)_{k \in \mathbb{N}}$ des variables iid de loi $\mathcal{E}(p)$ et $(Q_k)_{k \in \mathbb{N}}$ des variables iid de loi $\mathcal{E}(q)$. Posons :

$$S_0 = 0, \quad \text{et} \quad S_n = \sum_{k=1}^n T_k + Q_k$$

On a donc :

$$\begin{cases} X_t = 1 & \text{si } t \in \bigcup_{n \in \mathbb{N}} [S_n, S_n + T_{n+1}[\\ X_t = 2 & \text{si } t \in \bigcup_{n \in \mathbb{N}} [S_n + T_{n+1}, S_{n+1}[\end{cases}$$

On peut donc représenter la trajectoire de ce processus par le graphe suivant :



Exemple (Espace d'états infini) Posons maintenant $E = \mathbb{N}^*$ et soit $(X_t)_{t \geq 0}$ un processus de Markov sur E de générateur :

$$Q_{n,n} = -n^2, \quad \text{et} \quad Q_{n,n+1} = n^2$$

Le processus passe donc de l'état n à $n + 1$ à taux n^2 . Le squelette de la chaîne est donc le processus

$$(1, 2, 3, 4, \dots)$$

avec $S_n = T_1 + T_2 + \dots + T_n$ où $(T_n)_{n \in \mathbb{N}}$ sont indépendantes et de loi $\mathcal{E}(k^2)$. On peut donc en déduire que :

$$S_n = E_1 + \frac{E_2}{2^2} + \dots + \frac{E_n}{n^2}$$

où les $(E_i)_{i \in \mathbb{N}}$ sont iid de loi $\mathcal{E}(1)$. On a alors :

$$\bar{S} = \lim_{n \rightarrow \infty} S_n = \sum_{k=1}^{\infty} \frac{E_k}{k^2}$$

$$\mathbb{E}(\bar{S}) = \sum_{k=1}^{\infty} \frac{\mathbb{E}(E_k)}{k^2} = \sum_{k=1}^{\infty} \frac{1}{k^2} < \infty$$

d'où $\bar{S} < \infty$ presque sûrement. Cette chaîne est donc *explosive*.

3.2.3 Simulation

Proposition La construction des processus de Markov donné ici est parfois appelé *processus de Gillespie*. On peut définir l'algorithme suivant pour les simuler :

```

1 def procMarkov(t,x,Q):
2     tau = - 1 * log(rand()) / (- Q(x,x))
3     if tau > t :
4         return x
5     # On tire x selon la loi -Qx,. / -Qx,x
6     return procMarkov(t - tau, y, Q)

```

Exemple (Processus de naissance et de mort) Soit $(X_t)_{t \geq 0}$ un processus de Markov sur $\mathbb{N}^*$ et pour tout $i \geq 0$ soit b_i le taux de naissance en i et d_i le taux de mort en i .

$$\text{i.e } \begin{cases} X_t \text{ saute de } i \text{ en } i+1 \text{ à taux } b_i \\ X_t \text{ saute de } i \text{ en } i-1 \text{ à taux } d_i \end{cases}$$

On ne simule évidemment pas ce processus à partir de son générateur, mais plutôt par un tirage successif des durées de séjour exponentielles et des transitions, comme dans le code ci-dessous :

```

1 def simulMortNaissance(x,t,b,d):
2     etat = x
3     tps = - log(rand()) / (b[etat] + d[etat])
4     while tps < t :
5         if rand() < b[etat] / (b[etat] + d[etat]):
6             etat += 1
7         else :
8             etat -= 1
9         tps += - log(rand()) / (b[etat] + d[etat])
10    return etat

```

Proposition (Construction par le lemme des réveils) Il existe une deuxième construction des processus de Markov, fondée sur le *lemme des réveils*.

Lorsque le processus (X_t) est en état $x \in E$, on associe à chaque $y \in E \setminus \{x\}$ une variable aléatoire indépendante T_y de loi exponentielle $\mathcal{E}(Q_{x,y})$. On pose alors :

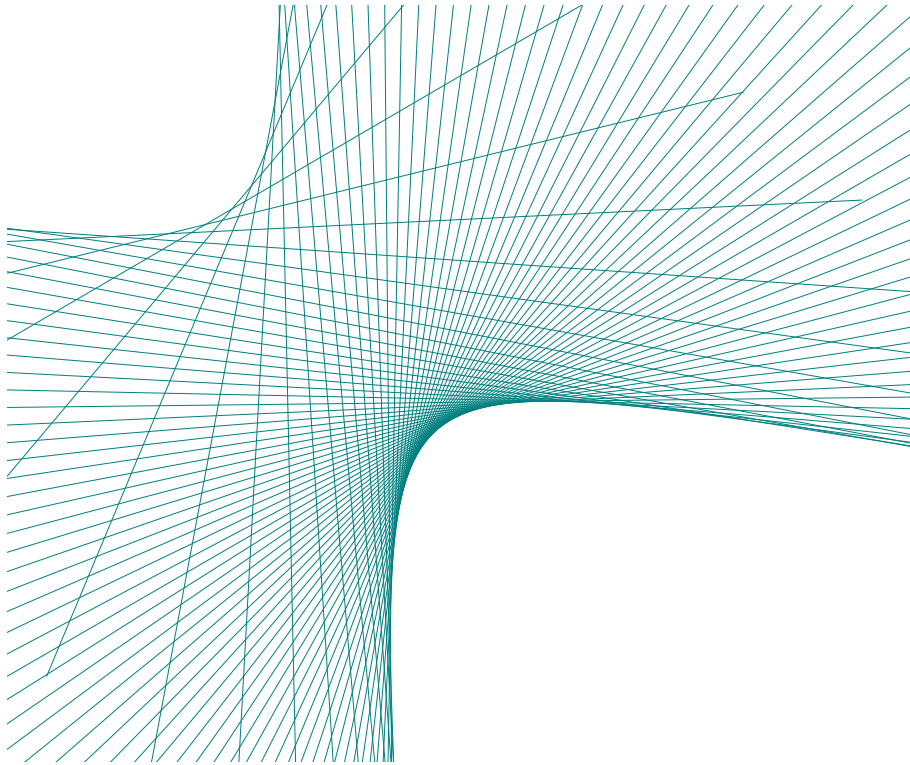
$$T = \min_{y \neq x} T_y, \quad Y = \arg \min_{y \neq x} T_y.$$

Le processus reste dans l'état x pendant une durée T , puis saute en l'état Y .

Ainsi, la durée de séjour en x suit une loi exponentielle de paramètre $-Q_{x,x} = \sum_{y \neq x} Q_{x,y}$, et la probabilité de transition vérifie :

$$\mathbb{P}(X_{t+T} = y \mid X_t = x) = \frac{Q_{x,y}}{Q_{x,x}}.$$

Optimisation



Chapitre 1

Optimisation sans contraintes

Contents

1.1	CNS d'optimalité sans contraintes	63
1.1.1	Conditions Nécessaires	63
1.1.2	Conditions Suffisantes	63
1.2	Algorithmes de Descente	64
1.2.1	Principe	64
1.2.2	Plus profonde descente	65
1.2.3	Recherche Linéaire : Conditions	66
1.2.4	Algorithme de Recherche Linéaire	67
1.2.5	Méthode de Newton	67
1.2.6	Méthode de quasi-Newton (BFGS)	69
1.3	Problèmes Quadratiques	70
1.3.1	Généralités	71
1.3.2	Algorithme	71
1.4	Récapitulatif	73

En informatique et en mathématiques, l'optimisation est la recherche d'un optimum pour un problème posé. On s'intéresse ici à optimisation dite *continue*. On peut alors formuler un tel problème sous la forme suivante :

$$\begin{array}{ll} \text{minimize} & f(x) \\ \text{subject to} & x \in C \end{array} \qquad \begin{array}{ll} \min & f(x) \\ \text{s.t.} & x \in C \end{array}$$

On appelle x la *variable de décision* ou *variable de contrôle*. La fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est appelée *fonction objectif* ou le *critère*. L'ensemble $C \subset \mathbb{R}^n$ est appelée *ensemble des contraintes*.

Résoudre un tel problème signifie trouver $\bar{x} \in C \subset \mathbb{R}^n$ tel que :

$$\forall x \in C, \quad f(x) \geq \bar{x}$$

Remarque On peut remarquer que maximiser f revient à minimiser $-f$. On peut donc toujours se ramener à un problème de minimisation. De plus, lorsque $C = \mathbb{R}^n$ on dit que le problème est *non contraint*.

1.1 CNS d'optimalité sans contraintes

Dans cette section, nous nous intéressons aux problèmes de la forme suivante :

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & x \in \mathbb{R}^n \end{aligned}$$

où $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Nous commencerons par aborder l'optimalité locale puis nous passerons, grâce à la convexité, à l'optimalité globale.

1.1.1 Conditions Nécessaires

Évoquons tout d'abord quelques conditions nécessaires d'optimalité globale.

Théorème (CN1 - Principe de Fermat) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ et $\bar{x} \in \mathbb{R}^n$ un minimum local de f . Si f est *différentiable* alors :

$$\nabla f(\bar{x}) = 0$$

Définition (Matrice définie positive) . Soit $A \in \mathcal{M}_{n,n}(\mathbb{R})$ une matrice symétrique, on dit que A est *définie positive* si

$$\begin{aligned} \forall x \in \mathbb{R}^n \setminus \{0\}, \quad {}^t x A x &> 0 \\ \iff Sp(A) &\subset \mathbb{R}_+^* \end{aligned}$$

On note alors $A \succ 0$

Définition (Matrice semi-définie positive) . Soit $A \in \mathcal{M}_{n,n}(\mathbb{R})$ une matrice symétrique, on dit que A est *semi-définie positive* si

$$\begin{aligned} \forall x \in \mathbb{R}^n \setminus \{0\}, \quad {}^t x A x &\geq 0 \\ \iff Sp(A) &\subset \mathbb{R}_+ \end{aligned}$$

On note alors $A \succeq 0$

Théorème (CN2) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une fonction quelconque et soit $\bar{x} \in \mathbb{R}^n$ un minimum local de f . Si f est *deux fois différentiable en $\bar{x}$* alors :

$$\nabla^2 f(\bar{x}) \succeq 0$$

(i.e ssi la hessienne de f est semi-définie positive). On appelle cette condition la *condition d'optimalité du second ordre*.

1.1.2 Conditions Suffisantes

Passons maintenant aux conditions suffisantes d'optimalités...

Théorème (CS1) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une fonction quelconque et $\bar{x} \in \mathbb{R}^n$. On suppose qu'il existe $r > 0$ tel que :

- f est différentiable en tout point de $B(\bar{x}, r)$
- $\forall x \in B(\bar{x}, r) \setminus \{\bar{x}\}, \langle \nabla f(x), x - \bar{x} \rangle \geq 0$

Alors $\bar{x}$ est un *minimum local* de f .

Théorème (CS2) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une fonction quelconque et $\bar{x} \in \mathbb{R}^n$. Alors si :

- f est deux fois différentiable en $\bar{x}$
- $\nabla f(\bar{x}) = 0$
- $\nabla^2 f(\bar{x}) \succeq 0$

Alors $\bar{x}$ est un *minimum local* de f .

Le théorème suivant nous permet de généraliser ce minimum local en minimum global. Pour cela, il faut que la fonction ne décroît plus (i.e que $\bar{x}$ soit en bas de la parabole). Pour cela, on utilisera la notion de convexité.

Théorème (Convexité et Minimum) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une fonction quelconque et $\bar{x} \in \mathbb{R}^n$ un minimum local de f . Alors :

1. Si f est *convexe* alors $\bar{x}$ est un *minimum global* de f .
2. Si f est *strictement convexe* alors $\bar{x}$ est *l'unique minimum global* de f .

Ce résultat est fondamental : la convexité assure qu'un minimum local est automatiquement global, ce qui simplifie considérablement la recherche de solutions.

Définition (Fonction Coercive) . Une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est dite *coercive* si :

$$\forall x \in \mathbb{R}^n \text{ tq } \|x\| \rightarrow \infty \text{ alors } f(x) \rightarrow \infty$$

Intuitivement, cela signifie que f "repousse" les points qui s'éloignent à l'infini : la fonction ne peut pas être minimisée à l'extérieur d'un compact.

Théorème (Weierstrass) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Si f est *continue* et *coercive* alors elle admet un minimum global sur $\mathbb{R}^n$.

$$\text{i.e. } \exists x^* \in \mathbb{R}^n \text{ tel que } f(x^*) = \inf_{x \in \mathbb{R}^n} f(x)$$

Corollaire (Minimum Global) . Si f est convexe, continue et coercive, alors f admet un minimum global (unique si f est strictement convexe).

1.2 Algorithmes de Descente

1.2.1 Principe

Nous allons ici nous intéresser à des algorithmes permettant d'approximer un minimum d'un champ de scalaires. Ces algorithmes seront itératifs, autrement dit, à chaque pas de temps, nous approcherons de plus en plus notre solution. On les appelle des *algorithmes de descente*. La notion de "descente" se formalise par la construction d'une suite $(u_n)_{n \in \mathbb{N}}$ tel que :

$$\forall n \in \mathbb{N}, \quad f(u_{n+1}) \leq f(u_n)$$

Définition (Direction de descente) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une application différentiable et $x, d \in \mathbb{R}^n$. Le vecteur d est la *direction de descente* de f en x si :

$$\langle \nabla f(x), d \rangle < 0$$

Pour l'algorithme de descente, ce vecteur d nous donnera le "sens" dans lequel il faut descendre pour s'approcher du minimum. Or maintenant, que l'on sait de quel "coté" descendre, essayons de savoir "de combien" faut-il descendre.

Théorème (Existence du pas) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une application différentiable. Soient $x, d \in \mathbb{R}^n$ tels que $\nabla f(x) \neq 0$. Si d est une direction de descente de f en x , alors il existe $\mu > 0$ tel que :

$$\forall \alpha \in]0, \mu[, \quad f(x + \alpha d) < f(x)$$

De plus, pour tout $\beta < 1$, il existe $\mu > 0$ tel que :

$$\forall \alpha \in]0, \mu[, \quad f(x + \alpha d) < f(x) \alpha \beta \langle \nabla f(x), d \rangle$$

On appellera α le *pas* de l'itération.

Proposition Chaque itération du processus de descente se fait donc en 3 étapes :

1. Trouver une direction d_k telle que $\langle \nabla f(x_k), d_k \rangle < 0$
2. Trouver un pas α_k tel que $f(x_k + \alpha_k d_k) < f(x_k)$
3. Calculer $x_{k+1} = x_k + \alpha_k d_k$

Plus formellement, on a donc :

Algorithm 1 Algorithme de la Descente de Gradient

Require: Fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$, point initial $x_0 \in \mathbb{R}^n$, tolérance $\varepsilon > 0$

Ensure: Approximation $\bar{x}$ d'un minimum local de f

- 1: $k \leftarrow 0$
- 2: **while** $\|\nabla f(x_k)\| > \varepsilon$ **do**
- 3: Calculer la direction de descente :

$$d_k \leftarrow -\nabla f(x_k)$$

- 4: Déterminer un pas $\alpha_k > 0$ qui minimise $f(x_k + \alpha_k d_k)$
- 5: Mettre à jour :

$$x_{k+1} \leftarrow x_k + \alpha_k d_k$$

- 6: $k \leftarrow k + 1$
 - 7: **end while**
 - 8: **return** $\bar{x} \leftarrow x_k$
-

1.2.2 Plus profonde descente

Nous allons maintenant nous intéresser à différents types de descente. En effet, en fonction de l'application à minimiser nous utiliserons différents pas et directions lors de la descente. La méthode naturelle serait de prendre la direction dans laquelle les valeurs de l'application descendent le plus. Nous choisirons donc au début la direction $d = -\nabla f(x)$.

Théorème (Gradient comme direction) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une application différentiable. Soient $x \in \mathbb{R}^n$ alors pour tout $d \in \mathbb{R}^n$ tel que $\|d\| = 1$ on a :

$$\langle \nabla f(x), -\nabla f(x) \rangle \leq \langle \nabla f(x), d \rangle$$

On dira alors que $d^* = -\nabla f(x)$ est la *direction de plus profonde descente*.

Proposition Pour mettre en oeuvre notre plus profonde descente, à chaque itération, nous calculons donc :

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

Un théorème précédent nous assure de pouvoir trouver un α_k qui convient à chaque itération. Pour une valeur de α_k optimale, on pourrait résoudre le problème suivant à chaque itération :

$$\begin{aligned} \min \quad & f(x_k + \alpha d_k) \\ \text{s.t.} \quad & \alpha > 0 \end{aligned}$$

Mais nous ne sommes pas sûr de toujours pouvoir le résoudre.

1.2.3 Recherche Linéaire : Conditions

On remarque assez vite que choisir un pas constant très petit (comme 0.0001) rallonge considérablement la descente. D'autre part, nous savons que l'existence d'un minimum pour un champ de scalaire est une notion locale. Prendre un pas trop grand risquerait donc de nous faire rater le minimum et de faire augmenter la fonction objectif.

Or, résoudre un problème d'optimisation pour α_k à chaque itération n'est pas non plus une bonne solution car elle augmente la complexité de la méthode. Nous allons donc essayer de développer une méthode qui permet de résoudre approximativement la recherche d'un α_k à moindre coût. Pour cela, nous choisirons α_k en fonction de la rapidité ou non de la décroissance de notre fonction objectif.

Proposition Pour trouver un pas raisonnable à chaque itération, nous utiliserons l'information contenue dans le gradient de la fonction objectif. En effet, celui-ci nous permet de savoir comment celle-ci diminue et à quelle vitesse. On regardera, pour chaque pas possible, la *diminution* qu'il produit dans notre algorithme :

$$f(x_k) - f(x_k + \alpha_k d_k) > \gamma \alpha_k, \quad \text{pour } \gamma > 0$$

Diminution qui sera proportionnelle au pas. Ainsi, plus la pente $\langle \nabla f(x_k), d \rangle$ de la fonction objectif dans la direction de d sera importante, plus le pas sera grand.

Définition (Première condition de Wolfe - Condition d'Arjimo) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une application différentiable. Soient $x_k \in \mathbb{R}^n$ et $d_k \in \mathbb{R}^n$ une direction de descente de f en x_k et $\alpha_k > 0$. On dit que la fonction f *diminue suffisamment* en $x_k + \alpha_k d_k$ par rapport à x_k si :

$$f(x_k + \alpha_k d_k) < f(x_k) + \alpha_k \beta \langle \nabla f(x_k), d_k \rangle$$

avec $\beta \in]0, 1[$.

Cette condition nous permet de rejeter des pas trop grands qui ne fournissent pas de décroissance suffisante de la fonction.

Il peut arriver que les pas α_k deviennent très proches de 0 et empêchent la méthode de progresser. On va donc chercher à avancer dans notre descente mais sans dépasser le minimum (première condition de Wolfe), mais en avançant suffisamment pour arriver sur un point où la pente de la fonction objectif est moins raide (seconde condition de Wolfe).

Définition (Seconde Condition de Wolfe) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une application différentiable. Soit $x_k \in \mathbb{R}^n$, $d_k \in \mathbb{R}^n$ une direction de descente de f en x_k et $\alpha_k > 0$. On dira que le point $x_k + \alpha_k d_k$ apporte une *progression suffisante* par rapport à x_k si :

$$\begin{aligned} \langle \nabla f(x_k + \alpha_k d_k), d_k \rangle &\geq \beta \langle \nabla f(x_k), d_k \rangle \\ \iff \frac{\langle \nabla f(x_k + \alpha_k d_k), d_k \rangle}{\langle \nabla f(x_k), d_k \rangle} &\geq \beta \end{aligned}$$

Les algorithmes de *recherche linéaire* nous permettent d'identifier un pas qui satisfait les deux conditions de Wolfe.

1.2.4 Algorithme de Recherche Linéaire

L'idée de l'algorithme de recherche linéaire est de construire de manière itérative un intervalle $[\alpha_g, \alpha_d]$ contenant des valeurs qui satisfassent les deux conditions de Wolfe. On choisit ensuite un $\alpha \in [\alpha_g, \alpha_d]$ et s'il viole l'une des deux conditions de Wolfe, on réduit l'intervalle précédente et on recommence.

Algorithm 2 Recherche linéaire satisfaisant les conditions de Wolfe

Require: Fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$, point courant x_k , direction de descente d_k , pas initial $\alpha_0 > 0$, paramètres $0 < \beta_1 < \beta_2 < 1$, $\lambda > 1$

Ensure: Pas α satisfaisant les conditions de Wolfe

```

1:  $i \leftarrow 0$ ,  $\alpha_g \leftarrow 0$ ,  $\alpha_d \leftarrow \infty$ 
2:  $\alpha_i \leftarrow \alpha_0$ 
3: while  $\alpha_i$  viole l'une des conditions de Wolfe do
4:   if  $\alpha_i$  viole la première condition de Wolfe (Armijo) then
5:      $\alpha_d \leftarrow \alpha_i$  ▷ le pas est trop long
6:      $\alpha_{i+1} \leftarrow \frac{\alpha_g + \alpha_d}{2}$ 
7:   else if  $\alpha_i$  viole la seconde condition de Wolfe (curvature) then
8:      $\alpha_g \leftarrow \alpha_i$  ▷ le pas est trop court
9:     if  $\alpha_d < \infty$  then
10:       $\alpha_{i+1} \leftarrow \frac{\alpha_g + \alpha_d}{2}$ 
11:     else
12:       $\alpha_{i+1} \leftarrow \lambda \alpha_i$ 
13:     end if
14:   end if
15:    $i \leftarrow i + 1$ 
16:    $\alpha_i \leftarrow \alpha_{i+1}$ 
17: end while
18: return  $\alpha_i$ 

```

Théorème (Finitude de la recherche linéaire) . Soit f une application différentiable bornée inférieurement dans une direction d_k par rapport à $x_k \in \mathbb{R}^n$. Alors la recherche linéaire d'un pas se termine après un nombre fini d'itérations.

1.2.5 Méthode de Newton

Le principe de la méthode de Newton est d'approximer localement la fonction f à minimiser par un développement de Taylor à l'ordre 2 puis de trouver le minimum de cette approximation.

Dans sa forme la plus générale, la méthode de Newton sert à résoudre un système non linéaire défini par $g(x) = 0$ où $g : \mathbb{R}^n \rightarrow \mathbb{R}^n$. Elle consiste à construire une suite de points $(x_k)_{k \in \mathbb{N}}$ où à chaque étape on cherche un $d \in \mathbb{R}^n$ tel que :

$$\forall k \in \mathbb{N}, \quad g(x_k + d) = 0$$

Or, en remplaçant g par son approximation linéaire :

$$g(x_k + d) \simeq g(x_k) + J(x_k)d$$

La recherche de d tel que $g(x_k + d) = 0$ est alors équivalente à la recherche d'un d tel que :

$$\begin{aligned} g(x_k) + J(x_k)d &= 0 \\ \iff g(x_k) &= -J(x_k)d \end{aligned}$$

L'itéré suivant est alors $x_{k+1} = x_k + d$. On s'arrête lorsque $\|g(x_k)\|$ est plus petit qu'un seuil fixé.

Proposition Appliquons maintenant la méthode de Newton à la recherche d'un minimum d'une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$. On cherche donc à obtenir une condition d'optimalité de la forme $g(x) = \nabla f(x) = 0$. Dans le cadre précédent, on a donc $g(x) = \nabla f(x)$ et $J(x) = H_f(x)$. Ici, l'approximation de f par un développement de Taylor de la forme :

$$f(x) \simeq f(x_k) + \langle \nabla f(x_k), d \rangle + \frac{1}{2} \langle d, H_f(x_k)d \rangle$$

est minimisée si $H_f(x_k)$ est définie positive. Autrement dit, lorsque le gradient de g est nul soit :

$$\nabla f(x_k) + H_f(x_k)d = 0$$

On utilisera donc l'algorithme suivant :

Algorithm 3 Méthode de Newton (optimisation)

Require: Fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$, point initial x_0 , tolérance $\varepsilon > 0$

Ensure: Approximation x^* d'un point critique de f

```

1:  $k \leftarrow 0$ 
2: while  $\|\nabla f(x_k)\| > \varepsilon$  do
3:   Calculer le gradient  $\nabla f(x_k)$ 
4:   Calculer la hessienne  $H_f(x_k)$ 
5:   Résoudre  $H_f(x_k)d_k = -\nabla f(x_k)$ 
6:   Mettre à jour  $x_{k+1} \leftarrow x_k + d_k$ 
7:    $k \leftarrow k + 1$ 
8: end while
9: return  $x^* \leftarrow x_k$ 

```

L'avantage de la méthode de Newton est sa convergence quadratique lorsque les conditions favorables sont réunies. En revanche, elle possède plusieurs inconvénients qui nous pousseront à développer une méthode plus fine :

- Si le point de départ de la méthode est trop éloigné, elle peut diverger très violemment.
- Elle nécessite le calcul de la hessienne de la fonction à minimiser.
- Elle n'est pas définie si la hessienne n'est pas inversible.
- Dans le contexte de l'optimisation, même lorsque la suite $(x_k)_{k \in \mathbb{N}}$ converge, elle converge vers un point critique, c'est-à-dire une solution de l'équation $\nabla f(x) = 0$. Elle peut donc converger vers un maximum local.

Pour pallier ces contraintes, nous utiliserons la méthode de Newton hybride. Elle vise à améliorer la robustesse de la méthode de Newton classique, en garantissant que la direction de recherche reste toujours une direction de descente. Elle combine pour cela la régularisation de la hessienne et une étape de recherche linéaire. Ainsi, même lorsque la hessienne $H_f(x_k)$ n'est pas définie positive, la méthode reste stable et converge vers un minimum local.

Algorithm 4 Méthode de Newton hybride (régularisée avec recherche linéaire)

Require: Fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$, point initial x_0 , tolérance $\varepsilon > 0$

Ensure: Approximation x^* d'un point critique de f

```

1:  $k \leftarrow 0$ 
2: while  $\|\nabla f(x_k)\| > \varepsilon$  do
3:   Calculer le gradient  $\nabla f(x_k)$  et la hessienne  $H_f(x_k)$ 
4:   if  $H_f(x_k)$  est définie positive then
5:      $D_k \leftarrow H_f(x_k)$ 
6:   else
7:     Choisir  $\mu > 0$  tel que  $D_k = H_f(x_k) + \mu I$  soit définie positive
8:   end if
9:   Résoudre  $D_k d_k = -\nabla f(x_k)$ 
10:  Effectuer une recherche linéaire pour trouver  $\alpha_k > 0$  satisfaisant la condition de Wolfe
    ou d'Armijo
11:  Mettre à jour  $x_{k+1} \leftarrow x_k + \alpha_k d_k$ 
12:   $k \leftarrow k + 1$ 
13: end while
14: return  $x^* \leftarrow x_k$ 

```

1.2.6 Méthode de quasi-Newton (BFGS)

Comme dit précédemment, la méthode de Newton possède quelques désavantages que l'on souhaiterait résoudre. En effet, la hessienne est parfois impossible à calculer si la fonction n'est pas deux fois dérivable. De plus, une fois calculée, il peut être très coûteux de l'évaluer en un point. Nous allons donc ici chercher à estimer trouver une estimation M de cette hessienne ou de son inverse (notée W).

Le schéma d'un algorithme de quasi-Newton est le même que la méthode de Newton hybride dans lequel on remplace la résolution du système linéaire :

$$H_f(x_k)d_k = -\nabla f(x_k)$$

par la résolution d'un autre système :

$$M_k d_k = -\nabla f(x_k) \quad \text{ou} \quad d_k = -W_k \nabla f(x_k)$$

où M_k ou W_k sont des matrices construites grâce à des *formules de mise à jour*.

Proposition Soient

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$$

la mise à jour de W_k pour obtenir W_{k+1} est donnée par :

$$W_{k+1} = \left(I - \frac{s_k y_k^\top}{y_k^\top s_k} \right) W_k \left(I - \frac{y_k s_k^\top}{y_k^\top s_k} \right) + \frac{s_k s_k^\top}{y_k^\top s_k}$$

Cette formule garantit que W_{k+1} reste symétrique et définie positive à condition que $y_k^\top s_k > 0$. La direction de descente est alors simplement :

$$d_k = -W_k \nabla f(x_k)$$

Algorithm 5 Algorithme quasi-Newton (BFGS)

Require: Fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$, point initial x_0 , tolérance $\varepsilon > 0$, matrice initiale W_0 définie positive

Ensure: Approximation x^* d'un minimum local de f

```

1:  $k \leftarrow 0$ 
2: while  $\|\nabla f(x_k)\| > \varepsilon$  do
3:   Calculer le gradient  $g_k \leftarrow \nabla f(x_k)$ 
4:   Déterminer la direction de descente  $d_k \leftarrow -W_k g_k$ 
5:   Effectuer une recherche linéaire pour obtenir  $\alpha_k > 0$ 
6:   Mettre à jour  $x_{k+1} \leftarrow x_k + \alpha_k d_k$ 
7:   Calculer  $s_k \leftarrow x_{k+1} - x_k$ ,  $y_k \leftarrow \nabla f(x_{k+1}) - \nabla f(x_k)$ 
8:   Mettre à jour la matrice inverse de Hessienne :
```

$$W_{k+1} \leftarrow \left(I - \frac{s_k y_k^\top}{y_k^\top s_k} \right) W_k \left(I - \frac{y_k s_k^\top}{y_k^\top s_k} \right) + \frac{s_k s_k^\top}{y_k^\top s_k}$$

```

9:    $k \leftarrow k + 1$ 
10: end while
11: return  $x^* \leftarrow x_k$ 
```

1.3 Problèmes Quadratiques

Dans cette section, nous nous intéressons maintenant à des problèmes d'optimisation dits *quadratiques* de la forme :

$$\min_x f(x) = \frac{1}{2} \langle x, Qx \rangle + \langle b, x \rangle + c$$

où Q est symétrique définie positive, $b \in \mathbb{R}^n$ et $c \in \mathbb{R}$. On remarque tout de suite que :

- f est continue et coercive, donc elle admet un minimum global (pas nécessairement unique).
- f est strictement convexe, donc elle admet un unique minimum global $\bar{x}$ qui vérifie $\nabla f(\bar{x}) = Q\bar{x} + b = 0$ donc $\bar{x} = -Q^{-1}b$
- La solution ne dépend pas de la valeur de c que nous supposons nulle par la suite.

Proposition La première approche pour résoudre un tel problème est de simplement inverser la matrice Q en résolvant le système $Qx = b$. On peut par exemple utiliser la factorisation de Choleski de la forme $Q = L^t L$ où L est triangulaire inférieure. Il nous suffit alors de résoudre deux problèmes triangulaires inférieurs :

$$Ly = -b \quad \text{et} \quad {}^t Lx = y$$

Or inverser la matrice Q ou même en trouver une factorisation de Choleski peut être très coûteux. L'objectif de la méthode des gradient conjugués est justement de trouver une telle solution sans jamais avoir à inverser Q .

La descente de gradient classique progresse selon la direction du gradient, perpendiculaire aux courbes de niveau de f . Si la matrice Q déforme l'espace (par exemple dans une vallée elliptique), cette méthode "zigzague" et avance lentement.

L'idée du *gradient conjugué* est d'adapter la géométrie de la descente à celle de la fonction. Plutôt que de se déplacer selon des directions simplement orthogonales, on choisit des directions dites *Q-conjuguées*, c'est-à-dire orthogonales pour le produit scalaire défini par Q :

$$\langle d_i, Qd_j \rangle = 0 \quad \text{si } i \neq j.$$

Chaque direction de recherche “corrige” alors la solution dans une dimension nouvelle, sans défaire les progrès obtenus aux itérations précédentes.

Ainsi, en théorie, une famille de n directions Q -conjuguées permet d’atteindre la solution exacte en au plus n étapes. La méthode exploite donc la structure quadratique du problème pour converger beaucoup plus rapidement qu’une descente de gradient classique, tout en évitant le coût d’une inversion de matrice.

1.3.1 Généralités

Commençons donc par définir la notion de vecteurs conjugués.

Définition (Vecteurs conjugués) . Soit $d_1, \dots, d_m \in \mathbb{R}^n$ une famille de vecteurs. On dit qu’ils sont Q -conjugués lorsque :

- $\forall j \in \llbracket 1, m \rrbracket, d_j \neq 0$
- $\forall j, k \in \llbracket 1, m \rrbracket, j \neq k, \langle d_j, Qd_k \rangle = 0$

Lemme Une famille de vecteurs Q -conjugués est nécessairement *libre*.

Ce lemme nous assure que l’on ne peut pas construire une famille de plus de n vecteurs Q -conjugués. On peut donc construire une méthode itérative de la forme :

$$x_{k+1} = x_k + \alpha_k d_k$$

avec les caractéristiques suivantes :

- les directions d_k forment une famille de vecteurs Q -conjugués (donc nous aurons au plus n itérations).
- α_k minimise la fonction $\alpha \mapsto f(x_k + \alpha d_k)$ (possible car la fonction est quadratique)

Lemme Soit $(d_0, \dots, d_{n-1})$ une famille de vecteurs Q -conjugués et soit $x_1, x_2, \dots$ les itérés produits par la méthode précédente. Alors pour tout $k \in \{0, \dots, n-1\}$ le pas optimal α_k est donné par :

$$\alpha_k = -\frac{\langle d_k, \nabla f(x_k) \rangle}{\langle d_k, Qd_k \rangle} = -\frac{\langle d_k, Qx_k + b \rangle}{\langle d_k, Qd_k \rangle}$$

Lemme (Fondamental) Pour tout $k \in \{0, n-1\}$ $\nabla f(x_k)$ est orthogonal à $\text{vect}(d_0, d_{k-1})$. En particulier, $\nabla f(x_n) = 0$.

Ce lemme nous montre donc que la solution à notre problème de minimisation est atteinte en au plus n itérations.

Proposition Ainsi, pour tout $k \in \{0, n-1\}$ l’itéré x_k minimise f sur le sous-espace affine

$$M_k = \{x_0\} + \text{vect}(d_0, \dots, d_{k-1})$$

1.3.2 Algorithme

Maintenant que nous savons comment construire une solution, il nous reste à construire une famille de vecteurs Q -conjugués. On peut remarquer que la Q -conjugaison est en réalité l’orthogonalité par rapport au produit scalaire $x, y \mapsto \langle x, Qy \rangle$. Définissons donc un procédé de Gram-Schmidt généralisé.

Proposition (Procédé de Gram-Schmidt généralisé) Soit $f_0, \dots, f_{m-1}$ une famille de vecteurs libres dans $\mathbb{R}^n$. Soit $Q \in \mathcal{M}_n(\mathbb{R})$ une matrice symétrique définie positive. On définit la famille $d_0, \dots, d_{m-1}$ par :

$$\begin{aligned} d_0 &= f_0 \\ d_k &= f_k - \sum_{j=0}^{k-1} \frac{\langle d_j, Qf_k \rangle}{\langle d_j, Qd_j \rangle} d_j \quad \forall k \in \{1, \dots, m-1\} \end{aligned}$$

c'est alors une base de $\text{vect}(f_0, \dots, f_{m-1})$ constituée de vecteurs Q -conjugués.

L'idée de la méthode du gradient conjugué est donc de construire successivement des directions Q -conjuguées, de manière à minimiser f sur des sous-espaces de plus en plus grands : après k itérations, x_k est le minimiseur de f sur le sous-espace affine $M_k = x_0 + \text{vect}(d_0, \dots, d_{k-1})$.

La direction initiale d_0 est choisie comme l'opposée du gradient initial $-y_0$, puis chaque nouvelle direction est obtenue par une combinaison linéaire du nouveau gradient et de la direction précédente, de façon à préserver la Q -conjugaison.

Cette construction conduit à l'algorithme suivant, qui atteint la solution exacte en au plus n itérations en arithmétique exacte (et converge très rapidement en pratique même pour des problèmes non exactement quadratiques).

Algorithm 6 Algorithme du Gradient Conjugué (GC)

Require: $Q \in \mathbb{S}_{++}^n$, $b \in \mathbb{R}^n$, point initial $x_0 \in \mathbb{R}^n$, tolérance $\varepsilon > 0$

Ensure: Approximation $\bar{x}$ du minimum global de $f(x) = \frac{1}{2}x^T Qx + b^T x$

1: $k \leftarrow 0$

2: Calculer le gradient initial $y_0 \leftarrow Qx_0 + b$

3: Initialiser la direction $d_0 \leftarrow -y_0$

4: **while** $\|y_k\| > \varepsilon$ **do**

5: Calculer le pas optimal

$$\alpha_k \leftarrow -\frac{\langle d_k, y_k \rangle}{\langle d_k, Qd_k \rangle}$$

6: Mettre à jour la position

$$x_{k+1} \leftarrow x_k + \alpha_k d_k$$

7: Calculer le nouveau gradient

$$y_{k+1} \leftarrow Qx_{k+1} + b$$

8: **if** $\|y_{k+1}\| \leq \varepsilon$ **then**

9: **return** $\bar{x} \leftarrow x_{k+1}$

▷ Condition d'arrêt

10: **end if**

11: Calculer le coefficient de conjugaison

$$\beta_{k+1} \leftarrow \frac{\|y_{k+1}\|^2}{\|y_k\|^2}$$

12: Mettre à jour la direction conjuguée

$$d_{k+1} \leftarrow -y_{k+1} + \beta_{k+1} d_k$$

13: $k \leftarrow k + 1$

14: **end while**

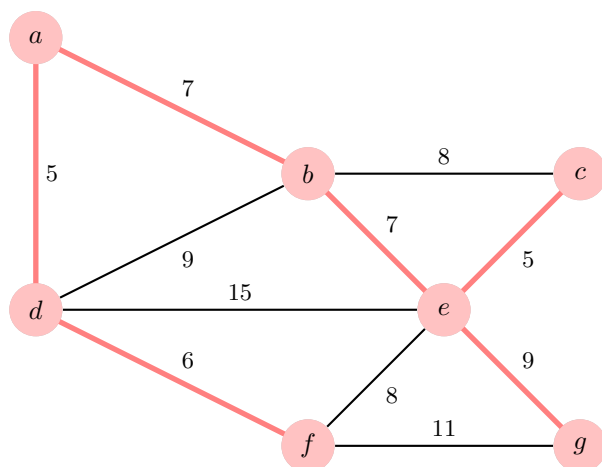
15: **return** $\bar{x} \leftarrow x_k$

1.4 Récapitulatif

Résumé des principales méthodes de descente

Méthode	Principe / Idée	Itération clé
Descente de gradient	Déplace x_k dans la direction opposée au gradient, qui indique la plus forte pente locale de décroissance.	$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$
Plus profonde descente	Choisit le pas α_k qui minimise exactement $f(x_k + \alpha d_k)$ le long de la direction de descente.	$\alpha_k = \arg \min_{\alpha > 0} f(x_k + \alpha d_k)$
Recherche linéaire (conditions)	On ne cherche pas α_k exact, mais on impose les conditions d'Armijo et de Wolfe pour garantir une décroissance suffisante.	$\begin{cases} f(x_k + \alpha_k d_k) \leq f(x_k) + c_1 \alpha_k \langle \nabla f(x_k), d_k \rangle, \\ \langle \nabla f(x_k + \alpha_k d_k), d_k \rangle \geq c_2 \langle \nabla f(x_k), d_k \rangle \end{cases}$
Méthode de Newton	Utilise l'approximation quadratique de f : ajuste la direction selon la courbure locale (Hessienne).	$x_{k+1} = x_k - [\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$
Méthode quasi-Newton (BFGS)	Approche itérative de Newton sans inverser la Hessienne : on met à jour une matrice $W_k \approx [\nabla^2 f(x_k)]^{-1}$.	$\begin{cases} d_k = -W_k \nabla f(x_k), \\ W_{k+1} = \left(I - \frac{s_k y_k^\top}{y_k^\top s_k} \right) W_k \left(I - \frac{y_k s_k^\top}{y_k^\top s_k} \right) + \frac{s_k s_k^\top}{y_k^\top s_k} \end{cases}$ avec $s_k = x_{k+1} - x_k$, $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$
Gradient conjugué	Exploite la structure quadratique : les directions successives sont Q -conjuguées, permettant de converger en au plus n étapes.	$\begin{cases} d_0 = -y_0, & y_k = Qx_k + b, \\ \alpha_k = -\frac{\langle d_k, y_k \rangle}{\langle d_k, Qd_k \rangle}, \\ \beta_{k+1} = \frac{\ y_{k+1}\ ^2}{\ y_k\ ^2}, \\ d_{k+1} = -y_{k+1} + \beta_{k+1} d_k, \\ x_{k+1} = x_k + \alpha_k d_k \end{cases}$

Algorithmique



Chapitre 1

Flot maximal dans un graphe

Contents

1.1	Rappels sur les graphes	75
1.2	Cycle et flot	77
1.3	Flot et coupure	78
1.3.1	Flot maximal dans un réseau	78
1.3.2	Théorème de Coupure minimale	79
1.4	Flot maximal : principe de Ford-Fulkerson	80
1.4.1	Recherche de Flot Maximal	80
1.4.2	Recherche de flot maximal de coût minimal	81

L'objectif de ce chapitre est d'être capable d'organiser efficacement un flot d'objets dans un réseau. Par exemple, on souhaite amener le maximum de camions d'un point A à un point B en passant par certaines routes. De même, on souhaiterait savoir comment router efficacement le plus de paquets dans un réseau informatique.

Tout ces problèmes se généralisent en ce que l'on appelle la *recherche de flot maximal dans un graphe orienté*. Nous allons ici définir proprement ce genre de problème, donner des conditions d'existence de flot maximal et proposer des algorithmes pour les construire.

1.1 Rappels sur les graphes

Pour représenter une structure de réseau (routier, informatique, etc...), on utilise les graphes. Ce cours fait donc suite à celui sur la théorie des graphes enseigné en licence. Effectuons quelques rappels.

Définition (Graphe Orienté) . On appelle graphe orienté un couple d'ensembles finis $G = (X, E)$ où $X = \{1, \dots, n\}$ représente les sommets du graphe et $E = \{(x_i, y_j), \dots\}$, $x_i, y_j \in X$ l'ensemble ordonné des arrêtes du graphe. Pour un graphe orienté, les notions de graphe simple et multigraphes sont les même que pour le cas non orienté.

Exemple (Graphes et leur représentation gravarphique) Soient G_1 et G_2 deux graphes, en voici une représentation :

Définition (Voisinage) . Le voisinage d'un sommet x de G est l'ensemble des sommets

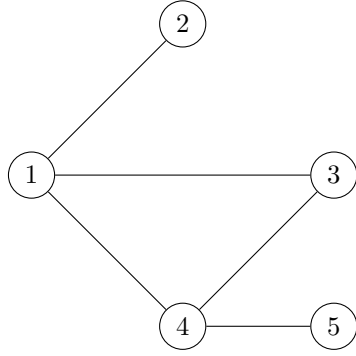


FIGURE 1.1 – Graphe non orienté

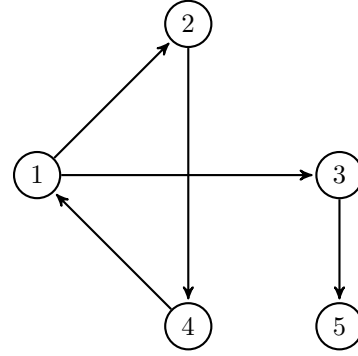


FIGURE 1.2 – Graphe orienté

y de G tels qu'il existe une arête entre x et y dans G . On le note $V(x)$.

Remarque Si x est dans le voisinage de y , on dira que x est adjacent à y et inversement.

Définition (Degré) . Le degré d'un sommet x de G est le cardinal du voisinage x . On le note $d(x)$. Un sommet de voisinage nul est dit *isolé* et un sommet de voisinage égal à 1 est dit *pendant*.

Définition (Voisinage entrant et sortant) . Soit G un graphe orienté et x un sommet de G . On définit deux types de voisinages :

- *Voisinage entrant* : noté $V^-(X)$ est l'ensemble des prédécesseurs de x .
- *Voisinage sortant* : noté $V^+(X)$ est l'ensemble des successeurs de x .

On définira comme précédemment le degré sortant et le degré entrant d'un sommet x .

Définition (Chaîne) . On appelle chaîne de $G = (X, E)$ de longueur n toute suite alternée de sommets et d'arêtes de G telle que :

$$c = (x_0, a_1, x_1, \dots, a_n, x_n), \text{ telle que } \forall i \in \llbracket 1, n \rrbracket, a_i = (x_{i-1}, x_i)$$

Ici, n représente le nombre d'arêtes de la chaîne. Dans le cas d'un graphe simple, on notera les chaînes de la façon suivante :

$$c = (x_0, \dots, x_n) \quad \text{ou} \quad c = x_0 - x_1 - \dots - x_n$$

Dans le cas des graphes orientés, on notera une chaîne simplement avec les arêtes dont elle est constituée :

$$c = (u_1, \dots, u_n), \forall i \in \llbracket 1, n \rrbracket u_i \in E$$

Définition (Accessibilité) . Soit $G = (X, E)$ un graphe. On a :

- Soient x et y deux sommets de G . On dit que y est accessible à partir de x s'il existe une chaîne joignant x et y dans G .
- G est dit *connexe* ssi $\forall x \in X, \forall y \in X, y$ est accessible à partir de x .

- L'accessibilité est une relation d'équivalence entre les sommets.
Ses classes d'équivalences sont les composantes connexes de G .

Définition (Chaîne Simple) . Une chaîne est *simple* si elle ne passe pas deux fois par la même *arrête* et elle est dite *élémentaire* si elle ne passe pas deux fois par le même *sommet*. On remarquera facilement qu'une chaîne élémentaire est simple.

Définition (Cycle) . Un cycle de G est une chaîne simple dont le départ et l'arrivée sont le même sommet. Un cycle est donc de la forme :

$$c = x_0 - x_1 - \dots - x_{n-1} - x_0$$

Théorème (Propriétés des cycles et chaînes) . Soit G un graphe à $n \in \mathbb{N}$ sommets.

- Toute chaîne élémentaire a une longueur inférieure à $n - 1$
- Toute cycle élémentaire a une longueur inférieure à n
- De toute chaîne, on peut en extraire une chaîne élémentaire.

1.2 Cycle et flot

Définissons maintenant la notion de flot dans un cas général.

Définition (Vecteur de chaîne) . Soit $c = (u_1, \dots, u_n)$ une *chaîne élémentaire* dans un graphe orienté $G = (X, E)$ tel que $|E| = n$ et d'arcs u_i . On appelle *vecteur de chaîne* associé à c l'application φ associée à c telle que :

$$\varphi : \begin{cases} E \longrightarrow \mathbb{Z} \\ u_i \longmapsto \begin{cases} 1 & \text{si } u_i \text{ est dans le sens de } c \\ -1 & \text{si } u_i \text{ n'est pas dans le sens de } c \\ 0 & \text{si } u_i \text{ n'est pas dans } c \end{cases} \end{cases}$$

On la note en général $\varphi = (\varphi(u_1), \varphi(u_2), \dots, \varphi(u_n)) \in \mathbb{Z}^n$.

Venons en maintenant à la définition phare de ce chapitre :

Définition (Flot) . Soit $G = (X, E, c)$ un graphe orienté. On appelle *flot* dans G tout vecteur de cycle φ de G qui vérifie la loi de Kirchoff :

$$\forall x \in X, \quad \sum_{u_i \in \omega^-(\{x\})} \varphi(u_i) = \sum_{u_i \in \omega^+(\{x\})} \varphi(u_i)$$

Et on note $\varphi^i = \varphi(u_i)$ le *flot* de l'arrête u_i .

Remarque Un flot est donc un chaîne élémentaire d'un graphe orienté dont, pour chaque noeud, la somme des flots sortants est égale à la somme des flots entrants.

Propriété (Flots) . Soit $G = (X, E, c)$ un graphe tel que $|E| = n$, alors :

- Le vecteur nul de $\mathbb{Z}^n$ est un flot de G .
- Toute combinaison linéaire de flots de G est aussi un flot de G .
- Tout vecteur cyclique de G est un flot de G .

Exemple Soit G le graphe suivant : Le vecteur $\varphi_1 = (2, -2, 1, 3, 4, -1, 1)$ est un flot de G . Le

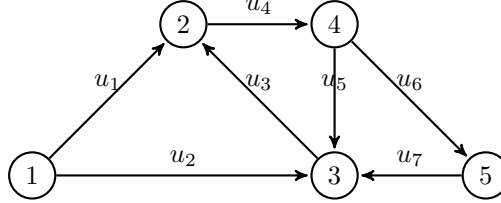


FIGURE 1.3 – flot dans un graphe

vecteur $\varphi_2 = (0, 0, 0, 0, 1, -1, 1)$ est aussi un flot. Soit $v = 2\varphi_1 + 3\varphi_2$, c'est aussi un flot de G .

Proposition (Loi de Kirchoff généralisée) Soit $G = (X, E, c)$ et $A \subset E$ non vide. Un chaîne φ est un flot de G ssi :

$$\sum_{u \in \omega^-(A)} \varphi(u) = \sum_{u \in \omega^+(A)} \varphi(u)$$

1.3 Flot et coupure

1.3.1 Flot maximal dans un réseau

Dans cette section, nous revenons à notre problème de départ en définissant la notion de graphe de transport. Graphes dans lesquels nous chercherons à construire des flots maximaux.

Définition (Réseau de Transport) . Un *réseau de transport* est un graphe orienté pondéré $R = (X, E, c)$ qui contient :

- un sommet sans prédécesseur appelé *source* (noté s).
- un sommet sans fils appelé *puits* (noté t).
- c , une application, qui à chaque arc, lui associe sa capacité notée c :

$$c : \begin{cases} E \longrightarrow \mathbb{R} \cup \{\infty\} \\ u = (x, y) \longmapsto c(u) \end{cases}$$

- un arc fictif de t à s doté d'une capacité infinie, appelé *arc retour*.

Définition (Flot compatible) . Un *flot compatible* sur un réseau $R = (X, E, c)$ tel que $|E| = n$ est un vecteur $\varphi \in \mathbb{Z}^n$ qui vérifie :

- (**Conservation de Kirchhoff**) pour tout $x \in X \setminus \{s, t\}$ (hors source s et puits t), on a :

$$\sum_{u \in \omega^+(x)} \varphi(u) = \sum_{u \in \omega^-(x)} \varphi(u)$$

où $\omega^+(x)$ (resp. $\omega^-(x)$) est l'ensemble des arcs sortants (resp. entrants) de x .

— (**Compatibilité avec les capacités**) :

$$\forall u \in E, \quad 0 \leq \varphi(u) \leq c(u).$$

La *valeur* du flot, notée $v(\varphi)$, est définie par :

$$v(\varphi) := \sum_{u \in \omega^+(s)} \varphi(u) - \sum_{u \in \omega^-(s)} \varphi(u),$$

c'est-à-dire le flux net sortant de la source (équivalent au flux net entrant dans le puits). Enfin, on dit qu'un arc u est *saturé* si $\varphi(u) = c(u)$.

Exemple Voici un exemple de réseau de transport :

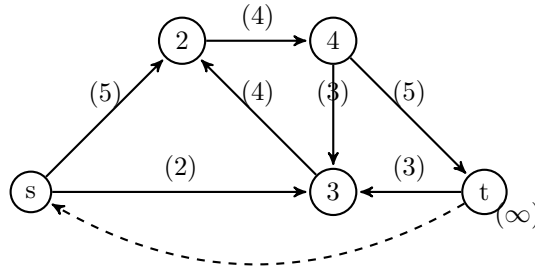


FIGURE 1.4 – Réseau de Transport

Définition (Flot maximal) . Un *flot maximal* d'un réseau est un flot compatible qui maximise la valeur $v(\varphi)$ parmi tous les flots compatibles du réseau.

1.3.2 Théorème de Coupure minimale

Pour construire un flot maximal dans un réseau, nous avons besoin d'un peu de théorie.

Définition (Coupure et cocycle) . Soit $R = (X, E, c)$ un réseau, où X est l'ensemble des sommets, E l'ensemble des arêtes et $c : E \rightarrow \mathbb{R}^+$ la fonction de capacité.

Une *coupure* ct séparant s de t est un couple $(A, X \setminus A)$ tel que :

$$A \subset E, \quad s \in A, \quad t \notin A.$$

L'ensemble des arcs de la coupure, noté $\text{arcs}(ct)$ est l'ensemble :

$$\text{arcs}(ct) = \omega^+(A) := \{u = (i, j) \in E \mid i \in A, j \in X \setminus A\}.$$

Qui correspond à l'ensemble des arcs qui, si on les enlève, coupe tout les chemins entre s et t .

Définition (Capacité d'une coupure) . La *capacité* d'une coupure ct est la somme des

capacités de ses arêtes :

$$\text{capa}(ct) := \sum_{u \in \text{arcs}(ct)} c(u).$$

Théorème (Flot et coupure minimale) . Pour tout flot compatible φ et pour toute coupure ct d'un réseau R , on a :

$$v(\varphi) \leq \text{capa}(ct)$$

1.4 Flot maximal : principe de Ford-Fulkerson

L'objectif de l'algorithme de Ford-Fulkerson est de construire de manière itérative un flot maximal. Pour cela, on part du flot nul dans le réseau. Et petit-à petit, on cherche des chemins non saturés de la source s vers le puit t pour lesquels on peut augmenter leur flot. Une fois que l'on n'est plus capable de trouver de chemin, alors l'algorithme se termine. La définition ci-dessous détaille le processus de construction d'un tel chemin.

1.4.1 Recherche de Flot Maximal

Définition (Principe de Ford-Fulkerson) . Soit $R = (X, E, c)$ un réseau et φ un flow sur R . L'algorithme de construction d'un chemin non saturé est :

1. **Initialisation** : On marque s
2. **Itération** : Pour tout sommet marqué $x, y \in X$ peut être marqué si :
 - y n'est pas encore marqué
 - $\begin{cases} \exists u = (x, y) \in E, \varphi(u) < c(u) & \text{(marquage direct)} \\ \exists u = (y, x) \in E, \varphi(u) < c(u) & \text{(marquage indirect)} \end{cases}$

L'algorithme se termine si on arrive à marquer le puit t . Dans ce cas, la chaîne produite est alors appelée *chaîne augmentante*, notée ch . On note ch^+ les sommets de ch marqués directement et ch^- les sommets marqués indirectement.

Propriété (Calcul d'un nouveau flot) . Soit $R = (X, E, c)$ un réseau et φ un flow sur R . Soit ch une chaîne augmentante de R . Il est alors possible d'augmenter la valeur du flot φ de $k \in \mathbb{R}$ en ajoutant à chaque arcs le flot défini par :

$$k = \min \left(\min_{u \in ch^+} \{c(u) - \varphi(u)\}, \min_{u \in ch^-} \{\varphi(u)\} \right)$$

Le nouveau flot est obtenu en augmentant de k le flow des arcs de ch^+ et en diminuant le flot de k des arcs de ch^- .

Théorème (Coupure Minimale - Flot Maximal) . Soit $R = (X, E, c)$ un réseau. Soit F l'ensemble des flots possibles sur R et K l'ensemble des coupures possibles de R . Alors la valeur du flot maximal est égale à la capacité de la coupure minimale.

$$\text{i.e. } \max_{\varphi \in F} \{v(\varphi)\} = \min_{ct \in K} \{\text{capa}(ct)\}$$

Ce théorème nous permet donc de trouver directement la valeur du flot maximal que l'on peut faire passer dans notre réseau sans avoir à en construire un.

Remarque La dernière itération de l'algorithme de Ford-Fulkerson nous donne une coupure minimale. Les sommets marqués correspondent aux sommets de l'ensemble A de la coupure $ct = (AX/A)$.

1.4.2 Recherche de flot maximal de coût minimal

Rajoutons maintenant une contrainte dans la recherche de notre flot maximal. On souhaite trouver un flot maximal dans un réseau pour lequel le coût du flot sera le plus faible. On cherche donc à faire passer le plus de marchandise dans notre réseau par les routes qui "coûtent le moins cher".

Nous devons donc définir un nouveau type de réseau intégrant cette notion de coût.

Définition (Réseau à coût) . Soit $R = (X, E, c)$ un réseau de noeuds E , d'arcs E et de fonction de capacité c . On appelle *réseau à coût* le réseau R auquel on a rajouté une fonction de coût γ , qui à chaque arête lui associe un coût :

$$\gamma : \begin{cases} E \longrightarrow \mathbb{R} \\ (x, y) \longmapsto \gamma(x, y) \end{cases}$$

Exemple On représentera donc un réseau à coût comme un réseau normal mais en rajoutant une valeur numérique sur chaque arête correspondant à son coût :

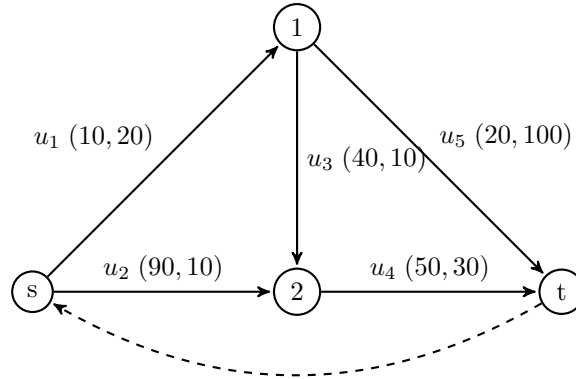


FIGURE 1.5 – Réseau de transport à coût

Remarque Chercher un flot maximal de coût minimal dans R revient donc à construire un flot via l'algorithme de Ford-Fulkerson en choisissant une chaîne augmentante de coût minimal à chaque itération.

Pour trouver cette chaîne augmentante de coût minimal, nous devons définir le graphe résiduel d'un réseau.

Définition (Graphe Résiduel) . Soit $R = (X, E, c, \gamma)$ un réseau et φ un flot sur R , on définit le *graphe résiduel* $G_R(\varphi) = (X, U_\varphi, r_\varphi)$ associé à R et φ comme le graphe tel que pour tout arc $(x, y) \in E$, on ait :

- un *arc direct* $(x, y) \in U_\varphi$ si $\varphi(x, y) < c(x, y)$ avec $r_\varphi = c(x, y) - \varphi(x, y)$
- un *arc indirect* $(y, x) \in U_\varphi$ si $\varphi(y, x) > 0$ avec $r_\varphi = \varphi(y, x)$.

Chapitre 2

Programmation Linéaire

Contents

2.1	Programmation Linéaire	82
2.1.1	Généralités	82
2.1.2	Solution Géométrique	85
2.1.3	Algorithme du Simplexe	86
2.1.4	Dualité	89
2.2	Programmation Linéaire en Nombres Entiers	91
2.2.1	Définition et relaxation	91
2.2.2	Algorithme "Branch and Bound"	93

Intéressons nous maintenant aux problèmes de programmation linéaire. Même si le nom peut prêter à confusion, il s'agit en réalité de problèmes d'optimisation d'une *fonction objectif* en respectant certaines *contraintes*. Le caractère linéaire de tels problèmes vient du fait que nous ne traiterons que des fonctions objectifs et des contraintes linéaires.

Ces problèmes de programmation linéaires peuvent être résolus en temps polynomial, notamment grâce à l'algorithme du simple. Les problème de flots maximaux, vus précédemment, peuvent s'exprimer comme des problèmes de programmation linéaire.

2.1 Programmation Linéaire

2.1.1 Généralités

Définition (Programme linéaire) . Un *programme linéaire* ou *problème de programmation linéaire* est un problème d'optimisation de $n \in \mathbb{N}$ variables et $m \in \mathbb{N}$ contraintes de la forme :

$$\begin{aligned} \text{minimize} \quad & z = \sum_{j=1}^n c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^n a_{ij} x_j \preceq b_i \quad \forall i \in \llbracket 1, m \rrbracket, \\ & x \succeq 0 \quad \forall j \in \llbracket 1, n \rrbracket \end{aligned}$$

où $\preceq \in \{\leq, \geq, =\}$. Un programme linéaire est donc un simple problème d'optimisation sur un polyèdre convexe. Dans le cas défini plus haut, on dit que le programme est sous forme générale. Différentes formes d'un même problème existent.

Exemple Voyons dès à présent un exemple de programme linéaire.

Un barman possède 30 litres de jus d'orange, 24 litres de pamplemousse et 18 litres de jus de framboise. Il peut préparer 2 types de cocktails :

- *Le "sunrise", vendu 80 euros le baril de 10 litres composé de 5 litres de jus d'orange, de 2 litres de jus de pamplemousse, d'un litre de jus de framboise et de 2 litres d'eau.*
- *Le "swing", vendu 60 euros le baril et composé de 3 litres de jus de pamplemousse, 3 litres de jus d'orange, 3 litres de jus de framboise et d'un litre d'eau.*

Il souhaite préparer la plus importante quantité des deux cocktails pour maximiser son profit.

Pour résoudre ce problème, nous pouvons construire un programme linéaire de deux variables x et y représentant chacune le nombre de litres d'un des deux cocktails à produire. La fonction objectif, décrivant le prix de revente de la marchandise est donc :

$$z = 80x + 60y$$

Et nous avons les contraintes suivantes :

$$\begin{aligned} 5x + 3y &\leq 30 && \text{(stock de jus d'orange)} \\ 2x + 3y &\leq 24 && \text{(stock de jus de pamplemousse)} \\ x + 3y &\leq 18 && \text{(stock de jus de framboise)} \\ x, z &\geq 0 \end{aligned}$$

On peut donc formuler le problème suivant :

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y \leq 30, \\ & 2x + 3y \leq 24, \\ & x + 3y \leq 18, \\ & x, y \geq 0 \end{aligned}$$

Proposition Comme dit dans la définition d'un programme linéaire, il peut exister différentes formes d'un même problème en fonction de la méthode de résolution utilisée. Le caractère linéaire de ces programmes nous permet de passer facilement d'une forme à une autre. Nous définissons la forme canonique d'un programme linéaire comme suit :

$$\begin{aligned} \text{maximize} \quad & z = \sum_{j=1}^n c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^n a_{ij} x_j = b_i, \quad \forall i \in \llbracket 1, m \rrbracket, \\ & x_j \geq 0, \quad \forall j \in \llbracket 1, n \rrbracket \end{aligned}$$

Nous pouvons ensuite passer à la formulation dite « standard » où les contraintes d'ordre sont remplacées par des contraintes d'égalités. Il faut donc rajouter un certain nombre de variables à notre programme.

FIGURE 2.2 – Formulation standard d'un programme linéaire de maximisation

$$\begin{aligned}
&\text{maximize} && z = \sum_{j=1}^n c_j x_j \\
&\text{subject to} && \sum_{j=1}^n a_{ij} x_j \preceq b_i, \quad \forall i \in \llbracket 1, m \rrbracket, \\
&&& x_j \succeq 0, \quad \forall j \in \llbracket 1, n \rrbracket
\end{aligned}$$

FIGURE 2.1 – Formulation canonique d'un programme linéaire

$$\begin{aligned}
&\text{maximize} && z = CX \\
&\text{subject to} && AX = B, \\
&&& X \geq 0_{\mathbb{R}^n}
\end{aligned}$$

FIGURE 2.3 – Formulation matricielle d'un programme linéaire de maximisation

Enfin, nous avons la forme matricielle, qui nous permettra d'utiliser des algorithmes itératifs pour résoudre ce genre de problèmes :
où $C \in \mathbb{R}^n$, $A \in \mathcal{M}_{m,n}(\mathbb{R})$ et $B \in \mathbb{R}^m$.

Exemple Pour le problème de notre barman, nous pouvons ainsi représenter les différentes formes de notre problème :

$$\begin{aligned}
&\text{maximize} && z = 80x + 60y \\
&\text{subject to} && 5x + 3y \leq 30, \\
&&& 2x + 3y \leq 24, \\
&&& x + 3y \leq 18, \\
&&& x, y \geq 0
\end{aligned}$$

FIGURE 2.4 – Forme canonique du problème du barman

$$\begin{aligned}
&\text{maximize} && z = 80x + 60y \\
&\text{subject to} && 5x + 3y + e_1 = 30, \\
&&& 2x + 3y + e_2 = 24, \\
&&& x + 3y + e_3 = 18, \\
&&& x, y, e_1, e_2, e_3 \geq 0
\end{aligned}$$

FIGURE 2.5 – Forme standard du problème du barman

Et enfin :

$$\begin{aligned}
&\text{maximize} && z = \underbrace{(80, 60, 0, 0, 0)}_C \times \underbrace{\begin{pmatrix} x \\ y \\ e_1 \\ e_2 \\ e_3 \end{pmatrix}}_X \\
&\text{subject to} && \underbrace{\begin{pmatrix} 5 & 3 & 1 & 0 & 0 \\ 2 & 3 & 0 & 1 & 0 \\ 1 & 3 & 0 & 0 & 1 \end{pmatrix}}_A \times \underbrace{\begin{pmatrix} x \\ y \\ e_1 \\ e_2 \\ e_3 \end{pmatrix}}_X = \underbrace{\begin{pmatrix} 30 \\ 24 \\ 18 \end{pmatrix}}_B
\end{aligned}$$

Maintenant que nous avons défini formellement un programme linéaire, passons à l'étude de ses solutions.

Définition (Solutions d'un programme linéaire) . Soit P un programme linéaire sous forme matricielle standard. Le vecteur $X = (x_1, \dots, x_n)$ est :

- une solution de P si $A^t X = B$
- une solution possible de P si $A^t X = B$ et $X \geq 0_{\mathbb{R}^n}$
- une solution optimale de P si c'est une solution possible et qu'elle maximise z parmi l'ensemble des solutions possibles.

Proposition Comme énoncé en introduction, on peut représenter un problème de maximisation de flot dans un réseau par un programme linéaire. Soit $R = (X, E, c)$ un réseau tel que $|E| = n$, avec $s, t \in X$ la source et le puits. On note $c(x, y)$ la capacité de l'arête entre x et y dans le réseau. On cherche une application

$$\varphi : E \longrightarrow \mathbb{R}, \quad (x, y) \longmapsto f(x, y)$$

qui maximise le flot entre la source s et le puits t .

Plus formellement, si l'on ordonne les arêtes E sous la forme $E = \{e_1, \dots, e_n\}$ et que l'on note $\varphi(E) = (f_1, \dots, f_n) \in \mathbb{R}^n$, alors on cherche à maximiser

$$z = \sum_{(x,y) \in v^-(s)} \varphi(x, y)$$

sous les contraintes :

$$0 \leq f(x, y) \leq c(x, y), \quad \forall (x, y) \in E$$

ainsi que les contraintes de conservation du flot pour tout sommet $v \in X \setminus \{s, t\}$:

$$\sum_{(u,v) \in v^-(v)} f(u, v) = \sum_{(v,w) \in v^+(v)} f(v, w).$$

2.1.2 Solution Géométrique

La recherche d'une solution d'un programme linéaire revient à trouver une droite de la forme $z = \sum_{i=1}^n x_i c_i$ pour laquelle z est maximal tout en respectant les contraintes du programme. En petite dimension (2 variables ou moins) il est possible de tracer ce problème et de le résoudre géométriquement. Détaillons cela autour d'un exemple.

Exemple (Solution Géométrique) Reprenons le problème de notre barman énoncé comme suit :

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y \leq 30, \\ & 2x + 3y \leq 24, \\ & x + 3y \leq 18, \\ & x, y \geq 0 \end{aligned}$$

On peut tracer les courbes des contraintes pour identifier la zone contenant les solutions possibles :

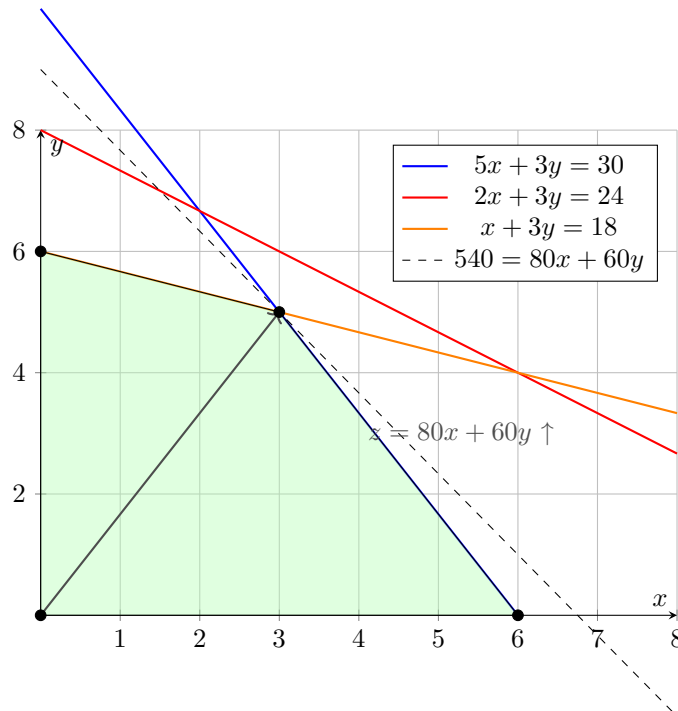


FIGURE 2.6 – Espace des solutions réalisables du programme linéaire
Le barman peut donc produire 80 litres de "sunrise" et 3 litres de "swing" pour un profit total de 540 euros.

Remarque En pratique, il sera très rare de pouvoir résoudre un tel problème graphiquement car le nombre de variable sera bien supérieur à 2.

2.1.3 Algorithme du Simplexe

Comme dit précédemment, le caractère linéaire des contraintes d'un programme linéaire nous permettent d'en déduire que l'espace des solutions possible est un polyèdre. D'où le théorème suivant.

Théorème (Solutions Possibles) . L'ensemble S des solutions possibles d'un programme linéaire est convexe et est un polyèdre non borné ou un polytope. Si S est non vide et borné, alors il existe au moins un vecteur de S pour lequel le maximum de la fonction objectif est atteint.

Remarque Ce théorème nous permet de distinguer plusieurs cas pour l'ensemble des solutions :

- Le programme linéaire a au moins une solution :
 - soit elle est unique
 - soit il en existe une infinité (i.e le maximum est atteint en plusieurs vecteurs) on parle alors de cas dégénéré.
- Le programme linéaire n'a pas de solution, lorsque les contraintes ne sont pas compatibles, l'ensemble des solutions est donc vide.
- Le programme linéaire n'a pas d'optimum (i.e l'ensemble des solutions n'est pas borné).

Proposition (Résolution d'un programme linéaire par le simplexe) Nous avons précédemment résolu nos programmes linéaires graphiquement. Nous allons voir que nous pouvons

aussi les résoudre *algébriquement*, en utilisant l'*algorithme du simplexe*. Soit le programme suivant :

$$\begin{aligned} \text{maximize} \quad & z = x + 2y \\ \text{subject to} \quad & 4x + 2y \leq 300, \\ & 5x \leq 310, \\ & 2x + 4y \leq 200, \\ & 3y \leq 100, \\ & x, y \geq 0 \end{aligned}$$

On passe ce programme sous *forme standard* en introduisant des variables d'écart $e_1, e_2, e_3, e_4 \geq 0$:

$$\begin{aligned} \text{maximize} \quad & z = x + 2y \\ \text{subject to} \quad & 4x + 2y + e_1 = 300, \\ & 5x + e_2 = 310, \\ & 2x + 4y + e_3 = 200, \\ & 3y + e_4 = 100, \\ & x, y, e_1, e_2, e_3, e_4 \geq 0 \end{aligned}$$

Le vecteur $(x, y, e_1, e_2, e_3, e_4) = (0, 0, 300, 310, 200, 100)$ est une *solution de base réalisable* : toutes les contraintes sont vérifiées et les variables sont positives. L'idée du simplexe est de :

1. Choisir une *base initiale* (ici, les variables d'écart e_i).
2. Exprimer les variables hors base (x, y) en fonction des variables de base.
3. *Augmenter la valeur de la fonction objectif* z en remplaçant une variable de base par une variable hors base qui améliore z .
4. Répéter jusqu'à ce qu'aucune amélioration ne soit possible : la solution est alors optimale.

Formellement, le simplexe cherche à écrire le système sous la forme :

$$\begin{cases} z = C + \sum_i c_i x_i \\ x_B = b - Ax_N \end{cases} \quad \text{où } x_B \geq 0, x_N \geq 0$$

À chaque itération :

- une variable hors base entre dans la base (celle qui augmente le plus z),
- une variable de base quitte la base (celle qui devient nulle en premier).

Lorsque tous les coefficients $c_i \leq 0$, aucune amélioration n'est possible et la solution actuelle est *optimale* : la valeur optimale est alors $z = C$.

Exemple (Résolution algébrique du programme par le simplexe) Considérons le programme linéaire suivant :

$$\begin{aligned} \text{maximize} \quad & z = x + 2y \\ \text{subject to} \quad & 4x + 2y \leq 300, \\ & 5x \leq 310, \\ & 2x + 4y \leq 200, \\ & 3y \leq 100, \\ & x, y \geq 0 \end{aligned}$$

1. **Mise sous forme standard :** On introduit une variable d'écart pour chaque contrainte :

$$\begin{cases} 4x + 2y + e_1 = 300 \\ 5x + e_2 = 310 \\ 2x + 4y + e_3 = 200 \\ 3y + e_4 = 100 \end{cases} \quad \text{avec } e_i \geq 0.$$

La fonction objectif s'écrit :

$$z = x + 2y.$$

Une solution de base réalisable évidente est obtenue en prenant $x = y = 0$, ce qui donne :

$$(e_1, e_2, e_3, e_4) = (300, 310, 200, 100), \quad z = 0.$$

C'est notre solution de base initiale.

2. **Recherche d'une direction d'amélioration :** Comme $z = x + 2y$, augmenter y est plus rentable que x (le coefficient 2 est supérieur à 1). On va donc chercher à **augmenter** y tant que c'est possible sans violer les contraintes. On part du système :

$$\begin{cases} e_1 = 300 - 2y - 4x \\ e_2 = 310 - 5x \\ e_3 = 200 - 2x - 4y \\ e_4 = 100 - 3y \end{cases}$$

Comme on garde $x = 0$ au départ, ces équations deviennent :

$$\begin{cases} e_1 = 300 - 2y \\ e_2 = 310 \\ e_3 = 200 - 4y \\ e_4 = 100 - 3y \end{cases}$$

Pour rester faisable, il faut $e_i \geq 0$. On obtient donc :

$$y \leq 150, \quad y \leq 50, \quad y \leq 25, \quad y \leq 33,3.$$

La contrainte la plus restrictive est $y \leq 25$ (provenant de $e_3 = 200 - 4y$). On choisit donc $y = 25$ et $e_3 = 0$. Cela définit une nouvelle base : $\{e_1, e_2, e_4, y\}$.

3. **Amélioration suivante :** Pour cette valeur $y = 25$, les variables d'écart valent :

$$e_1 = 300 - 2(25) = 250, \quad e_2 = 310, \quad e_3 = 0, \quad e_4 = 100 - 3(25) = 25.$$

La fonction objectif vaut alors :

$$z = x + 2y = 2 \times 25 = 50.$$

On peut encore améliorer z en augmentant x , tant que toutes les contraintes restent valides. En remplaçant $y = 25$, on obtient :

$$\begin{cases} e_1 = 300 - 2(25) - 4x = 250 - 4x \\ e_2 = 310 - 5x \\ e_3 = 200 - 2x - 4(25) = 100 - 2x \\ e_4 = 100 - 3(25) = 25 \end{cases}$$

Les contraintes donnent :

$$x \leq 62,5, \quad x \leq 62, \quad x \leq 50.$$

Le plus restrictif est $x \leq 50$ (provenant de $e_3 \geq 0$). On prend donc $x = 50$ et $e_3 = 0$.

4. **Calcul de la solution optimale** : On obtient :

$$\begin{cases} x = 50, & y = 25, \\ e_1 = 300 - 4(50) - 2(25) = 100, \\ e_2 = 310 - 5(50) = 60, \\ e_3 = 0, \\ e_4 = 100 - 3(25) = 25. \end{cases}$$

La fonction objectif vaut :

$$z = x + 2y = 50 + 2 \times 25 = 100.$$

Si on essaye d'augmenter y ou x davantage, on viole immédiatement une contrainte. La solution est donc **optimale**.

5. **Conclusion** : Le programme atteint son maximum en :

$$x^* = 50, \quad y^* = 25, \quad z_{\max} = 100.$$

Cette résolution illustre la logique du simplexe : on part d'une solution réalisable triviale, puis on améliore la fonction objectif pas à pas en déplaçant la base vers les sommets adjacents du polytope faisable, jusqu'à atteindre un sommet où toute augmentation violerait une contrainte.

2.1.4 Dualité

Définition (Programme Primal) . Le *problème primal* (ou primaire) est la forme standard d'un programme linéaire :

$$\begin{aligned} &\text{maximize} && z = {}^t c x \\ &\text{subject to} && A x \leq b, \\ &&& x \geq 0 \end{aligned}$$

où :

- $x \in \mathbb{R}^n$ est le vecteur des variables de décision ;
- $c \in \mathbb{R}^n$ est le vecteur des coefficients de la fonction objectif ;
- $A \in \mathbb{R}^{m \times n}$ est la matrice des coefficients des contraintes ;
- $b \in \mathbb{R}^m$ est le vecteur des bornes du système d'inégalités.

Définition (Problème Dual) . À tout problème primal est associé un *problème dual* défini par :

$$\begin{aligned} &\text{minimize} && w = {}^t b y \\ &\text{subject to} && {}^t A y \geq c, \\ &&& y \geq 0 \end{aligned}$$

où :

- $y \in \mathbb{R}^m$ est le vecteur des variables duales ;

- chaque contrainte du primal correspond à une variable du dual ;
- chaque variable du primal correspond à une contrainte du dual.

Remarque (Interprétation) Le problème *primal* cherche à *maximiser un profit* ou une valeur objective en respectant des contraintes de ressources. Le problème *dual* cherche à *minimiser le coût* de ces ressources tout en garantissant que la valeur de chaque ressource couvre la contribution de chaque variable du primal. Autrement dit :

Le primal cherche ce qu'on peut obtenir au maximum avec des ressources limitées, tandis que le dual cherche le coût minimal que doivent avoir ces ressources pour que le système reste équilibré.

Exemple En reprenant l'exemple de notre barman, le programme primal cherche à maximiser le profit du barman en fixant le prix des barrils c_j de cocktails j qu'il produit en utilisant au plus b_i litres de jus i . Les variables x_j représentent alors le nombre de barrils de cocktail j . Ici, le programme dual cherche à minimiser le profit total obtenu grâce aux stocks b_i de chaque jus i si le barman vend chaque cocktail j pour un prix minimal de c_j . Les variables y_i représentent donc le profit que le barman réalise par litre de jus i .

Voici les expressions mathématiques de ces deux programmes :

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y \leq 30, \\ & 2x + 3y \leq 24, \\ & x + 3y \leq 18, \\ & x, y \geq 0 \end{aligned}$$

FIGURE 2.7 – Programme primal du barman

$$\begin{aligned} \text{minimize} \quad & w = 20y_1 + 24y_2 + 18y_3 \\ \text{subject to} \quad & 5y_1 + 2y_2 + y_3 \geq 80, \\ & 3y_1 + 3y_2 + 3y_3 \geq 60, \\ & y_1, y_2, y_3 \geq 0 \end{aligned}$$

FIGURE 2.8 – Programme dual du barman

La solution optimale du primal est $(x_1, x_2) = (3, 5)$ pour un profit maximal de 540. En revanche la solution du dual est $(y_1, y_2, y_3) = (15, 0, 5)$ pour un profit de 540 aussi.

Il existe un lien direct entre les solutions du dual et celles du primal. Elles sont décrites par les théorèmes de dualité.

Théorème (Dualité Faible) . La solution optimale du programme primal est inférieure ou égale à celle du dual.

Théorème (Dualité Forte) . Nous avons les deux implications suivantes :

1. Si l'un des deux programmes (primal ou dual) admet une solution optimale finie, alors l'autre aussi. De plus, la valeur optimale pour des deux programmes est la même.
2. Si l'ensemble des solutions de l'un des deux programmes est non borné, alors celui de l'autre est vide.

Remarque Le théorème de coupure minimale/flot maximal dans un réseau est l'équivalent du théorème de dualité forte ici. En effet, le problème de coupure minimale dans un réseau est l'exact dual du problème de flot maximal dans le même réseau.

Exemple (Utilité de la dualité) Considérons le programme linéaire suivant :

$$\begin{aligned} \text{maximize} \quad & x + 3x + 6y + z \\ \text{subject to} \quad & w + x + 5y + 6z \leq 20, \\ & w + 2x + y + 3z \leq 30, \\ & x, y, w, z \geq 0 \end{aligned}$$

Ce programme ne peut pas être résolu graphiquement du fait du trop grand nombre de variables. En passant au dual, nous aurons un programme linéaire à deux variables et 4 contraintes que l'on pourra résoudre graphiquement. Attention, la résolution du dual nous donnera seulement une idée de la valeur de la solution maximale.

Remarque Ainsi, passer au problème dual peut être avantageux dans plusieurs cas :

- Le dual peut être plus simple à résoudre (moins de variables ou moins de contraintes).
- Il peut exister une solution optimale évidente pour le dual mais pas pour le primal.
- La valeur de n'importe quelle solution du dual nous donne une borne supérieure pour la solution du primal.
- La solution optimale du dual nous donne des informations sur la solution optimale du primal.

2.2 Programmation Linéaire en Nombres Entiers

Dans cette section, nous allons traiter un nouveau type de problèmes : les *programmes linéaires en nombres entiers*. Vous l'aurez compris, il s'agit maintenant de considérer des programmes linéaires à variables entières. Nous allons donc une méthode pour les résoudre ainsi que quelques techniques de modélisation. Commençons tout d'abord par un exemple introductif.

Exemple Un commerçant souhaite remplir son sac à dos d'au plus n objets de poids et valeurs respectifs $p_i \in \mathbb{N}$ et $v_i \in \mathbb{N}$. Or le sac du commerçant ne peut contenir qu'un poids maximal $C \in \mathbb{N}$. Le commerçant veut choisir les objets à prendre de manière à maximiser la valeur emportée. On modélisera donc le problème de la façon suivante :

$$\begin{aligned} \text{maximize} \quad & V = \sum_{i=1}^n v_i \times x_i \\ \text{subject to} \quad & \sum_{i=1}^n p_i x_i \leq C, \\ & \forall i \in \llbracket 1, n \rrbracket, x_i \in \mathbb{N} \end{aligned}$$

Ici le vecteur $(x_1, \dots, x_n) \in \{0, 1\}^n$ représentera les objets que l'on prend. En effet, $x_i = 0$ si le commerçant ne prend pas l'objet i et 1 sinon.

Nous avons donc affaire à un programme linéaire. Or sa résolution pourrait non conduire à une solution comportant des valeurs $x_i \notin \{0, 1\}^n$ alors que l'on considère que l'on ne peut pas prendre une partie d'un objet (ex : 3/4 de l'objet i). Nous devons donc chercher une nouvelle méthode pour résoudre ce genre de programmes.

2.2.1 Définition et relaxation

Définition (Programme linéaire en nombres entiers) . Un *programme linéaire en nombres entiers* est un programme linéaire comme défini précédemment où un certain nombre de variables sont à valeurs entières. Plus formellement, la forme standard d'un tel programme est de la forme :

$$\begin{aligned} & \text{maximize} && \sum_{j=1}^n c_j x_j \\ & \text{subject to} && \sum_{j=1}^n a_{ij} x_j = b_i \quad \forall i \in \llbracket 1, m \rrbracket, \\ & && x_j \geq 0 \quad \forall j \in \llbracket 1, n \rrbracket, \\ & && x_j \in \mathbb{N} \quad \forall j \in \llbracket 1, n \rrbracket \end{aligned}$$

où les constantes c_j, a_{ij} et b_i sont entières.

Définition (Relaxation linéaire) . Soit un programme linéaire en nombres entiers (PLNE) :

$$\begin{aligned} & \text{maximize} && \sum_{j=1}^n c_j x_j \\ & \text{subject to} && \sum_{j=1}^n a_{ij} x_j = b_i \quad \forall i \in \llbracket 1, m \rrbracket, \\ & && x_j \in \mathbb{N} \quad \forall j \in \llbracket 1, n \rrbracket \end{aligned}$$

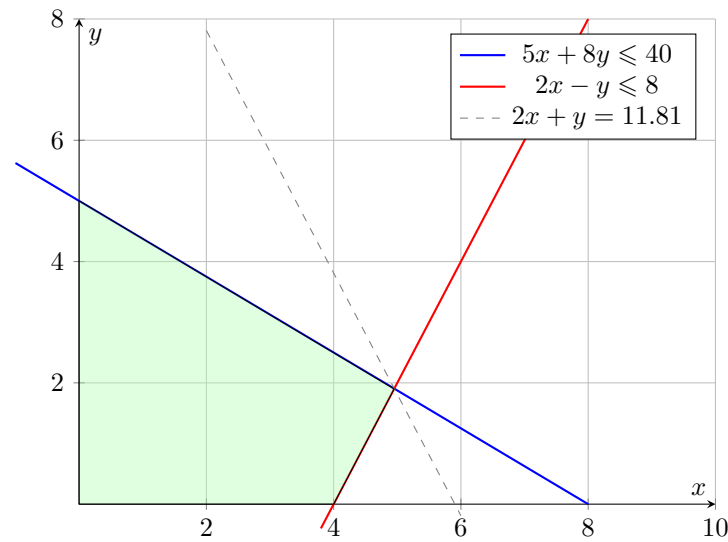
La *relaxation linéaire* de ce programme consiste à supprimer les contraintes de la forme :

$$x_j \in \mathbb{N} \quad \forall j \in \llbracket 1, n \rrbracket$$

On obtient ainsi un programme linéaire classique, plus facile à résoudre.

Exemple Considérons le programme linéaire suivant que nous allons résoudre graphiquement :

$$\begin{aligned} & \text{maximize} && z = 2x + 3y \\ & \text{subject to} && 5x + 8y \leq 40, \\ & && 2x - y \leq 8, \\ & && x, y \geq 0 \end{aligned}$$



Graphiquement, nous trouvons une solution optimale de valeur 11.81.

2.2.2 Algorithme "Branch and Bound"

Théorème (Lien entre PLNE et relaxation) . Soit un programme linéaire en nombres entiers (PLNE) et sa relaxation linéaire correspondante. Notons z_{int}^* la valeur optimale du PLNE et z_{relax}^* celle de la relaxation. Alors :

$$z_{\text{relax}}^* \geq z_{\text{int}}^* \quad (\text{en cas de maximisation}), \quad z_{\text{relax}}^* \leq z_{\text{int}}^* \quad (\text{en cas de minimisation}).$$

Autrement dit, la solution optimale de la relaxation fournit une *borne supérieure* (ou *borne inférieure*) pour le PLNE initial.

La méthode *Branch and Bound* est une technique de résolution des programmes linéaires en nombres entiers (PLNE) qui repose sur l'utilisation des relaxations linéaires pour obtenir des bornes sur la solution optimale. Cet algorithme construit un arbre de recherche binaire dont la racine est le problème initial Π . L'algorithme s'arrête lorsque tous les nœuds feuilles de l'arbre sont *fathomés* (c'est-à-dire qu'ils n'ont plus besoin d'être explorés).

Définition (Nœud fathomé) . Un nœud est dit *fathomé* si au moins une des conditions suivantes est vérifiée :

1. La relaxation linéaire est *infaisable*, c'est-à-dire qu'aucune solution réelle ne satisfait les contraintes.
2. La valeur optimale de la relaxation linéaire est inférieure ou égale à la meilleure solution entière connue.
3. La solution optimale de la relaxation linéaire est entière, et est donc également solution optimale du programme entier.

Algorithm 7 Branch and Bound pour un programme linéaire en nombres entiers

- 1: Résoudre la relaxation linéaire du problème Π .
- 2: **if** le nœud est fathomé **then**
- 3: arrêter l'exploration de ce nœud.
- 4: **else**
- 5: Identifier une variable x_i fractionnaire dans la solution x^* .
- 6: Créer deux sous-problèmes en ajoutant respectivement les contraintes :

$$x_i \leq \lfloor x_i^* \rfloor \quad \text{et} \quad x_i \geq \lceil x_i^* \rceil$$

- 7: Appeler **Branch and Bound** récursivement sur ces sous-problèmes.
 - 8: **end if**
-

Remarque L'algorithme explore donc l'arbre de recherche en combinant deux techniques :

- Le *branching*, qui divise le problème en sous-problèmes plus simples.
- Le *bounding*, qui utilise la relaxation linéaire pour élaguer les sous-problèmes ne pouvant pas améliorer la meilleure solution entière connue.

Compilation

Chapitre 1

Analyse Syntaxique

1.1 Analyse Syntaxique et approche descendante (Grammaires $LL(k)$)

Lors de la dérivation d'un mot par une grammaire, on peut se heurter à un problème de taille : quel règle de dérivation choisir pour obtenir un arbre de dérivation acceptant ? Nous allons donc introduire un type de grammaire nous permettant de régler ce problème. Un seul type de dérivation sera possible et nous saurons exactement combien de caractères regarder dans notre mot pour choisir la règle de dérivation à appliquer.

Les grammaires $LL(k)$ vont nous permettre de faire directement le lien avec le processus de *parsing*, indispensable en compilation.

1.1.1 Définition

Définition (Grammaire $LL(k)$) . Soit $k \in \mathbb{N}$ et G une grammaire. On dit que G est $LL(k)$ si on peut analyser n'importe quel mot de son langage de gauche à droite, en construisant la dérivation la plus à gauche, et en regardant au plus k symboles d'entrée à la fois, pour décider de manière univoque quelle production appliquer à chaque étape.

Remarque Détaillons un peu cette définition. La grammaire G est $LL(k)$ si elle satisfait trois critères principaux :

- **L (Left to right)** : Lorsque l'on dérive un mot par la grammaire, on veut pouvoir le lire de gauche à droite en commençant par le premier symbole.
- **L (Leftmost derivation)** : Lorsque l'on remplace une variable non terminale de l'expression en cours de dérivation, on veut que ce soit celle la plus à gauche.
- **k (k lookahead)** : k est le nombre de symbole que le parser peut regarder pour déterminer quelle règle utiliser sans les consommer.

Intuitivement, on cherche une grammaire prédictive. Pour chaque variable à chaque étape de la dérivation, le parser doit savoir exactement quelle règle appliquer en regardant k lettres. Évidemment, on ne considèrera pas de grammaire ambiguë. Nous allons donc développer un algorithme pour construire une *table de dérivation* qui nous donnera la règle à appliquer à chaque étape de l'analyse syntaxique.

1.1.2 Ensemble First, Follow, k -lookahead

Définition (k - Lookahead) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$ et une règle de production $A \rightarrow \alpha$. On définit l'ensemble $LA(k, A \rightarrow \alpha)$ comme l'ensemble des chaînes de longueur $\leq k$ qui déclenchent la règle de production $A \rightarrow \alpha$ tel que :

$$LA(k, A \rightarrow \alpha) := \{w \in \Sigma^{\leq k} \mid w \text{ peut apparaître si on choisit la règle } A \rightarrow \alpha\}$$

$LA(k, A \rightarrow \alpha)$ est donc l'ensemble des lettres que doit regarder le parser pour déterminer quelle règle de production appliquer. Son calcul permet aussi de vérifier qu'une grammaire est $LL(k)$. En effet si les k -lookahead des règles d'un même non terminal ont des mots en communs, la grammaire n'est pas $LL(k)$ (i.e on ne saura pas choisir quelle règle appliquer).

Définition (Ensemble First) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$. Soit $X \in V$ un état *non terminal*. On définit l'ensemble $First(X)$ comme l'ensemble des *éléments terminaux* de Σ qui peuvent apparaître en premier lors d'une dérivation de X . De plus, si X peut produire un ε , alors $\varepsilon \in First(X)$. Plus formellement :

$$First_k(\alpha) := \left\{ u \in \Sigma^*, \begin{cases} \text{si } |u| < k \text{ et } \alpha \rightarrow^* u \\ \text{si } |u| = k \text{ et } \alpha \rightarrow^* u\beta \text{ avec } \beta \in (V \cup \Sigma)^* \end{cases} \right\}$$

Cet ensemble permet de déterminer l'ensemble des mots de longueur inférieure ou égale à k qui peuvent dériver du mot auquel on l'applique. En général on commence le calcul à partir d'un non-terminal.

Propriété (Règles de la calcul (First)) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$, on a les règles de calcul suivantes :

- Soit $a \in \Sigma$ un terminal, on a : $First(a) = \{a\}$
- $First(\varepsilon) = \{\varepsilon\}$
- Soit $A \in V$ une variable avec les productions : $A \rightarrow \alpha_1 \mid \dots \mid \alpha_n$ on a :

$$First(A) = \bigcup_{i \in [1, n]} First(\alpha_i)$$

Exemple (Ensemble First) Soit G la grammaire aux règles de production suivantes :

$$\begin{cases} S' \rightarrow S\$ \\ S \rightarrow aAe \mid bc \mid \varepsilon \\ A \rightarrow d \mid \varepsilon \end{cases}$$

On a :

$$First_3(S) = First_3(aAe) \cup First_3(bc) \cap First_3(\varepsilon)$$

Or :

$$First_3(aAe) = a \cdot First_2(A) \cdot e = a\{d, \varepsilon\}e = \{ade, ae\}$$

D'où :

$$First_3(S) = \{ade, ae, bd, \varepsilon\}$$

Définition (Ensemble Follow) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$. Soit $X \in V$ un état *non terminal* et $k \in \mathbb{N}$. On définit $Follow_k(A)$ comme l'ensemble des mots de longueur exactement k qui peuvent suivre le non terminal A .

Propriété (Règle de calcul (Follow)) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$, on a alors :

$$Follow_k(A) = \bigcup_{Y \rightarrow \gamma A \delta} First_k(\delta Follow_k(Y))$$

Si $\varepsilon \in First_k(\delta)$, on ajoute aussi $Follow_k(Y)$.

Exemple (Ensemble Follow) Soit G la grammaire aux règles de production suivantes :

$$\begin{cases} S' \rightarrow S\$ \\ S \rightarrow aA|bABc \\ A \rightarrow d \\ B \rightarrow e|\varepsilon \end{cases}$$

On remarque que :

- $First_3(Bc) = First_3(B) \cdot c = \{ec, c\}$
- $Follow_3(S) = First_3(Follow_3(\$^3)) = \{\$^3\}$

$$\begin{aligned} Follow_3(A) &= First_3(Follow_3(S)) \cup First_3(BcFollow_3(S)) \\ &= \{\$^3\} \cup First_3(Bc \cdot Follow_3(S)) \\ &= \{\$^3\} \cup \{ec \cdot First_1(Follow_1(S))\} \cup \{c \cdot First_2(Follow_2(S))\} \\ &= \{\$^3\} \cup \{ec\$ \} \cup \{c\$^2\} \\ &= \{\$^3, ec\$, c\$^2\} \end{aligned}$$

Exemple (Follow avec récursivité) Soit G la grammaire aux règles de production suivantes :

$$\begin{cases} S' \rightarrow S\$^3 \\ S \rightarrow aA \\ A \rightarrow bAc|\varepsilon \end{cases}$$

On a ici :

$$\begin{aligned} Follow_3(A) &= First_3(Follow_3(S)) \cup First_3(Follow_3(A)) \\ &= First_3(\$^3 \cdot Follow_3(S')) \cup c \cdot First_2(Follow_2(A)) \\ &= \{\$^3\} \cup \{\$^2\} \cup c \cdot First_1(Follow_1(A)) \\ &= \{\$^3, c\$^2, cc\$, ccc\} \end{aligned}$$

1.1.3 Règles de calcul et table d'analyse

Théorème (Calcul de $LA(k)$) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$ et une règle de production $A \rightarrow \alpha$. On a :

$$LA(k, A \rightarrow \alpha) = First_k(\alpha) \cup \{x \in Follow_k(A) | \alpha \text{ peut dériver } \varepsilon\}$$

plus précisément :

- Si α ne peut pas dériver ε : $LA(k, A \rightarrow \alpha) = First_a(\alpha)$
- Si α peut dériver ε : $LA(k, A \rightarrow \alpha) = First_a(\alpha) \cup Follow_k(A)$

Proposition On peut réduire le théorème suivant à cette simple formule :

$$LA(k, A \rightarrow \alpha) = First_k(\alpha Follow_k(A))$$

L'ensemble de ces règles de calcul vont nous permettre de définir quelle règle de dérivation appliquer à chaque étape de l'analyse syntaxique d'un mot. Nous regrouperons toutes ces règles dans une table appelée la *table d'analyse*.

Définition (Table d'analyse) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$. Pour chaque règle de production $A \rightarrow \alpha$ et chaque $w \in LA(k, A \rightarrow \alpha)$, on définit :

$$M[A, w] = \alpha$$

Autrement dit, $M[A, w]$ contient la partie droite de la règle de production à utiliser quand le non-terminal courant est A et que les k prochains symboles de l'entrée forment w .

La table d'analyse d'une grammaire $LL(k)$ permet, pour chaque non-terminal A et chaque séquence de k symboles de lookahead w , de déterminer quelle production appliquer. Pour construire la table d'analyse d'une grammaire $LL(k)$, nous devons calculer les k -lookahead pour chaque règle de la grammaire.

Exemple Soit G la grammaire suivante :

$$\begin{cases} S' \rightarrow S\$^3 \\ S \rightarrow aAb \\ A \rightarrow c|\varepsilon \end{cases}$$

Calculons sa table d'analyse. Pour cela, commençons par le calcul des 3-lookahead :

$$\begin{aligned} LA(3, S' \rightarrow S\$^3) &= First_3(S\$^2 \cdots Follow_3(S)) \\ &= First_3(S\$^3) \\ &= First_3(acb\$^3) \cup First_3(ab\$^3) \\ &= \{acb, ab\$ \} \end{aligned}$$

$$\begin{aligned} LA(3, S \rightarrow aAb) &= First_3(aAb \cdot Follow_3(S)) \\ &= First_3(aAb \cdots First_3(\$^3)) \\ &= First_3(aAb\$^3) \\ &= First_3(acb\$^3) \cup First_3(ab\$^3) \\ &= \{acb, ab\$ \} \end{aligned}$$

$$\begin{aligned} LA(3, A \rightarrow c) &= First_3(c \cdot Follow_3(A)) \\ &= First_3(c \cdot First_3(b \cdot Follow_3(S))) \\ &= First_3(c \cdot First_3(c\$^3)) \\ &= First_3(c \cdot \$^2) = \{cb\$ \} \end{aligned}$$

$$\begin{aligned} LA(3, A \rightarrow \varepsilon) &= First_3(Follow_3(A)) \\ &= First_3(b\$^2) = \{b\$^2 \} \end{aligned}$$

On peut donc construire la table d'analyse de la grammaire :

Non-terminal	acb	$ab\$$	$cb\$$	$b\2
S'	$S' \rightarrow S\3	$S' \rightarrow S\3		
S	$S \rightarrow aAb$	$S \rightarrow aAb$		
A			$A \rightarrow c$	$A \rightarrow \epsilon$

Chaque cellule indique la production à appliquer si le flux d'entrée commence par la chaîne correspondante de longueur 3.

Ces nouveaux modèles de grammaires, quoique puissants ne permettent pas d'effectuer l'analyse syntaxique de n'importe quel langage. En effet, toutes les grammaires ne sont pas $LL(k)$ nous devrions donc modifier celles qui ne le sont pas pour effectuer l'analyse.

1.2 Analyse Syntaxique et approche ascendante (Grammaires $LR(k)$)

1.2.1 Principe

Une autre approche de l'analyse syntaxique grâce aux grammaires et d'utiliser l'approche *descendante*. En effet, depuis le début nous cherchons à dériver un mot à partir de l'axiome de départ en descendant vers des états terminaux. Nous allons ici chercher à faire le contraire : à partir d'un mot nous chercherons à le réduire vers l'axiome. C'est ce que l'on appelle l'approche *ascendante*, aussi appelée *analyse par décalage/réduction* (*shift-reduce*).

Exemple Commençons par étudier un exemple pour bien comprendre le principe de ces grammaires. Soit la grammaire G_1 suivante :

$$\begin{cases} E \rightarrow E + E \\ E \rightarrow E * E \\ E \rightarrow (E) \\ E \rightarrow id \end{cases}$$

Soit $w \in \Sigma^*$ tel que $w := id + id * id$. Nous cherchons à réduire w grâce aux règles de productions de G_1 jusqu'à l'axiome S . Résumons tout ça dans un tableau :

Proto-phrase de droite	Poignée (handle)	Production de réduction
$id + id * id$	id	$E \rightarrow id$
$E + id * id$	id	$E \rightarrow id$
$E + E * id$	id	$E \rightarrow id$
$E + E * E$	$E * E$	$E \rightarrow E * E$
$E + E$	$E + E$	$E \rightarrow E + E$
E		

1.2.2 Outils Fondamentaux

Définissons maintenant quelques outils pour effectuer rigoureusement cette analyse ascendante.

Définition (proto-mot) . Soit $G = (V, \Sigma, S, P)$ une grammaire. Un *proto-mot* (ou *chaîne sententielle*) est toute chaîne de symboles, terminaux et/ou non terminaux, que l'on peut obtenir en partant de l'axiome et en effectuant un nombre quelconque de dérivations. For-

mellement, pour la grammaire G , on a :

$$\alpha \in (V \cup \Sigma)^* \text{ est un proto-mot si } S \rightarrow^* \alpha$$

Un proto-mot est simplement une "étape intermédiaire" de la dérivation d'un mot. Dans l'ensemble précédent, cela correspond à $E + E * id$. Il est composé du terminal id et du non terminal E .

Définition (Poignée (Handle)) . Soient $G = (V, \Sigma, S, P)$ une grammaire et w un proto-mot de G . Une *poignée* de w est occurrence précise du corps d'une production $A \rightarrow \alpha$ dans w telle que :

$$S \rightarrow^* \gamma A \delta \rightarrow \gamma \alpha \delta = w$$

Une poignée correspond donc à une sous-chaîne à réduire dans notre mot w à un instant donné de l'analyse ascendante. Dans notre exemple précédent, pour le proto-mot $E + E * id$, une poignée peut correspondre au non terminal id qui se réduit en E en une étape grâce à la règle $E \rightarrow id$.

Définition (Préfixe Viable) . Soient $G = (V, \Sigma, S, P)$ une grammaire et w un proto-mot de G . Une *préfixe viable* est toute chaîne de symbole $\alpha \in V^*$ telle que :

$$\exists \beta \gamma \text{ tels que } S' \rightarrow^* \alpha \beta \gamma \text{ et } \beta \gamma \text{ contient une poignée complète}$$

où $S' \rightarrow S$ est l'axiome augmenté.

Un préfixe viable est exactement ce que l'on veut avoir au dessus de la pile d'un analyseur LR si l'on ne veut pas de retrouver dans une impasse :

- Si la pile contient un préfixe viable, on est dans une configuration valide et l'analyse peut continuer.
- Si la pile contient quelque chose qui n'est pas un préfixe viable, alors l'analyseur détecte une erreur syntaxique.

Définition (Configuration Valide) . Soient $G = (V, \Sigma, S, P)$ une grammaire. Une configuration d'un analyseur ascendant se décrit par :

- Le *contenu de la pile*
- Le *reste de l'entrée* non consommé.

Une configuration est dite *valide* si la pile contient un *préfixe viable* qui peut mener à une dérivation complète de l'axiome.

Une configuration valide est une sorte d'instantané de l'analyse où l'on n'a pas fait d'erreur de parsing.

1.2.3 Analyseur LR(k)

Cette ensemble de définitions nous permet de définir formellement les grammaires $LR(k)$ de la façon suivante.

Définition (Grammaire $LR(k)$) . Soit $G = (V, \Sigma, P, S)$ une grammaire, où :

- V est l'ensemble des symboles (terminaux et non-terminaux),
- $\Sigma \subseteq V$ est l'ensemble des terminaux,
- P est l'ensemble des productions de la forme $A \rightarrow \alpha$,
- $S \in V \setminus \Sigma$ est l'axiome.

On dit que G est une *grammaire LR(k)* si et seulement si, pour toute dérivation à droite :

$$S \Rightarrow^* \gamma A \omega \Rightarrow \gamma \beta \omega$$

avec $A \rightarrow \beta \in P$, $\gamma, \omega \in V^*$, il existe un *unique* choix de production $A \rightarrow \beta$ qui peut être appliqué pour réduire la *poignée* β , en ne regardant que :

- le *préfixe viable* $\gamma\beta$ (ce qui est déjà sur la pile),
- et au plus k symboles de lookahead dans ω .

En d'autres termes, la suite β est **une manche unique et non ambiguë** que l'analyseur peut identifier avec seulement k symboles d'anticipation.

Remarque On parlera identiquement de *grammaire LR(k)* ou d'*analyseur LR(k)*. Le terme *LR(k)* signifie :

- **Left to right** : l'analyseur lit les mots de gauche à droite.
- **Rightmost derivation** : les motifs spécifiés par la grammaire sont recherchés par suffixes de la chaîne lue.
- k : correspond au nombre de lettres non consommées considérées pour le choix de la dérivation à choisir.

Définissons maintenant un automate capable de reconnaître de telles grammaires.

Définition (Automate LR (bottom-up)) . Un *automate LR* est un *automate à pile* qui reconnaît les préfixes viables d'une grammaire hors contexte $G = (V, \Sigma, S, P)$ où :

- la *pile* contient les symboles et états représentant un *préfixe viable*.
- l'entrée est la chaîne de symboles à analyser terminant par le caractère de fin \$.

L'objectif est de lire la chaîne de gauche à droite en réduisant progressivement toutes poignées dans le but d'obtenir l'axiome sur le sommet de la pile et d'avoir consommé toute l'entrée.

Formellement, l'automate peut être défini par :

$$M = (Q, \Sigma, \Gamma, ACTION, GOTO, q_0, Z_0, F)$$

où :

- Q est l'ensemble des états de l'automate.
- Σ représente l'alphabet terminal (lettres).
- $\Gamma = \Sigma \cup V_N$ est l'ensemble des symboles de la pile.
- q_0 est l'état initial.
- Z_0 est le symbole initial de la pile.
- F est l'ensemble des états acceptants.

On définit en plus deux tables qui définissent la dynamique de l'analyseur :

- La table *ACTION* qui détermine quelle action effectuer à la lecture d'une lettre :

$$ACTION : Q \times (\Sigma \cup \{\$, \}) \longrightarrow \{shift(q), reduce(A \rightarrow \beta), accept, error\}$$

Elle gère les transitions sur les terminaux.

- La table *GOTO* qui détermine l'état dans lequel doit se trouver l'automate après une réduction :

$$\boxed{GOTO : Q \times V \longrightarrow Q}$$

Elle gère les transitions sur les non terminaux.

Définition (Shift et Reduce) . Comme dit précédemment, nous n'avons que deux opérations à effectuer lors de la lecture d'un mot par une grammaire $LR(k)$ (ou un automate LR) : *shift* ou *reduce*. Définissons les proprement :

1. L'opération *shift* consiste à déplacer le prochain symbole à lire sur la pile et aller à l'état correspondant. On avance donc dans la lecture mais sans faire de réduction.
2. L'opération *reduce* consiste, lorsque le sommet correspond à une poignée β pour une règle de production $A \rightarrow \beta$, de :
 - Dépiler $|\beta|$ symboles et états
 - Empiler le non-terminal A
 - Aller dans l'état déterminé par la table *GOTO*
3. *Accept* : si la pile est seulement constituée de S' et que l'entrée est \$, alors le mot est reconnu.
4. *Error* : si aucune opération shift/reduce n'est possible alors la configuration est invalide et le mot n'est pas reconnu.

Nous savons maintenant analyser un mot par un automate mais il reste un problème : comment définir les états de l'automate ? Pour cela, nous allons avoir besoin de définir le concept de *fermeture* ou *clôture*.

Définition (Fermeture) . Soit $G' = \{V, \Sigma, P, S'\}$ une grammaire augmentée. Un *item* $LR(0)$ est une règle de la forme :

$$A \rightarrow \alpha \cdot \beta$$

où "." indique la position de la l'analyse dans la production $A \rightarrow \alpha\beta$. La *fermeture* d'un ensemble d'items I , notée $cl(I)$ est définie récursivement ainsi :

1. $\forall \text{ item} \in I, \text{ item} \in cl(I)$.
2. Pour chaque item $A \rightarrow \alpha \cdot B\beta \in cl(I)$, et pour chaque production $B \rightarrow \gamma \in P$, alors $B \rightarrow \cdot \mu \in cl(I)$.

Exemple (Calcul de fermeture) Soit G une grammaire dont les règles de production sont :

$$\begin{cases} S' \rightarrow S \\ S \rightarrow AA \\ A \rightarrow a \end{cases}$$

On pose :

$$I_0 := \{S' \rightarrow \cdots S\}$$

Calculons maintenant les fermetures :

- Le point est avant S donc on ajoute $S \rightarrow \cdot AA$ grâce à la règle $S \rightarrow AA$.
- Récursivement, le point est avant A donc on ajoute $A \rightarrow \cdot a$ grâce à la règle $A \rightarrow a$.
- Le point $a \in \Sigma$ est un terminal on s'arrête donc ici.

On a donc :

$$cl(I_0) = \{S' \rightarrow \cdot S, S \rightarrow \cdot AA, A \rightarrow \cdot a\}$$

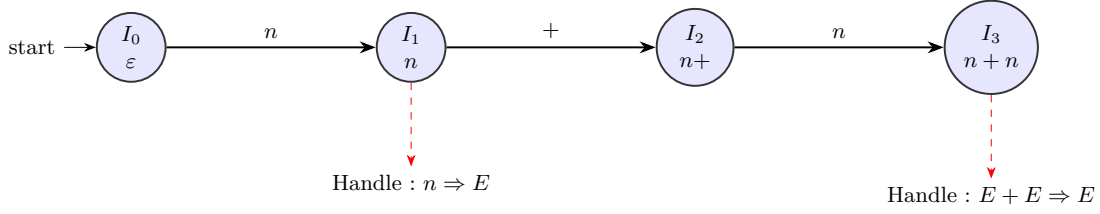


FIGURE 1.1 – Préfixes viables et leur correspondance avec les états de l'automate LR. Chaque état I_i représente un ensemble de *préfixes viables* atteignables. Les flèches indiquent les transitions de lecture (shift). Les lignes rouges en pointillés indiquent les poignées que l'on peut réduire à partir de cet état.

Exemple Travaillons autour d'un exemple pour bien saisir toutes ces nouvelles notions. Soit G une grammaire dont les règles de production sont :

$$\begin{cases} S' \rightarrow S \\ S \rightarrow AA \\ A \rightarrow a \end{cases}$$

On a ajouté un symbole S' fictif pour avoir un axiome de départ unique. Comme vu précédemment, commençons par calculer les clôtures :

- On calcule la clôture de I_0 qui donne :

$$cl(I_0) = \{S' \rightarrow \cdot S, S \rightarrow \cdot AA, A \rightarrow \cdot a\}$$

- Après la lecture de S nous devons passer dans un nouvel état I_1 d'où :

$$cl(I_1) = \{S' \rightarrow S \cdot\}$$

- Après la lecture de A dans I_0 on obtient l'ensemble I_2 où :

$$cl(I_2) = \{S \rightarrow A \cdot A, A \rightarrow \cdot a\}$$

- Après la lecture de A depuis I_2 , on obtient I_3 :

$$cl(I_3) = \{S \rightarrow AA \cdot\}$$

- Pour finir, après la lecture de a depuis I_0 ou I_2 , on a :

$$cl(I_4) = \{A \rightarrow \cdot a\}$$

On a donc les transitions suivantes :

$$\text{GOTO}(I_0, S) = I_1, \quad \text{GOTO}(I_0, A) = I_2, \quad \text{GOTO}(I_0, a) = I_5$$

$$\text{GOTO}(I_2, A) = I_4, \quad \text{GOTO}(I_2, a) = I_4$$

On peut donc construire la table action (transitions sur les terminaux) :

État	a	$\$$
I_0	shift I_4	error
I_1	error	accept
I_2	shift I_4	error
I_3	reduce $S \rightarrow AA$	reduce $S \rightarrow AA$
I_4	reduce $S \rightarrow A$	reduce $S \rightarrow A$

Et la table goto sur les non terminaux :

État	S	A
I_0	I_1	I_2
I_1	—	—
I_2	—	I_3
I_3	—	—
I_4	—	—

Résumons le traitement de l'entrée $w = a a \$$:

Pile (état/symbole)	Entrée	Action	Commentaire
I_0	a a \$	s I_4	Décalage sur a, empiler I_4
I_0 a I_4	a \$	r($A \rightarrow a$)	Réduction : dépiler a et I_4 , empiler A puis GOTO[I_0 ,A]= I_2
I_0 A I_2	a \$	s I_4	Décalage sur a, empiler I_4
I_0 A I_2 a I_4	\$	r($A \rightarrow a$)	Réduction : dépiler a et I_4 , empiler A puis GOTO[I_2 ,A]= I_3
I_0 A I_2 A I_3	\$	r($S \rightarrow AA$)	Réduction : dépiler deux A et états, empiler S puis GOTO[I_0 ,S]= I_1
I_0 S I_1	\$	accept	Mot reconnu, analyse terminée

